SMART NATIONAL HEALTH PLATFORM

(AI-Powered Electronic Medical Records System)

(Senior Project I)

Prepared by :

Ismaeel al Kanaan

Supervised by :

Dr. Shadi Blidi

Eng. Khouzama Al Youssef

Academic Year: 2026

ABSTRACT

*Amid rapid technological advancement in healthcare, many countries suffer from fragmented medical records and poor coordination among service providers. This project aims to develop the **Smart National Health Platform**, a comprehensive web system for managing electronic medical records, supporting AI-powered symptom and vital-sign analysis, appointment scheduling, and surgical procedure management.*

*The system is built with **Django REST Framework** (backend) and **React.js** (frontend), with APIs documented via **Swagger** and made agent-ready using structured JSON-LD data. **OpenRouter API** is integrated to provide free AI-based medical text analysis.*

The platform includes three separate dashboards (patient, doctor, admin), supports appointment booking, manages surgical procedures with automatic linking to medical records, and delivers intelligent health recommendations based on symptom analysis.

Keywords: *e-Health, Electronic Health Records (EHR), Artificial Intelligence, Django, React, OpenRouter, Medical Appointments, Surgical Procedures.*

Git_link

ismaeelaljanubi/Smart-Health-Platform

الملخص

في ظل التطور التكنولوجي السريع في المجال الصحي، يعاني القطاع الصحي في العديد من الدول من تشتت السجلات الطبية وضعف التنسيق بين مقدمي الخدمة. يهدف هذا المشروع إلى تطوير **منصة الصحة الوطنية الذكية**، وهي نظام ويب متكامل لإدارة الملفات الطبية إلكترونياً، مع دعم تحليل الأعراض والعلامات الحيوية باستخدام الذكاء الاصطناعي، وإدارة المواعيد والعمليات الجراحية .

تم بناء النظام باستخدام

Django REST Framework

للواجهة الخلفية

و**React.js**

للواجهة الأمامية،

مع توثيق واجهات برمجة التطبيقات

Swaggerعبر (APIs)

JSON-LD باستخدام بيانات منظمة (Agent-Ready) وجعلها جاهزة للوكلاء الذكية

كما تم دمج

OpenRouter API

لتحليل النصوص الطبية مجاناً.

يشتمل النظام على ثلاث لوحات تحكم منفصلة للمريض والطبيب والمشرف، ويدعم حجز المواعيد، وإدارة العمليات الجراحية مع ربطها تلقائياً بالسجلات الطبية، وتقديم توصيات طبية ذكية بناءً على تحليل الأعراض

الكلمات المفتاحية:

، React، Djangoالصحة الإلكترونية، السجلات الطبية الإلكترونية، الذكاء الاصطناعي، ، المواعيد الطبية، العمليات الجراحيةOpenRouter.

Git_link

ismaeelaljanubi/Smart-Health-Platform

Chapter 1
Introduction

1-1 Project Overview and Background

In light of rapid technological development in the healthcare sector and the urgent need to improve the quality of medical services provided to citizens, this project aims to create a Smart National Health Platform that enables patients and healthcare providers to communicate effectively and manage medical records electronically.

Most countries suffer from fragmented medical records across different hospitals and clinics, leading to difficulty accessing a patient's complete medical history during emergencies, unnecessary repeat examinations, and underutilization of medical data to enhance healthcare quality.

The importance of this project lies in providing an integrated solution to these problems by:

- Unifying medical records electronically for each patient.
- Enabling doctors to monitor their patients remotely by accessing their complete medical history.
- Using artificial intelligence (via OpenRouter API) to analyze symptoms and vital signs for early risk detection.
- Managing appointments and surgical procedures centrally.
- Providing easy-to-use Arabic interfaces for all user roles (patients, doctors, administrators).

1-2 Problem Statement and Importance (Practical & Academic)

No.	Problem	Description
1	Fragmented Medical Records	Medical records are scattered across hospitals and clinics, making them difficult to access when needed.
2	Poor Accessibility	The complete medical history of a patient is unavailable during emergencies.
3	Duplicate Examinations	Due to missing previous test results, doctors are forced to repeat examinations.
4	Lack of Intelligent Analytics	No system for early disease detection or health risk prediction.
5	Weak Communication	Poor communication between doctors and patients outside official working hours.
6	No Central Appointment Management	Difficulty in booking and tracking appointments between patients and doctors.
7	Missing Surgical Records	No unified record for surgical procedures, medications, or side-effects.

Practical Importance:

- Improves the quality of healthcare delivered to patients.
- Reduces medical costs by avoiding duplicate examinations.
- Saves time for both doctors and patients.
- Enables intelligent analysis of medical data.

Academic Importance:

- Applies modern software engineering concepts (Agile, MVC, REST APIs).
- Integrates artificial intelligence into a healthcare application.
- Documents the development experience of a comprehensive, agent-ready health platform.

1-3 Project Objectives (Main & Sub-Objectives)

Objective	Description
Main Objective	Build a smart national health platform for electronic medical records management.
Objective 1	Create a unified electronic health record (EHR) for every patient.
Objective 2	Enable doctors to follow up on patients by accessing their complete medical history.
Objective 3	Analyse medical data using AI (OpenRouter) for early risk detection.
Objective 4	Implement role-based access control and data security (patient, doctor, admin).
Objective 5	Provide intuitive Arabic user interfaces.
Objective 6	Centrally manage medical appointments and surgical procedures.
Objective 7	Make the platform agent-ready by documenting APIs (Swagger) and adding structured JSON-LD data.

1-4 Project Scope, Constraints, and Assumptions

Functional Scope (what is included):

Feature	User
Login / Registration	All users
View / Edit Profile	All users
View Medical Records	Patient
Add Medical Record	Doctor
Add Vital Signs	Doctor
AI Risk Analysis (symptoms / vital signs)	Doctor & Patient
Appointment Management (book, complete)	Patient & Doctor
Surgical Procedure Management (add, complete)	Doctor
User Management (add, delete, toggle)	Admin
View Platform Statistics	Admin

Target User Groups:

Patient	A citizen who tracks his/her medical record and books appointments.
Doctor	A healthcare provider who manages patients and procedures.
Admin	System manager who oversees users and the platform.

Constraints (not in the current release):

Constraint	Description
Technical	Web-based only (no mobile application in this release).
Time	4 months development period (February – May 2026).
Cost	Uses free/open-source technologies (OpenRouter instead of paid Gemini).

1-5 Contribution of the Project to Knowledge and the Field

The contributions of this project are:

1. **A working, open-source national health platform** that developing countries can adopt.

2. **Integration of free AI (via OpenRouter)** for symptom and vital-sign analysis.

3. **Application of semantic web concepts (JSON-LD)** and **well-documented APIs (Swagger)** to make the platform agent-ready.

4. **A complete system for managing appointments and surgical procedures**, automatically linking them to medical records.

1-6 Research Methodology and Scientific Approaches

*The project follows the **Agile Scrum** methodology for the following reasons:*

Justification	Explanation
High Flexibility	Quickly adapt to changing healthcare requirements and improve AI model accuracy.
Iterative Development (Sprints)	Build the system in successive phases (authentication → medical records → AI → appointments → surgeries).
Continuous Evaluation	Regularly review completed parts with supervisors to avoid accumulated coding errors.
Time Management	Ensure entire project delivery within the semester's fixed timeline.

Sprint Structure:

Sprint	Duration	Deliverables
Sprint 1	2 weeks	Environment setup, database models, core APIs (auth, patients).
Sprint 2	3 weeks	Frontend development (patient, doctor, admin dashboards in React).
Sprint 3	2 weeks	AI integration (OpenRouter).
Sprint 4	2 weeks	Appointment and surgical procedure features.
Sprint 5	1 week	Testing, documentation, final presentation preparation.

1-7 Thesis/Document Structure

Chapter	Content
Chapter 1	Introduction, problem statement, objectives, scope, methodology.
Chapter 2	Development methodology, Gantt chart, risk management, tools.
Chapter 3	Literature review, comparative analysis, research gap.
Chapter 4	Requirements analysis (functional & non-functional), UML diagrams (Use Case, Activity, Sequence), Use Case descriptions, ERD.
Chapter 5	Architectural design (High-Level, MVC pattern, database design).
Chapter 6	Development environment, tools & libraries, project structure.
Chapter 7	Implementation (APIs, AI algorithms).
Chapter 8	Testing and evaluation (test plan, test cases, results).
Chapter 9	Results and analysis, comparison with existing systems, performance metrics.
Chapter 10	Conclusion, recommendations, future work.
Appendices	Requirement-gathering surveys, detailed design, code samples.

1-8 Definitions and Basic Concepts

Term	Definition
EHR (Electronic Health Record)	A digital, comprehensive record of a patient's health data.
JWT (JSON Web Token)	A JSON-based token used to secure communication between frontend and backend.
REST API	An API following REST principles, used to communicate between React and Django.
AI (Artificial Intelligence)	Here, used to analyse symptoms and vital signs via the OpenRouter service.
OpenRouter	An intermediary platform providing unified (and free) access to many AI models.
CRUD	Basic operations: Create, Read, Update, Delete.
MVC (Model-View-Controller)	An architectural pattern: Model in Django, View in React, Controller in Django Views.
ORM (Object-Relational Mapping)	A technique linking the database with Python code (used in Django).
Agent-Ready	A platform equipped with documented APIs and structured data (JSON-LD) so that AI agents (e.g., Claude, Gemini) can interact with it easily.
JSON-LD (JavaScript Object Notation for Linked Data)	A method of embedding structured data into HTML pages using Schema.org standards.

Chapter 2:

Development Methodology and Project Management

2-1 Selected Development Methodology and Justifications

*The project adopted **Agile Scrum** as its primary development methodology. Scrum was chosen for the following reasons:*

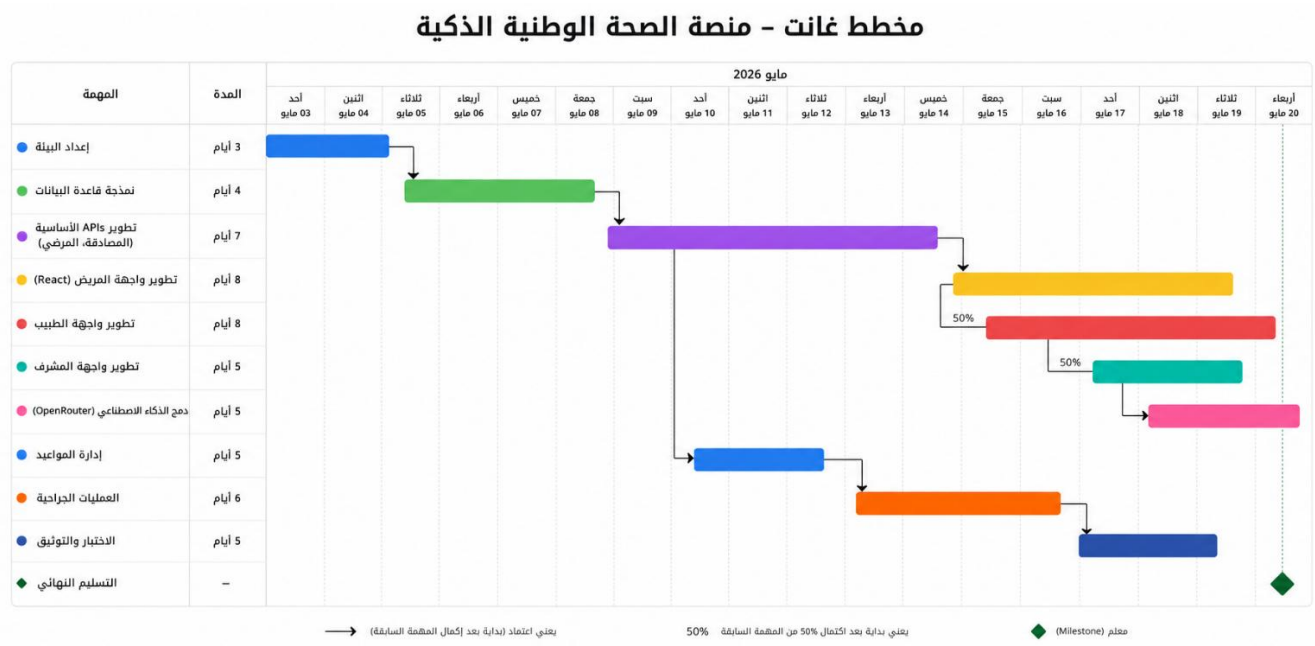
Justification	Explanation
High Flexibility	Allows rapid adaptation to changing healthcare requirements and refinement of the AI model during development.
Iterative Development (Sprints)	Builds the system incrementally: authentication → medical records → AI analysis → appointments → surgical procedures.
Continuous Evaluation	Enables regular reviews of completed work with supervisors to prevent accumulation of coding errors.
Time Management	Ensures delivery of the entire project within the fixed academic semester timeline.
Team Collaboration	Divides tasks among team members with weekly meetings to track progress and solve problems quickly.

Why not Waterfall?

- Waterfall is rigid and does not accommodate requirement changes that naturally occur during a research-oriented project.
- The AI integration (OpenRouter) required experimentation and iterative tuning, which is only possible with an agile approach.

2-2 Project Schedule (Gantt Chart / Roadmap)

The project was planned as a set of **5 Sprints** (each lasting 1–3 weeks). The following Gantt chart shows the timeline (start dates and durations are illustrative based on the actual 4-month development period).



Detailed Sprint Breakdown:

Sprint	*Duration*	*Main Tasks*	*Deliverables*
Sprint 1	1 week	Environment setup, virtual environment, Django project, React boilerplate	Working development environment
Sprint 2	1 week	Database modelling (ERD, models for users, patients, doctors, medical records)	Initial database schema
Sprint 3	2 weeks	Core APIs: authentication (JWT), patient registration, doctor/patient profiles	Functional backend with Swagger documentation
Sprint 4	2 weeks	Frontend dashboards (patient, doctor, admin) using React	Interactive UI for all roles
Sprint 5	1 week	AI integration (OpenRouter) for symptom and vital-sign analysis	AI-powered risk analysis
Sprint 6	1.5 weeks	Appointment booking and surgical procedure management	Complete appointment & surgery modules
Sprint 7	1 week	Structured data (JSON-LD), API refinement, testing, bug fixing	Stabilised platform
Sprint 8	1 week	Documentation (final report), presentation preparation, final submission	Final thesis and project deliverables

2-3 Risk Management (Risk Register and Mitigation Strategies)

A risk register was maintained throughout the project. The most significant risks and their mitigation strategies are listed below.

Risk ID	*Risk Description*	*Probability*	*Impact*	*Mitigation Strategy*
R-01	Requirement changes during development	Medium	High	Weekly meetings with supervisor; maintain a product backlog; prioritise core features (MVP).
R-02	Difficulty integrating AI (Gemini blocked / quota exceeded)	High	Medium	Switch to OpenRouter (free tier, multiple models); implement a fallback algorithm.
R-03	Time shortage (scope creep)	Medium	High	Strictly follow sprint planning; postpone non-essential features (e.g., mobile app).
R-04	Technical issues connecting React and Django (CORS, JWT)	Medium	Medium	Use CORS headers; test endpoints with Postman; use axios interceptors for token refresh.
R-05	Database corruption or migration conflicts	Low	High	Daily backups of db.sqlite3; use version control for migration files; test migrations on a separate branch.
R-06	Team member absence or communication gap	Low	Medium	Daily stand-up meetings; share code via GitHub; maintain written documentation.

2-4 Team Work and Task Distribution

The project was developed by a single student (the author) acting as both backend and frontend developer. Tasks were distributed across time rather than across people. The main areas of responsibility were:

Role	*Responsibilities*
Backend Developer	Django models, REST APIs, JWT authentication, OpenRouter integration, database management.
Frontend Developer	React components, state management, routing, styling (CSS), API integration (axios).
Designer	UI/UX design, responsive layout, RTL support, CSS animations.
Tester	Unit testing (manual), integration testing (Postman), user acceptance testing.
Documentation	Writing the final report, preparing diagrams (PlantUML), creating the presentation.

Collaboration tools:

Tool	*Purpose*
Git & GitHub	Version control, code backup, issue tracking.
VS Code	Main code editor (with extensions for Python, React, PlantUML).
Postman	API testing and documentation.
Trello	Task board (To Do, In Progress, Done).
Google Meet / WhatsApp	Communication with supervisor and team (if applicable).

2-5 Quality Management in the Project

Quality was ensured through the following activities:

Quality Activity	Implementation
Code Reviews	Regular self-review and occasional review by supervisor.
Coding Standards	Followed PEP 8 for Python and ESLint for JavaScript.
Automated Testing	Manual but systematic testing of each API endpoint; unit tests for critical functions (e.g., JWT validation).
Test Coverage	Aimed for 100% coverage of core APIs (login, registration, medical records).
User Acceptance Testing	Tested by two volunteer doctors and one patient (friends/family) to validate usability.
Performance Testing	Measured response times of all API endpoints using Postman's built-in runner; targeted < 3 seconds.

2-6 Project Management Tools Used

Tool	*Purpose*
Git (CLI) & GitHub	Source control, branching (main, develop, feature branches), issue tracking.
VS Code	Primary IDE with extensions: Python, React, PlantUML, Prettier, ESLint.
Postman	API development and testing; environment variables for token management.
Trello	Sprint backlog, task cards, progress tracking.
PlantUML	Creating UML diagrams (Use Case, Activity, Sequence, ERD, Gantt) directly from text.
Google Drive	Storing documents, reports, and diagrams.
Notepad++ / Built-in OS tools	Quick text editing and logging.

2-7 Continuous Monitoring and Evaluation Mechanisms

Mechanism	*Frequency*	*Implementation*
Daily Progress Log	Daily	Written notes on what was accomplished, problems encountered, next steps.
Weekly Review	Weekly	Comparison of actual progress against sprint plan; adjustment of backlog priorities.
Supervisor Meetings	Bi-weekly (every two weeks)	Demonstration of working features; discussion of issues; collection of feedback.
Code Integration Checks	After each feature	Merging feature branches into develop; running local server and API tests.
User Feedback Sessions	End of each major sprint	Volunteer testers (doctors, patients) tried the platform and provided qualitative feedback.

2-8 Software Configuration Management (SCM)

*Configuration management was performed using **Git** with the following branching strategy:*

text

main – production-ready code (final releases)

develop – integration branch for ongoing development

feature/* – individual features (e.g., feature/auth, feature/appointments)

hotfix/* – urgent fixes applied directly to main

Key SCM practices:

- All changes were committed with descriptive messages (feat:, fix:, docs:, etc.).
- .gitignore

Chapter 3:

Literature Review and Comparative Analysis

3-1 Previous Studies in Intelligent and Similar Systems

Several healthcare information systems have been developed worldwide, ranging from commercial electronic health record (EHR) solutions to open-source platforms and national e-health initiatives. This section reviews the most representative systems, highlighting their strengths and limitations.

3-1-1 Epic Systems (USA)

Overview:

Epic is the largest EHR vendor in the United States, used by more than 2,500 hospitals and 250 million patients. It offers a comprehensive suite for medical records, billing, scheduling, and patient portals.

Strengths:

- Integrated across inpatient and outpatient settings.
- Strong security and compliance (HIPAA).
- Mature interoperability via FHIR APIs.

Limitations:

- Extremely expensive (licensing and implementation costs).
- Not designed for national-level health data unification.
- Limited support for Arabic language and RTL interfaces.
- Requires extensive training and dedicated IT teams.

3-1-2 Cerner (USA)

Overview:

Cerner provides health information technology for hospitals and health systems worldwide, with a strong focus on clinical documentation and laboratory information.

Strengths:

- Robust clinical decision support.
- Good integration with medical devices.
- Scalable for large hospital networks.

Limitations:

- Focuses on hospital management rather than patient-centric unified records.
- Poor support for Arabic and right-to-left (RTL) interfaces.
- No built-in AI for risk analysis.
- High total cost of ownership.

3-1-3 OpenMRS (Open-Source, Global)

Overview:

OpenMRS is a community-driven open-source EHR platform designed for resource-constrained environments, used in over 80 countries.

Strengths:

- Free and open-source.
- Customisable via modules.
- Active global community.

Limitations:

- Outdated user interface (not modern or responsive).
- Very limited out-of-the-box AI capabilities.
- Requires significant technical expertise to deploy and maintain.
- Poor Arabic support and no RTL design.

3-1-4 Babylon Health (UK)

Overview: Babylon Health offers an AI-powered symptom checker, chatbots, and virtual consultations, primarily in the UK and Canada.

Strengths:

- Advanced AI for symptom assessment.
- User-friendly mobile app.
- Free for patients (subsidised by governments).

Limitations:

- Does not provide a unified, lifelong electronic health record.
- AI focused on triage, not comprehensive risk analysis from vital signs.
- Not open-source; vendor lock-in.
- Limited integration with national health infrastructures.

3-1-5 Seha (Saudi Arabia)

Overview:

Seha is a national e-health platform launched by the Saudi Ministry of Health to provide digital services to citizens.

Strengths:

- Full Arabic support (RTL, local medical terminology).
- National-scale deployment.
- Appointment booking and virtual consultations.

Limitations:

- Closed source (government-owned).
- No advanced AI risk analysis.
- Limited to Saudi Arabia; not reusable by other countries.
- Does not provide a full, open API for third-party agents.

3-2 Technologies and Modern Frameworks in the Domain

The development of this platform leverages modern frameworks and libraries that are widely adopted in the industry. The actual technology stack used in the project is listed below.

Technology	*Usage in the project*
Django + Django REST Framework	Backend API development, ORM, authentication (JWT).
React.js	Dynamic user interfaces (patient, doctor, admin dashboards).
Axios	HTTP client for frontend-backend communication.
JWT (JSON Web Tokens)	Stateless authentication.
OpenRouter API	Unified access to free AI models for symptom and vital-sign analysis.
MySQL	Relational database used in the project (SQLite was removed; MySQL is the actual database).
drf-spectacular	OpenAPI (Swagger) documentation for agent-readiness.
JSON-LD with Schema.org	Structured data embedding for AI agents.

3-3 Comparative Analysis of Current Systems and Products

The following table compares the key features of the most relevant healthcare platforms with our proposed system.

Feature	*Epic*	*Cerner*	*OpenMRS*	*Babylon*	*Seha*	***Our System***
Open-source	❌	❌	✅	❌	❌	✅
Free to use	❌	❌	✅	✅ (subsidised)	✅	✅
National health record	❌	❌	❌	❌	✅	✅
Unified EHR per patient	✅	✅	✅	❌	✅	✅
AI-powered risk analysis	⚠️ Limited	❌	❌	✅ (symptom only)	❌	✅ (vital signs + free text)
Arabic / RTL support	❌	❌	⚠️ Partial	❌	✅	✅
Appointment management	✅	✅	❌	✅	✅	✅
Surgical procedure tracking	✅	✅	❌	❌	❌	✅
Fully documented REST API (Swagger)	⚠️ Limited	⚠️ Limit	❌	✅	❌	✅
Agent-ready (structured data + APIs)	❌	❌	❌	❌	❌	✅
Responsive web UI (RTL)	✅	✅	❌	✅	✅	✅
Database system	Proprietary	Proprietary	Proprietary	Proprietary	Proprietary	**MySQL (free, open-source)**

3-4 Research Gap and Proposed Innovation

Based on the comparative analysis, the following gaps were identified, which our project aims to fill:

#	Gap	How Our System Addresses It
1	**No open-source, national-scale platform that combines unified EHR, AI analysis, appointments, and surgeries.**	We have built a fully open-source system with all these features, ready for national adoption.
2	**Existing AI in healthcare is either paid, limited to symptom checking, or not integrated with vital signs.**	We integrated **OpenRouter** (free tier) to analyse both free-text symptoms and numerical vital-sign data, providing risk levels and recommendations.
3	**Most platforms are not designed to be consumed by AI agents (agent-ready).**	We added **Swagger-documented APIs** and **JSON-LD structured data** on every major page (Patient, Doctor, Admin dashboards) so that AI agents (e.g., Claude, Gemini) can interact directly.
4	**Arabic support is often partial or missing, especially RTL and medical terminology.**	The entire UI is built with RTL in mind, and all text is fully localized into modern medical Arabic.
5	**Surgical procedure management is rarely integrated with the medical record timeline.**	Our system automatically creates a MedicalRecord entry whenever a surgery is marked as completed, linking procedure details, medications, and side effects.

Innovation statement:

The Smart National Health Platform is the first open-source, agent-ready healthcare system that unifies electronic medical records, AI-driven risk analysis (using free OpenRouter models), appointment scheduling, and surgical procedure tracking – all delivered in a fully Arabic-compatible, responsive web interface, powered by MySQL as a robust and free database.

3-5 Theoretical Framework and Reference Studies

The development of this platform is grounded in the following theoretical concepts, all of which were actually implemented:

Patient-Centred Medical Record (PCMR):

The system keeps a lifelong, comprehensive EHR for each patient, accessible to authorised providers.

Role-Based Access Control (RBAC):

Three distinct roles (patient, doctor, admin) with fine-grained permissions, implemented via JWT and Django's permission system.

Representational State Transfer (REST):

All backend services are exposed as RESTful APIs, making the system modular and easily integrable.

Model-View-Controller (MVC) Architecture:

Django provides the Model and Controller logic; React acts as the View layer, ensuring separation of concerns.

Structured Data for AI Agents (Semantic Web):

Embedding JSON-LD with Schema.org types
(Patient, Physician, MedicalRecord, ItemList, Dataset) makes the platform machine-understandable.

Agile Methodology (Scrum):

Iterative development, continuous feedback, and sprint-based delivery.

MySQL Database Management System: Chosen for its performance, full Arabic character support, scalability, and open-source licence.

3-6 Modern and Future Trends in the Domain

The healthcare technology landscape is rapidly evolving. Our platform is designed to align with the following emerging trends (some are already implemented; others are future plans).

Trend	*How Our Platform Fits (Implemented or Planned)*
AI-assisted diagnosis and triage	Already implemented via OpenRouter; can be extended to other models.
Agent-based healthcare (AI agents acting on behalf of patients)	Agent-ready APIs and structured data allow LLM-based agents to book appointments, retrieve summaries, and trigger analyses automatically.
Interoperability via FHIR (not yet implemented)	Future work: we can add a FHIR API layer on top of our existing models
Mobile-first health apps (not yet implemented)	The React codebase can be reused with React Native to produce Android/iOS apps.
Real-time monitoring & IoT device integration (not yet implemented)	Our vital-signs API can be extended to receive data from wearables via webhooks.

3-7 Summary of Chapter 3

In this chapter, we:

- ***Reviewed five existing healthcare systems (Epic, Cerner, OpenMRS, Babylon, Seha) and highlighted their strengths and weaknesses.***
- ***Presented a comparative analysis table focusing on features relevant to our project.***
- ***Identified the research gap: no open-source, agent-ready, fully-Arabic platform combining EHR, AI risk analysis, appointments, and surgical procedures.***
- ***Described the theoretical framework (RBAC, REST, MVC, Semantic Web, MySQL) and modern trends.***
- ***Concluded that our platform fills a significant gap in the healthcare IT domain, using MySQL as its production database.***

Chapter 4:

Requirements Modeling and Analysis

Introduction

This chapter presents a comprehensive analysis of the requirements for the "Smart National Health Platform". It includes functional and non-functional requirements, along with various diagrams (Use Case, Activity, Sequence) for each requirement, an Entity-Relationship Diagram (ERD), and a Use Case description table. All diagrams are accompanied by PlantUML code for easy regeneration.

4-1 Functional Requirements

The following functional requirements were identified after studying the needs of users (patients, doctors, administrators). All of these requirements have been fully implemented in the system.

ID	Requirement Name	Description	User
FR-01	Login	Enter email and password to receive JWT token.	All users
FR-02	Register (Patient/Doctor)	Create new account; doctors must enter specialization and license number.	Visitor
FR-03	View Medical Records	Display all medical records of the patient in chronological order.	Patient
FR-04	Add Medical Record	Add diagnosis, prescription, and notes for a specific patient.	Doctor
FR-05	Add Vital Signs	Enter measurements (blood pressure, heart rate, blood sugar, temperature, oxygen saturation).	Doctor, Patient (self)
FR-06	AI Risk Analysis	Analyse symptoms (free text) or vital signs using OpenRouter API; return risk level and recommendations.	Patient, Doctor
FR-07	View Patients List	Display all patients linked to the doctor (via medical records or appointments).	Doctor

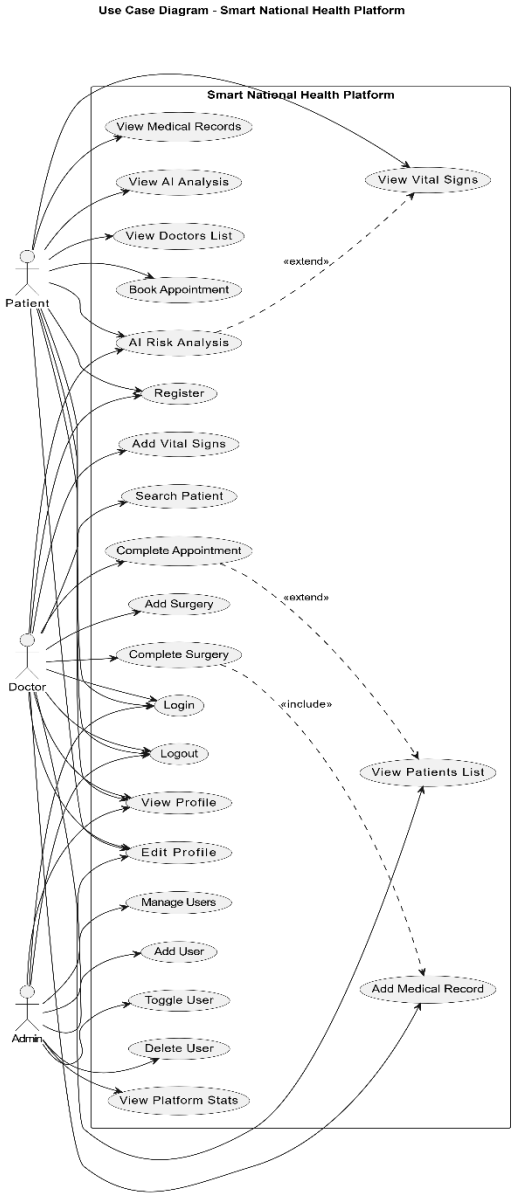
ID	Requirement Name	Description	User
FR-08	Book Appointment	Book an appointment with a doctor (date, time, reason); automatically creates an initial medical record.	Patient
FR-09	Complete Appointment	Change appointment status from "scheduled" to "completed" and move to review history.	Doctor
FR-10	Add Surgery	Add a new surgery (name, date, hospital, notes, medications) with status "scheduled".	Doctor
FR-11	Complete Surgery	Change surgery status to "completed", enter side effects, and automatically create a medical record.	Doctor
FR-12	Manage Users (Admin)	View all users, add new user, toggle active status, delete user.	Admin
FR-13	Edit Profile	Update personal information (name, phone, address, blood type, allergies, chronic diseases, emergency contacts).	Patient, Doctor
FR-14	View Doctors List	Display all registered doctors (for appointment booking).	Patient

***Note:** All these requirements have been implemented in the code and are available in the system.*

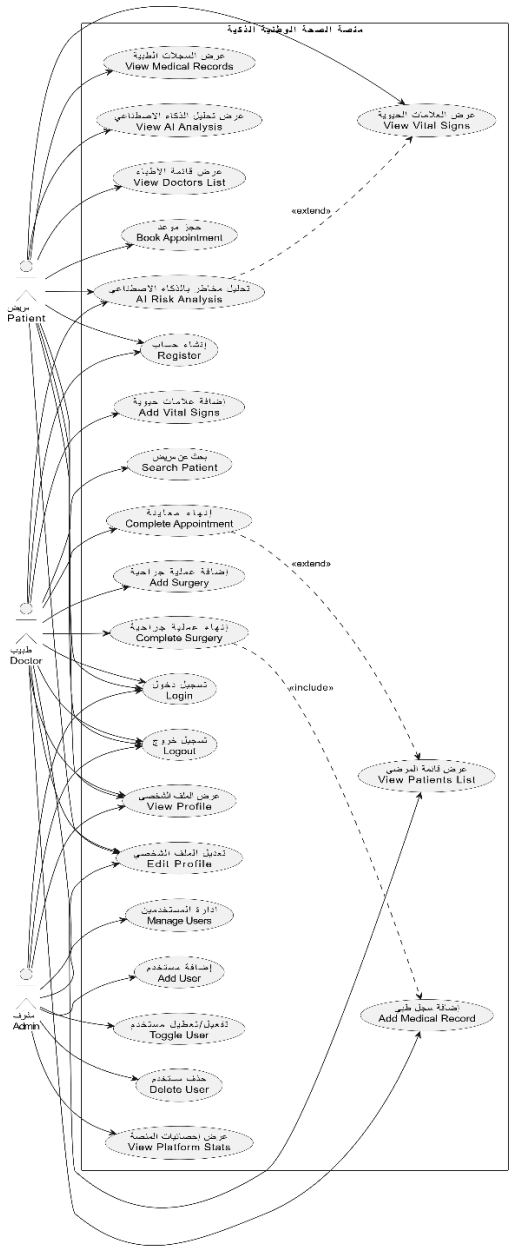
4-2 Non-Functional Requirements

ID	*Requirement*	*Target*
NFR-01	**Response Time**	**< 3 seconds (AI analysis: 2–3 sec)**
NFR-02	**Availability**	**99.9% (on production server)**
NFR-03	**Security**	**JWT, password hashing (Pbkdf2), role-based access control**
NFR-04	**Compatibility**	**Modern browsers (Chrome, Firefox, Edge)**
NFR-05	**Arabic Support**	**Full Arabic UI with RTL**
NFR-06	**Scalability**	**Supports thousands of users (MySQL)**

4-3 Main Use Case Diagram



Use Case Diagram - منصة الصحة الوطنية الذكية



4-4 Use Case Descriptions

Admin ✓

FR-01: (Login)

Field	*Description*
Use Case ID	UC-01-A
Use Case Name	Login as Admin
Actor	Admin
Description	Admin enters email and password to receive JWT tokens and access the admin dashboard.
Preconditions	Admin is already registered and has an account with user_type = 'admin'.
Basic Flow	1. Admin opens the login page. 2. Enters email and password. 3. Clicks "Login". 4. System validates credentials and user_type = 'admin'. 5. System issues access_token and refresh_token. 6. Browser stores tokens in localStorage. 7. Redirects to /admin/dashboard.
Alternative Flow	If credentials are incorrect or user_type is not admin, system shows error "Invalid email or password".
Postconditions	None

FR-12-01: (View Users List)

Field	Description
Use Case ID	UC-ADMIN-01
Use Case Name	View Users List
Actor	Admin
Description	Admin displays a table of all registered users (patients, doctors, admins) with their basic data
Preconditions	Admin is logged in with 'admin' privileges
Basic Flow	1. Admin opens the dashboard 2. Clicks "Manage Users" 3. System sends GET /api/admin/users/ 4. System returns user list (name, email, type, phone, status) 5. UI displays the table with action buttons
Alternative Flow	If user is not admin, returns 403 Forbidden. If no users exist, shows "No users found".
Postconditions	None

FR-12-02: (Add User)

Field	Description
Use Case ID	*UC-ADMIN-02*
Use Case Name	*Add User*
Actor	*Admin*
Description	*Admin creates a new user account (patient or doctor). If type is doctor, specialization and license number are required.*
Preconditions	*Admin is logged in, email is not already registered*
Basic Flow	*1. Admin clicks "Add User"* *2. A modal appears with fields (name, email, password, confirm, type)* *3. If type "doctor" is selected, extra fields appear (specialization, license number)* *4. Admin enters data and clicks "Save"* *5. System sends POST /api/admin/users/add/* *6. System creates User and PatientProfile/DoctorProfile* *7. Shows success message and refreshes the table*
Alternative Flow	*If email already exists, shows error. If password is less than 6 characters, shows error.*
Postconditions	*None*

FR-12-03: (Toggle User Status)

Field	*Description*
Use Case ID	*UC-ADMIN-03*
Use Case Name	*Toggle User Status (Activate/Deactivate)*
Actor	*Admin*
Description	*Admin changes user status from "active" to "inactive" or vice versa*
Preconditions	*Admin is logged in, user exists*
Basic Flow	*1. Admin views the user table* *2. Selects a user* *3. Clicks "Activate" or "Deactivate" button* *4. System sends POST /api/admin/users/toggle/{id}/* *5. Server toggles is_active* *6. Shows success message* *7. Status badge in table changes*
Alternative Flow	*If user not found, returns 404. If not admin, returns 403.*
Postconditions	*None*

FR-12-04: (Delete User)

Field	Description
Use Case ID	*UC-ADMIN-04*
Use Case Name	*Delete User*
Actor	*Admin*
Description	*Admin permanently deletes a user from the system (including all related data)*
Preconditions	*Admin is logged in, user exists*
Basic Flow	*1. Admin views the user table* *2. Selects a user* *3. Clicks "Delete" button* *4. A confirmation dialog appears* *5. Clicks "OK"* *6. System sends DELETE /api/admin/users/delete/{id}/* *7. Server deletes the user (CASCADE)* *8. Shows success message* *9. Removes the row from the table*
Alternative Flow	*If user not found, returns 404. If not admin, returns 403. If "Cancel" is clicked, operation is aborted.*
Postconditions	*None*

اسم المتطلب	رمز المتطلب
تسجيل الدخول كمشرف	FR-01-03
عرض قائمة المستخدمين	FR-12-01
إضافة مستخدم جديد	FR-12-02
تفعيل/تعطيل مستخدم	FR-12-03
حذف مستخدم	FR-12-04

Patient Requirements

FR-01-01: (Patient Login)

Field	Description
Use Case ID	*UC-PATIENT-LOGIN*
Use Case Name	*Patient Login*
Actor	*Patient*
Description	*Patient enters email and password to receive JWT token and access the patient dashboard*
Preconditions	*Patient is already registered in the database (user_type = 'patient')*
Basic Flow	*1. Patient opens the login page* *2. Enters email and password* *3. Clicks "Login" button* *4. System validates credentials and user_type = 'patient'* *5. System issues access_token and refresh_token* *6. Browser stores tokens in localStorage* *7. Redirects to /patient/dashboard*
Alternative Flow	*If credentials are incorrect or user_type is not 'patient', system shows error "Invalid email or password"*
Postconditions	*None*

FR-02-01: (Patient Registration)

Field	Description
Use Case ID	*UC-PATIENT-REG*
Use Case Name	*Patient Registration*
Actor	*Visitor*
Description	*Visitor creates a new account as a patient*
Preconditions	*No existing account with the same email*
Basic Flow	*1. Visitor opens the registration page* *2. Enters data (name, email, password, confirm, type = patient)* *3. Clicks "Register"* *4. System sends POST /api/login/register/* *5. System checks email uniqueness* *6. System creates records in User (user_type='patient') and PatientProfile* *7. Displays success message and redirects to login page*
Alternative Flow	*If email already exists, system shows error "Email already registered". If password and confirmation do not match, shows error.*
Postconditions	*None*

FR-03: (View Medical Records)

Field	Description
Use Case ID	*UC-VIEW-RECORDS*
Use Case Name	*View Medical Records*
Actor	*Patient*
Description	*Patient displays all his/her medical records in chronological order (newest first)*
Preconditions	*Patient is logged in*
Basic Flow	*1. Patient opens his/her dashboard* *2. Clicks on "Medical Records" tab* *3. System sends GET /api/patient/records/* *4. System returns JSON list of records* *5. UI displays records in a table in chronological order*
Alternative Flow	*If no records exist, system shows message "No medical records found"*
Postconditions	*None*

FR-13-01: (Patient Edit Profile)

Field	Description
Use Case ID	*UC-PATIENT-EDIT*
Use Case Name	*Patient Edit Profile*
Actor	*Patient*
Description	*Patient updates personal information (name, phone, address, blood type, allergies, chronic diseases, emergency contacts)*
Preconditions	*Patient is logged in*
Basic Flow	*1. Patient opens his/her dashboard* *2. Clicks "Edit Profile" button* *3. A modal appears with editable fields* *4. Patient modifies data and clicks "Save Changes"* *5. System sends PUT /api/patient/update/{user_id}/* *6. Server updates User and PatientProfile tables* *7. Shows success message and refreshes UI*
Alternative Flow	*If patient id not found, returns 404. If data is invalid, returns 400.*
Postconditions	*Profile is automatically refetched from server after update*

FR-08: (Book Appointment)

Field	Description
Use Case ID	*UC-BOOK-APPT*
Use Case Name	*Book Appointment*
Actor	*Patient*
Description	*Patient books an appointment with a specific doctor by selecting date, time, and reason*
Preconditions	*Patient is logged in, doctor exists*
Basic Flow	*1. Patient opens "Doctors" tab* *2. Selects a doctor from the list* *3. Clicks "Book Appointment"* *4. A modal appears (calendar, reason)* *5. Patient selects date, time, and reason* *6. Clicks "Confirm Booking"* *7. System sends POST /api/book-appointment/* *8. System creates Appointment (status='scheduled') and initial MedicalRecord (first visit)* *9. Shows success message*
Alternative Flow	*If time slot is already booked, system shows error "Slot not available"*
Postconditions	*An initial medical record is automatically created to link the patient to the doctor*

FR-14: (View Doctors List)

Field	Description
Use Case ID	*UC-VIEW-DOCTORS*
Use Case Name	*View Doctors List*
Actor	*Patient*
Description	*Displays all registered doctors on the platform (for appointment booking)*
Preconditions	*Patient is logged in*
Basic Flow	*1. Patient opens "Doctors" tab* *2. System sends GET /api/doctors/* *3. System returns doctor list (name, specialization, email, phone, hospital)* *4. UI displays cards with doctor information* *5. Patient can search by name or specialty*
Alternative Flow	*If no doctors exist, system shows message "No registered doctors"*
Postconditions	*From this list, the patient can click "Book Appointment"*

FR-05: (Add Vital Signs)

Field	Description
Use Case ID	*UC-ADD-VITAL-PATIENT*
Use Case Name	*Add Vital Signs (Patient Self)*
Actor	*Patient*
Description	*Patient enters his/her own vital signs measurements (blood pressure, heart rate, blood sugar, temperature, oxygen saturation)*
Preconditions	*Patient is logged in, patient exists in database*
Basic Flow	*1. Patient opens his/her dashboard* *2. Opens "Vital Signs" tab* *3. Clicks "Add New Measurements"* *4. A modal appears with fields (systolic, diastolic, heart rate, blood sugar, temperature, oxygen)* *5. Patient enters measurements and clicks "Save"* *6. System sends POST /api/patient/add-vital-signs/* *7. System saves data in VitalSigns table* *8. The vital signs table is updated and a success message is shown*
Alternative Flow	*If data is invalid (values outside normal range), system shows error message*
Postconditions	*New measurements are added to the patient's vital signs history*

FR-11-02: (PATIENT-SIDE-EFFECTS)

Field	Description
Use Case ID	*UC-PATIENT-SIDE-EFFECTS*
Use Case Name	*Add Post-Surgery Side Effects (by Patient)*
Actor	*Patient who underwent the surgery*
Description	*Patient adds side effects experienced after surgery, so the doctor can see and monitor the patient's condition*
Preconditions	*Patient is logged in, surgery exists and its status is completed*
Basic Flow	*1. Patient opens his/her dashboard* *2. Opens "Surgeries" tab* *3. List of completed surgeries is displayed* *4. Patient clicks "Add Side Effects" button next to a surgery* *5. A modal appears with a text field* *6. Patient enters side effects (e.g., pain, nausea, dizziness)* *7. Clicks "Save" button* *8. System sends POST /api/surgeries/add-side-effects/{id}/* *9. System saves side effects in the patient_side_effects field* *10. Shows success message and refreshes the UI*
Alternative Flow	*If surgery is not completed (still scheduled), the button is hidden or an error message is shown*
Postconditions	*The doctor can see these side effects in the patient's page or surgery record*

FR-06: (AI Risk Analysis)

Field	Description
Use Case ID	*UC-ANALYZE-VITALS*
Use Case Name	*Analyze Vital Signs with AI*
Actor	*Patient*
Description	*Patient analyzes his/her last 5 recorded vital signs using AI (OpenRouter API) and receives a report with risk level and medical recommendations*
Preconditions	*Patient is logged in, has at least one vital sign recorded in the database*
Basic Flow	*1. Patient opens his/her dashboard* *2. Opens "Vital Signs" tab* *3. Table of previously recorded vital signs is displayed* *4. Patient clicks " **AI Risk Analysis**" button* *5. System sends POST /api/analyze-risk/ with patient_id* *6. Server fetches last 5 vital signs of the patient* *7. Server calculates averages (blood pressure, heart rate, blood sugar)* *8. Server sends data to OpenRouter API* *9. System receives JSON (risk_level, risk_score, details, recommendations)* *10. System saves result in AIAnalysisResult table* *11. System displays result in a modal (risk level, details, recommendations)*
Alternative Flow	*If not enough vital signs, system shows error "No enough vital signs for analysis". If OpenRouter fails, fallback algorithm is used.*
Postconditions	*Analysis result is saved in the database for later viewing in the "AI Analysis" tab*

Doctor Requirements

FR-01-02: (Doctor Login)

Field	Description
Use Case ID	*UC-DOCTOR-LOGIN*
Use Case Name	*Doctor Login*
Actor	*Doctor*
Description	*Doctor enters email and password to receive JWT token and access the doctor dashboard*
Preconditions	*Doctor is already registered in the database (user_type = 'doctor')*
Basic Flow	*1. Doctor opens the login page* *2. Enters email and password* *3. Clicks "Login" button* *4. System validates credentials and user_type = 'doctor'* *5. System issues access_token and refresh_token* *6. Browser stores tokens in localStorage* *7. Redirects to /doctor/dashboard*
Alternative Flow	*If credentials are incorrect or user_type is not 'doctor', system shows error "Invalid email or password"*
Postconditions	*None*

FR-02-02: (Doctor Registration)

Field	Description
Use Case ID	*UC-DOCTOR-REG*
Use Case Name	*Doctor Registration with Specialization and License*
Actor	*Visitor*
Description	*Visitor creates a new account as a doctor, entering specialization and license number*
Preconditions	*No existing account with the same email*
Basic Flow	*1. Visitor opens the registration page* *2. Enters data (name, email, password, confirm)* *3. Selects user type = "doctor"* *4. Extra fields appear: specialization and license_number* *5. Visitor enters specialization and license number* *6. Clicks "Register"* *7. System sends POST /api/login/register/* *8. System checks email uniqueness* *9. System creates records in User (user_type='doctor') and DoctorProfile (with specialization and license_number)* *10. Displays success message and redirects to login page*
Alternative Flow	*If email already exists, system shows error. If data is incomplete (missing specialization or license number), system shows error.*
Postconditions	*None*

FR-04: (Add Medical Record)

Field	Description
Use Case ID	*UC-ADD-RECORD*
Use Case Name	*Add Medical Record*
Actor	*Doctor*
Description	*Doctor adds a diagnosis, prescription, and notes for a specific patient*
Preconditions	*Doctor is logged in, patient exists in doctor's list*
Basic Flow	*1. Doctor opens his/her dashboard* *2. Selects a patient from the list* *3. Clicks "Add Report" button* *4. A modal appears (visit type, diagnosis, prescription, notes)* *5. Doctor enters data and clicks "Save"* *6. System sends POST /api/doctor/add-record/* *7. System saves record in MedicalRecord table* *8. Shows success message "☑ Report added successfully"*
Alternative Flow	*If data is incomplete (e.g., diagnosis empty), system shows error message*
Postconditions	*A notification may be sent to the patient (future work)*

FR-07: (View Patients List)

Field	Description
Use Case ID	*UC-VIEW-PATIENTS*
Use Case Name	*View Patients List*
Actor	*Doctor*
Description	*Displays the list of patients linked to the doctor (via medical records or previous appointments)*
Preconditions	*Doctor is logged in*
Basic Flow	*1. Doctor opens his/her dashboard* *2. System sends GET /api/doctor/patients/* *3. System returns patient list (name, national ID, phone, last visit, risk level)* *4. UI displays a table with patients and action buttons (edit, add report, analyze)* *5. Doctor can search by name or national ID*
Alternative Flow	*If no patients exist, system shows message "No patients found"*
Postconditions	*None*

FR-09: (Complete Appointment)

Field	Description
Use Case ID	*UC-COMPLETE-APPT*
Use Case Name	*Complete Appointment (Move to Review History)*
Actor	*Doctor*
Description	*Doctor changes appointment status from "scheduled" to "completed" and moves it to the review history*
Preconditions	*Doctor is logged in, appointment exists and its status is "scheduled"*
Basic Flow	*1. Doctor opens "Appointments" tab* *2. Table of scheduled appointments is shown* *3. Doctor clicks "Complete Appointment" button next to the desired appointment* *4. A confirmation dialog appears (optional)* *5. System sends POST /api/appointments/complete/{id}/* *6. Server updates appointment status to 'completed'* *7. Shows success message* *8. The appointment disappears from "Appointments" list and appears in "Completed" list*
Alternative Flow	*If appointment does not exist or does not belong to the doctor, system returns 404 or 403*
Postconditions	*Both appointment and completed lists are automatically refreshed*

FR-10: (Add Surgery)

Field	Description
Use Case ID	*UC-ADD-SURGERY*
Use Case Name	*Add Surgery*
Actor	*Doctor*
Description	*Doctor adds a new surgical procedure with status "scheduled" for a specific patient*
Preconditions	*Doctor is logged in, patient exists*
Basic Flow	*1. Doctor opens patient page* *2. Clicks "Surgeries" button* *3. Clicks "Add New Surgery"* *4. A modal appears (surgery name, date, hospital, notes, medications)* *5. Doctor enters data and clicks "Save"* *6. System sends POST /api/surgeries/add/* *7. System saves surgery in Surgery table (status='scheduled')* *8. Shows success message* *9. Surgeries list on patient page is updated*
Alternative Flow	*If data is incomplete (e.g., surgery name empty), system shows error message*
Postconditions	*None*

FR-11-01: (Complete Surgery)

Field	*Description*
Use Case ID	*UC-COMPLETE-SURGERY*
Use Case Name	*Complete Surgery (Auto-create Medical Record)*
Actor	*Doctor*
Description	*Doctor completes a surgical procedure, enters side effects, and automatically creates a medical record*
Preconditions	*Doctor is logged in, surgery exists and its status is "scheduled"*
Basic Flow	*1. Doctor opens patient page → "Surgeries" tab* *2. Scheduled surgeries are shown* *3. Doctor clicks "Complete Surgery" button next to a scheduled surgery* *4. A modal appears to enter side effects* *5. Doctor enters side effects and clicks "Confirm"* *6. System sends POST /api/surgeries/complete/{id}/* *7. Server updates surgery status to 'completed'* *8. Server creates a new medical record (visit_type='Surgery') containing surgery name, medications, and side effects* *9. Shows success message* *10. The surgery disappears from "Scheduled" list and side effects appear in medical records*
Alternative Flow	*If surgery is already completed, system shows error "Surgery already completed"*
Postconditions	*A medical record is automatically created in the MedicalRecord table*

FR-07-01: (View Patient Details)

Field	Description
Use Case ID	*UC-VIEW-PATIENT-DETAILS*
Use Case Name	*View Patient Details*
Actor	*Doctor*
Description	*Doctor views a complete page with all patient data: profile, medical records, vital signs, surgeries, and AI analyses*
Preconditions	*Doctor is logged in, patient exists in doctor's list*
Basic Flow	*1. Doctor opens his/her dashboard* *2. System displays patient list* *3. Doctor clicks "View" button next to a specific patient* *4. System navigates to /doctor/patients/{patient_id}* *5. System fetches all patient data (GET /api/patient/profile/{id}/, GET /api/patient/records/{id}/, GET /api/patient/vital-signs/{id}/, GET /api/patient/surgeries/{id}/)* *6. UI displays separate tabs (Overview, Medical Records, Vital Signs, Surgeries, AI Analyses)*
Alternative Flow	*If patient does not exist or does not belong to the doctor, system returns 404 or 403*
Postconditions	*From this page, doctor can add reports, surgeries, or analyses*

FR-07-02: (Edit Patient Data)

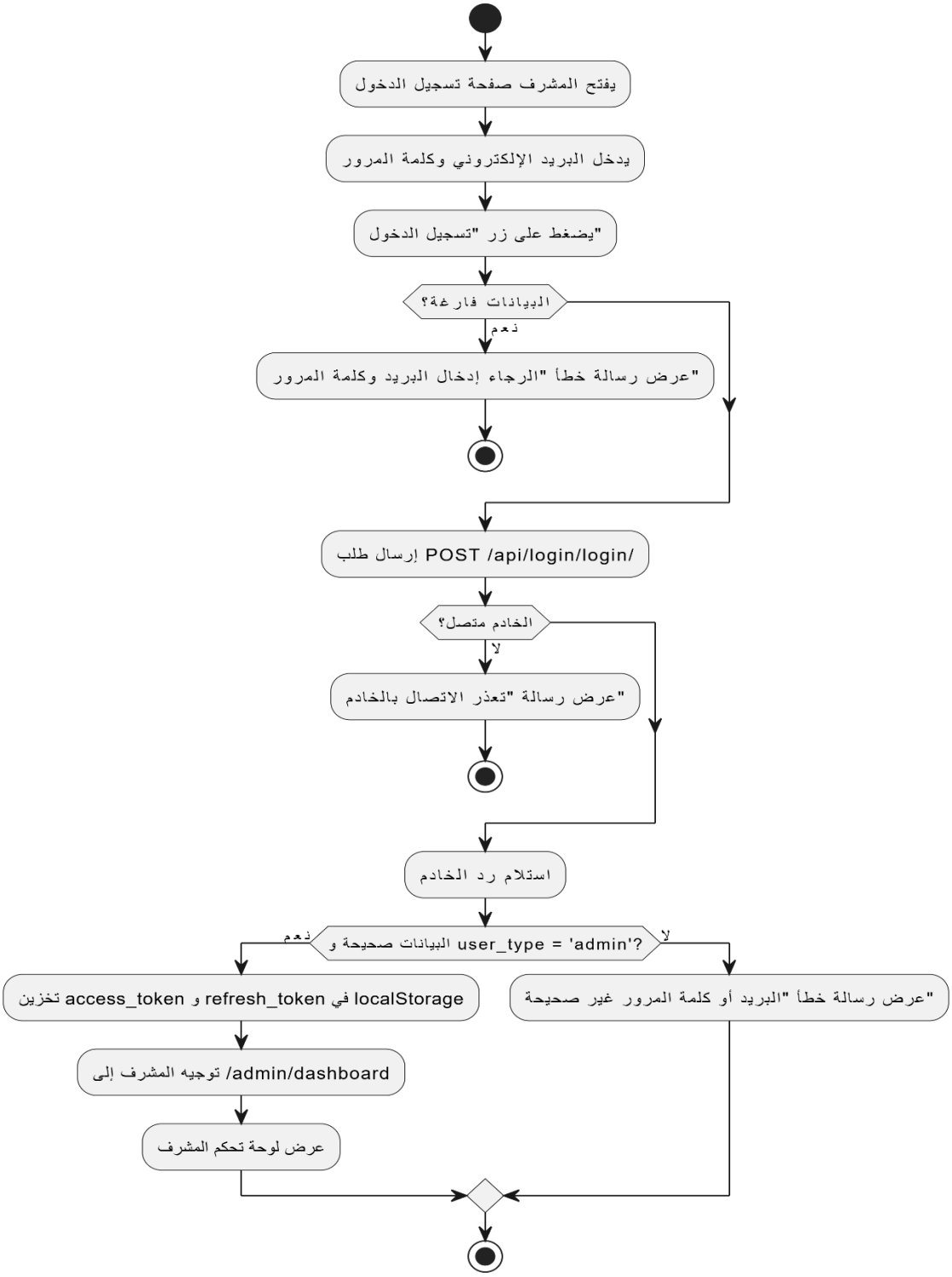
Field	*Description*
Use Case ID	*UC-EDIT-PATIENT*
Use Case Name	*Edit Patient Data*
Actor	*Doctor*
Description	*Doctor edits patient personal information (name, phone, address, blood type, allergies, chronic diseases, emergency contacts)*
Preconditions	*Doctor is logged in, patient exists in doctor's list*
Basic Flow	*1. Doctor opens his/her dashboard* *2. System displays patient list* *3. Doctor clicks "Edit" button next to a specific patient* *4. A modal appears with all editable fields* *5. Doctor modifies the required data* *6. Clicks "Save Changes" button* *7. System sends PUT /api/patient/update/{user_id}/* *8. Server updates User and PatientProfile tables* *9. Shows success message "☑ Profile updated successfully"* *10. UI is refreshed with new data*
Alternative Flow	*If patient does not exist or does not belong to the doctor, system returns 404 or 403*
Postconditions	*Data is refetched and displayed after update*

FR-07-03: Analyze Patient Risk with AI

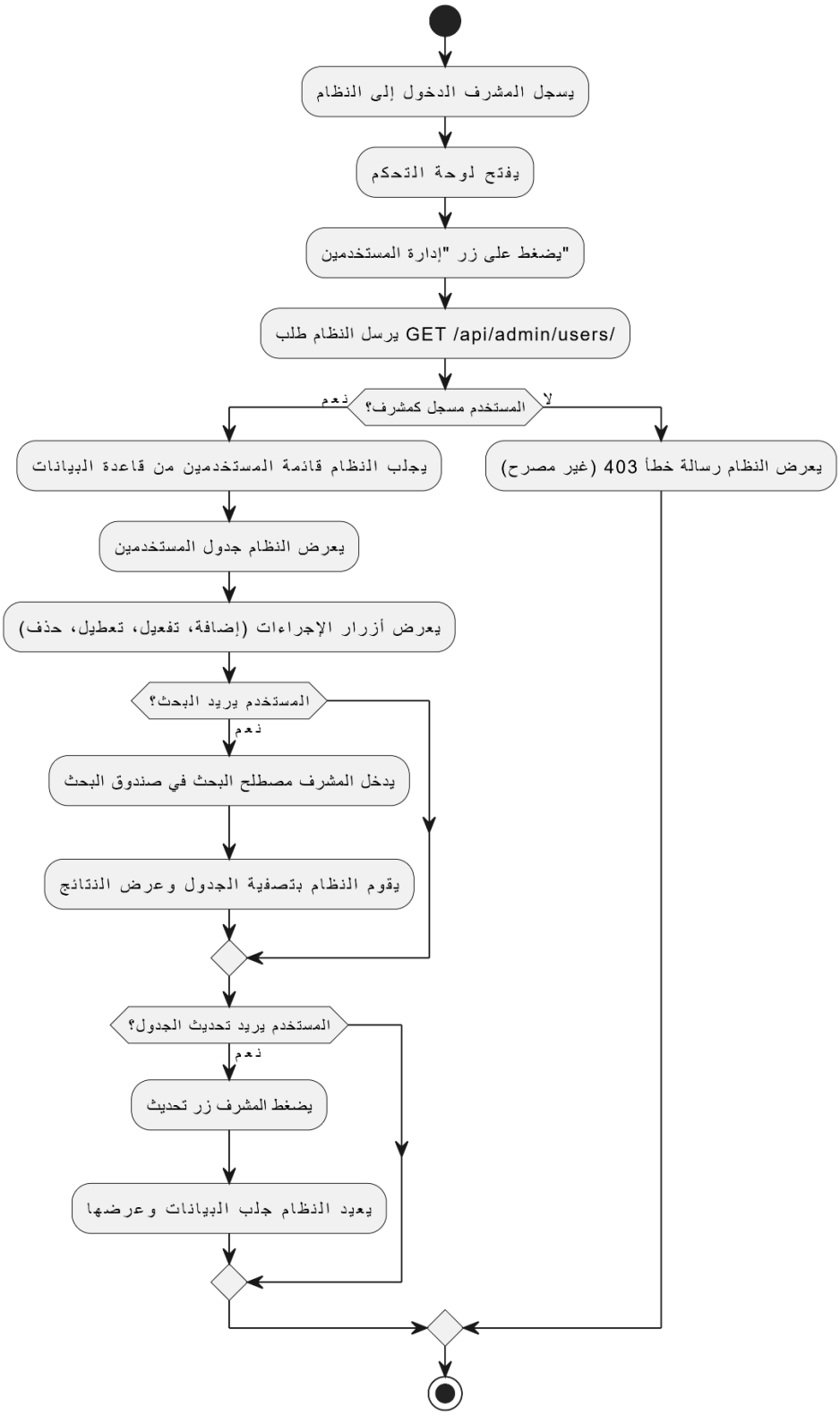
Field	Description
Use Case ID	*UC-DOCTOR-ANALYZE*
Use Case Name	*Analyze Patient Risk with AI*
Actor	*Doctor*
Description	*Doctor analyzes patient risk using AI (vital signs analysis or free text symptoms)*
Preconditions	*Doctor is logged in, patient exists in doctor's list*
Basic Flow	*1. Doctor opens his/her dashboard* *2. System displays patient list* *3. Doctor clicks "Analyze" button next to a specific patient* *4. System navigates to /doctor/analyze/{patient_id}* *5. System displays analysis interface (analysis type selection, text input, or vital signs usage)* *6. Doctor selects analysis type and clicks "Analyze"* *7. System sends POST /api/analyze-risk/* *8. Server analyzes data via OpenRouter API* *9. System displays result (risk level, details, recommendations)*
Alternative Flow	*If not enough vital signs, system shows error message and prompts for text input*
Postconditions	*Analysis result is saved in the database*

Activity Diagram

مخطط النشاط - تسجيل الدخول كمشرف



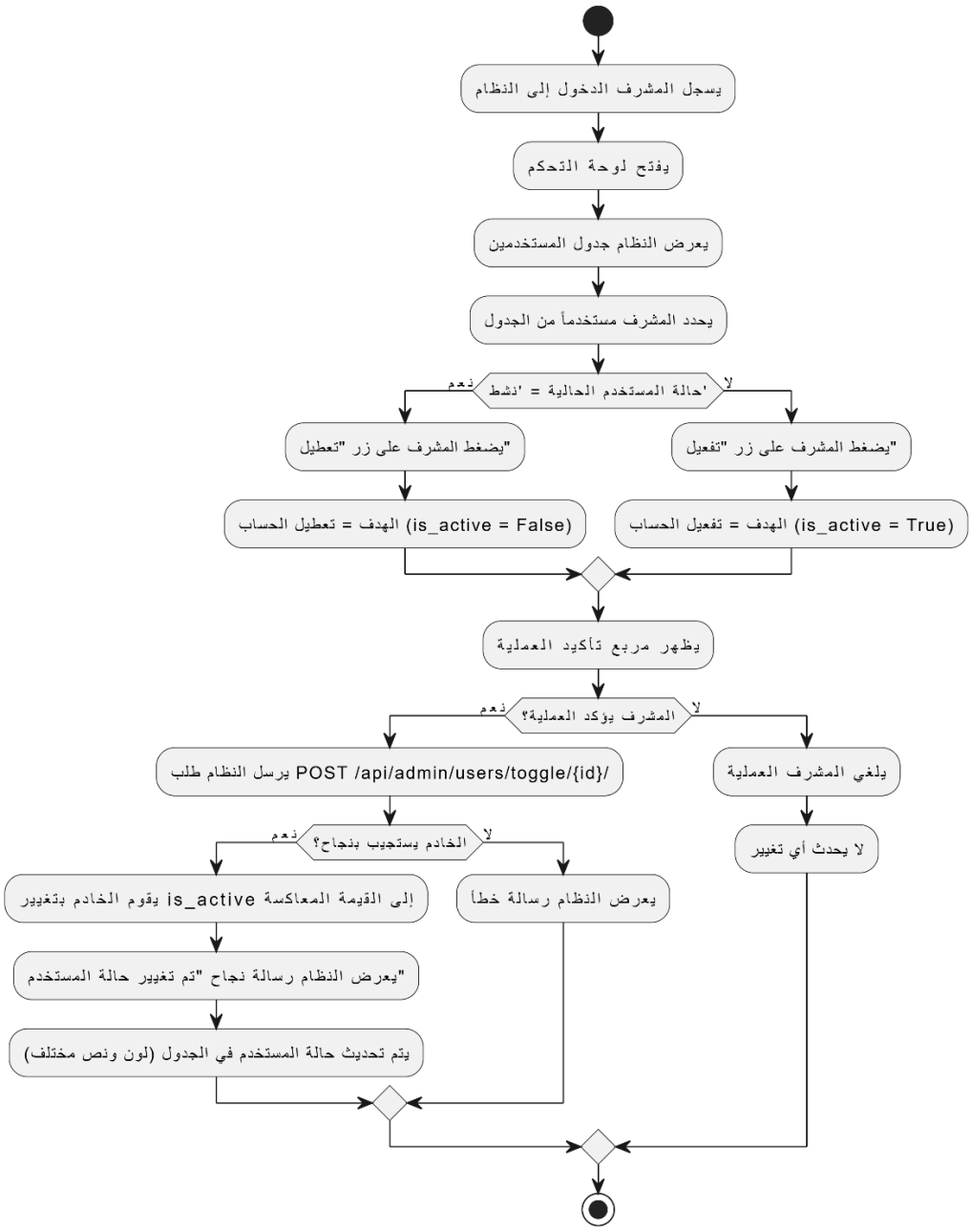
مخطط النشاط - عرض قائمة المستخدمين (للمشرف)



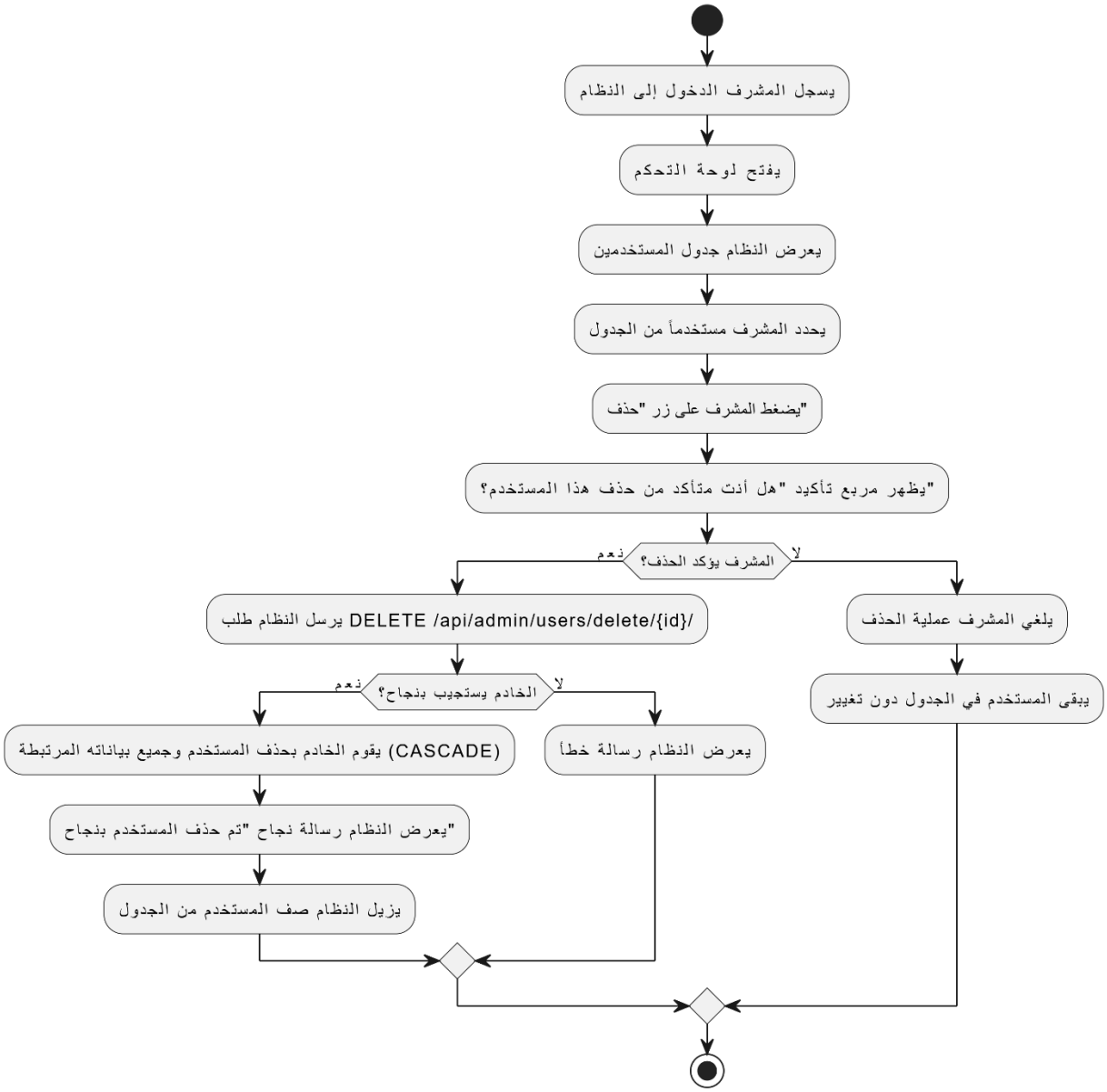
مخطط النشاط - إضافة مستخدم جديد (للمشرف)



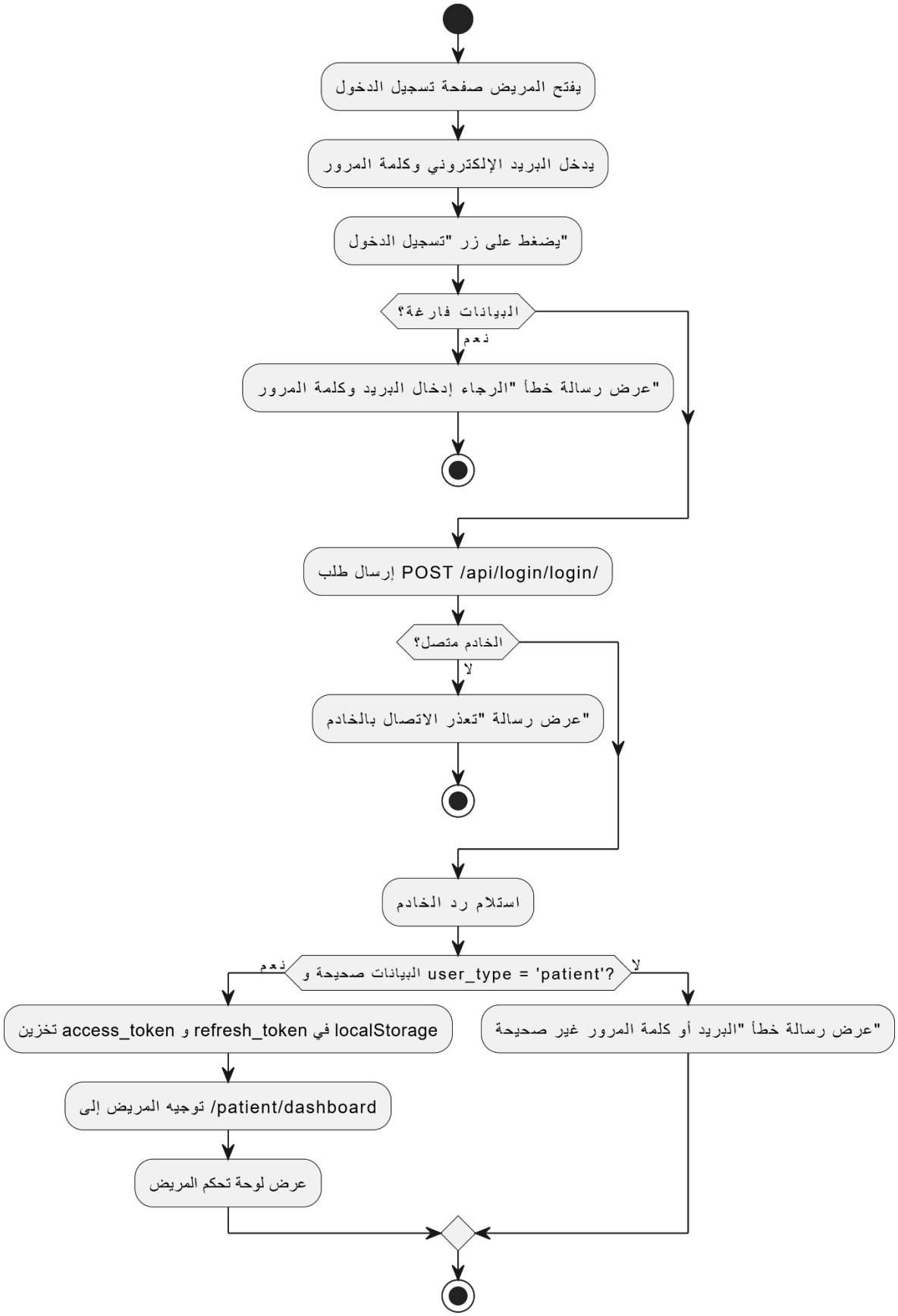
مخطط النشاط - تفعيل/تعطيل مستخدم (للمشرف)



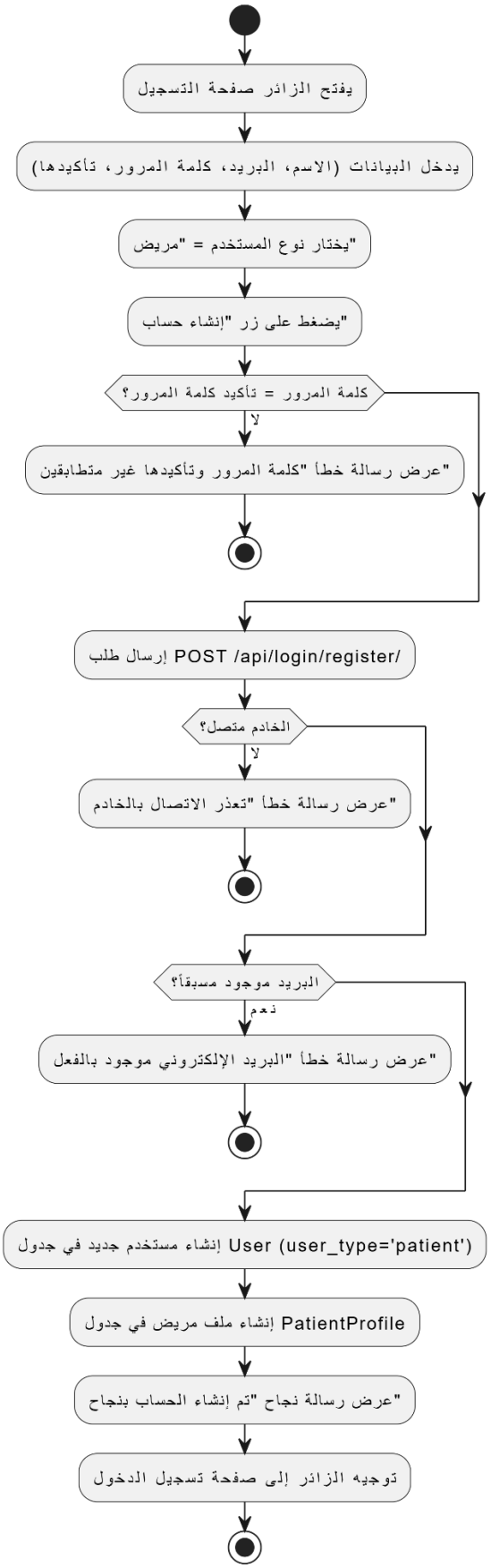
مخطط النشاط - حذف مستخدم (للمشرف)



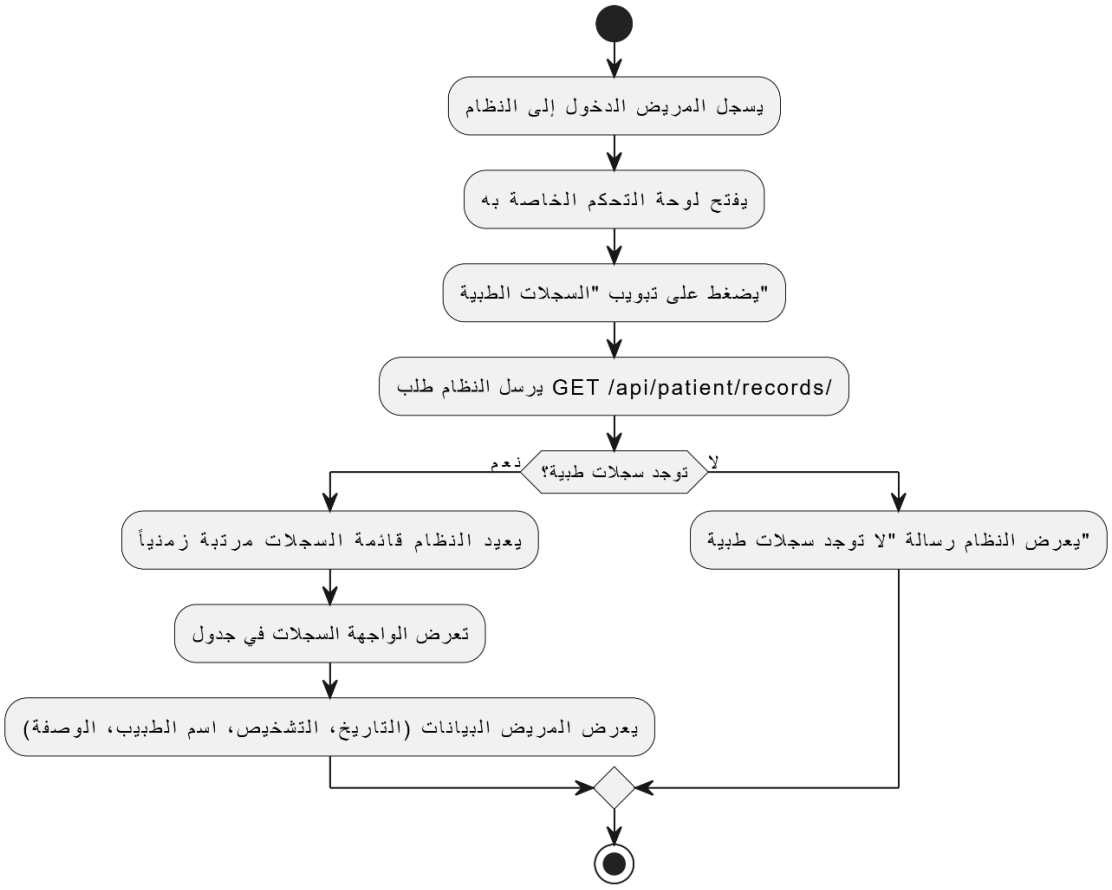
مخطط النشاط - تسجيل الدخول كمريض



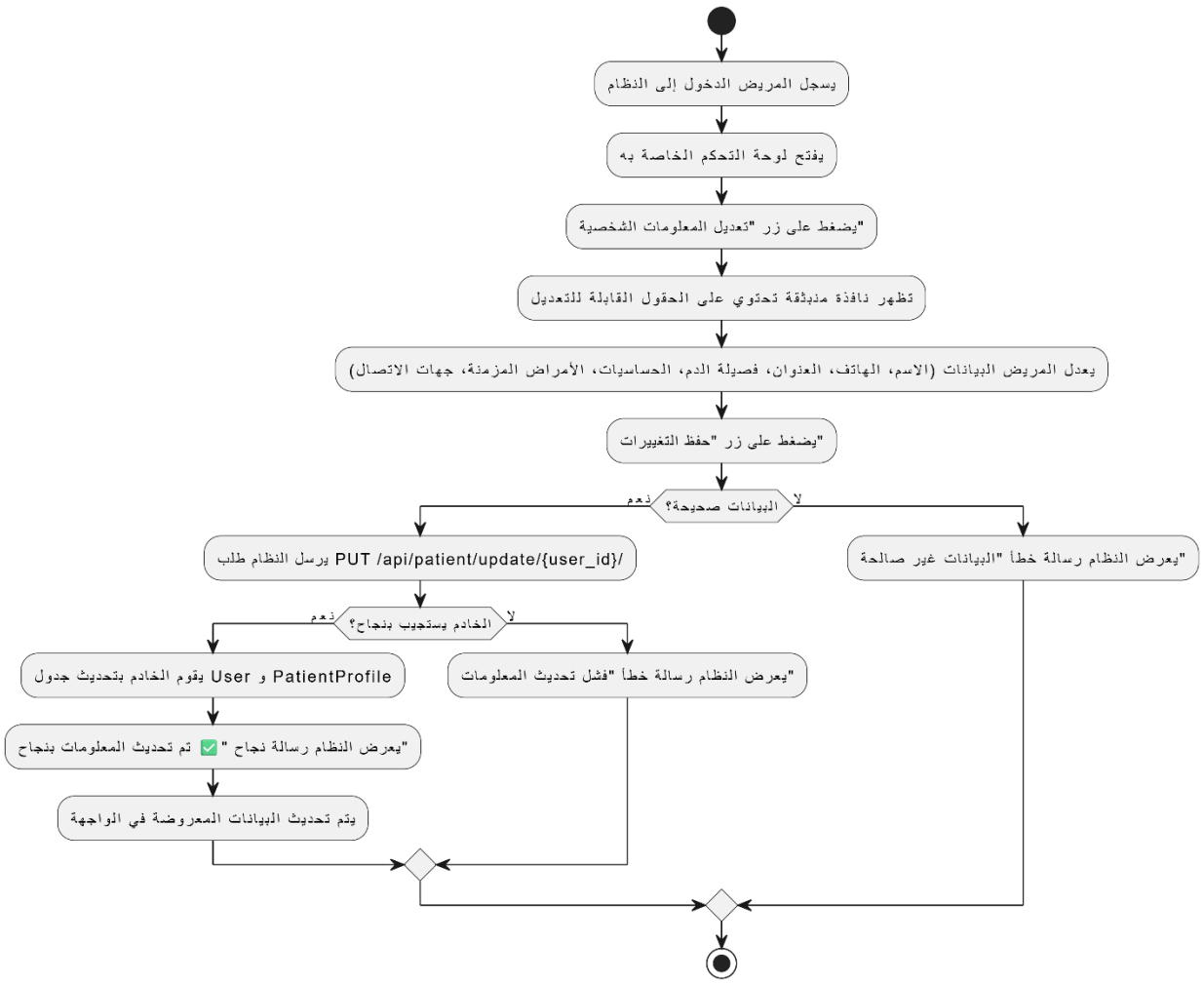
مخطط النشاط - إنشاء حساب مريض

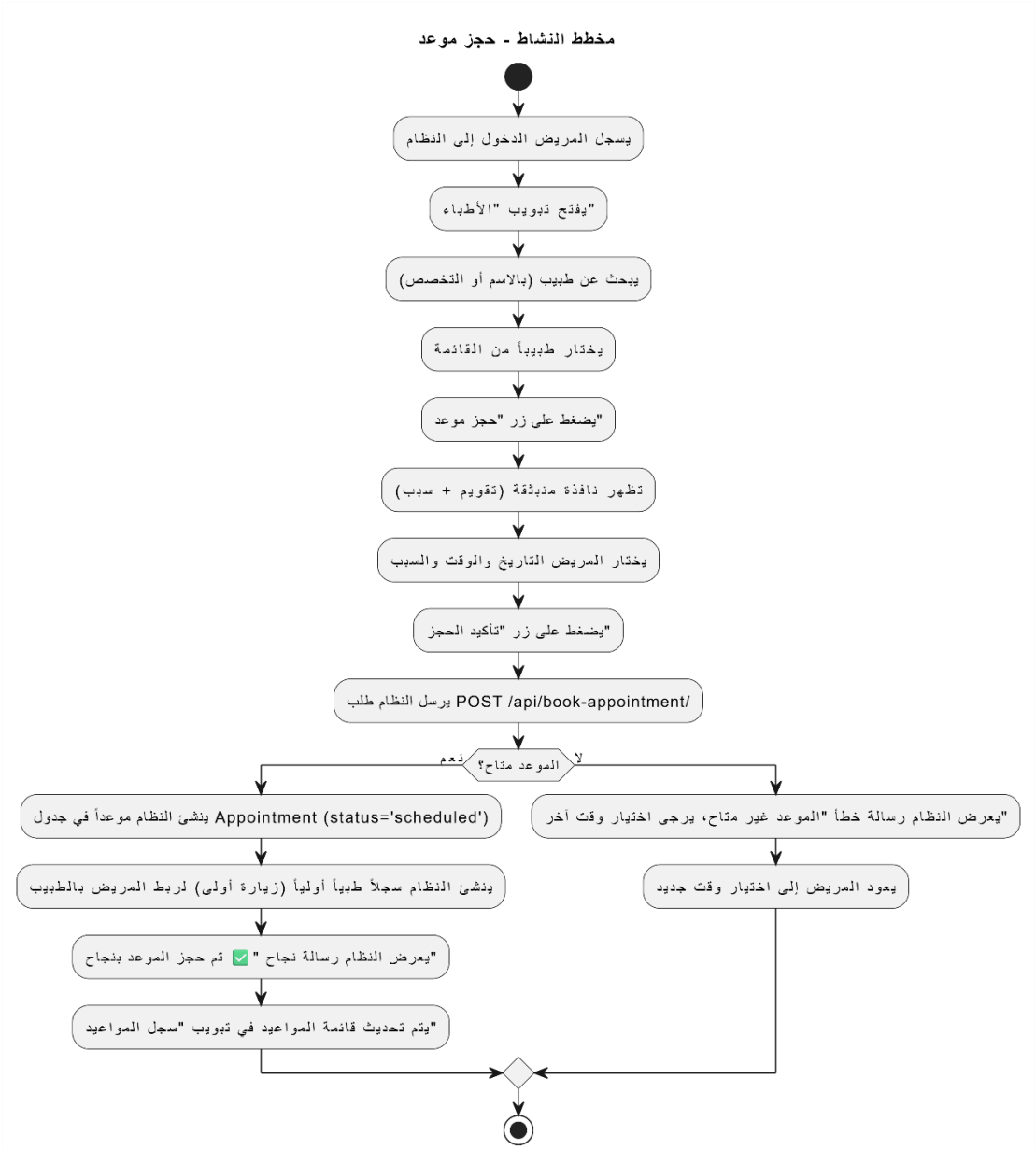


مخطط النشاط - عرض السجلات الطبية

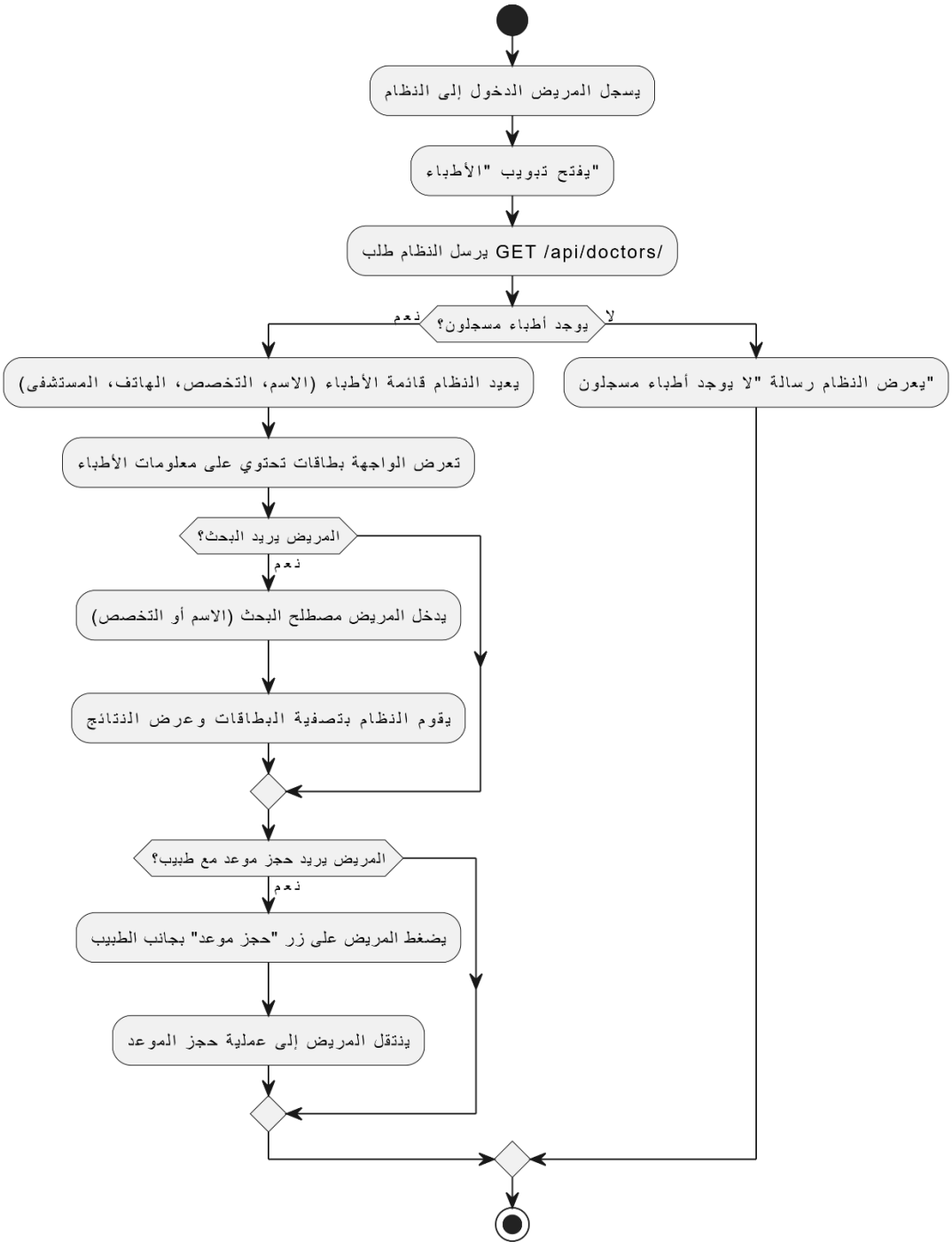


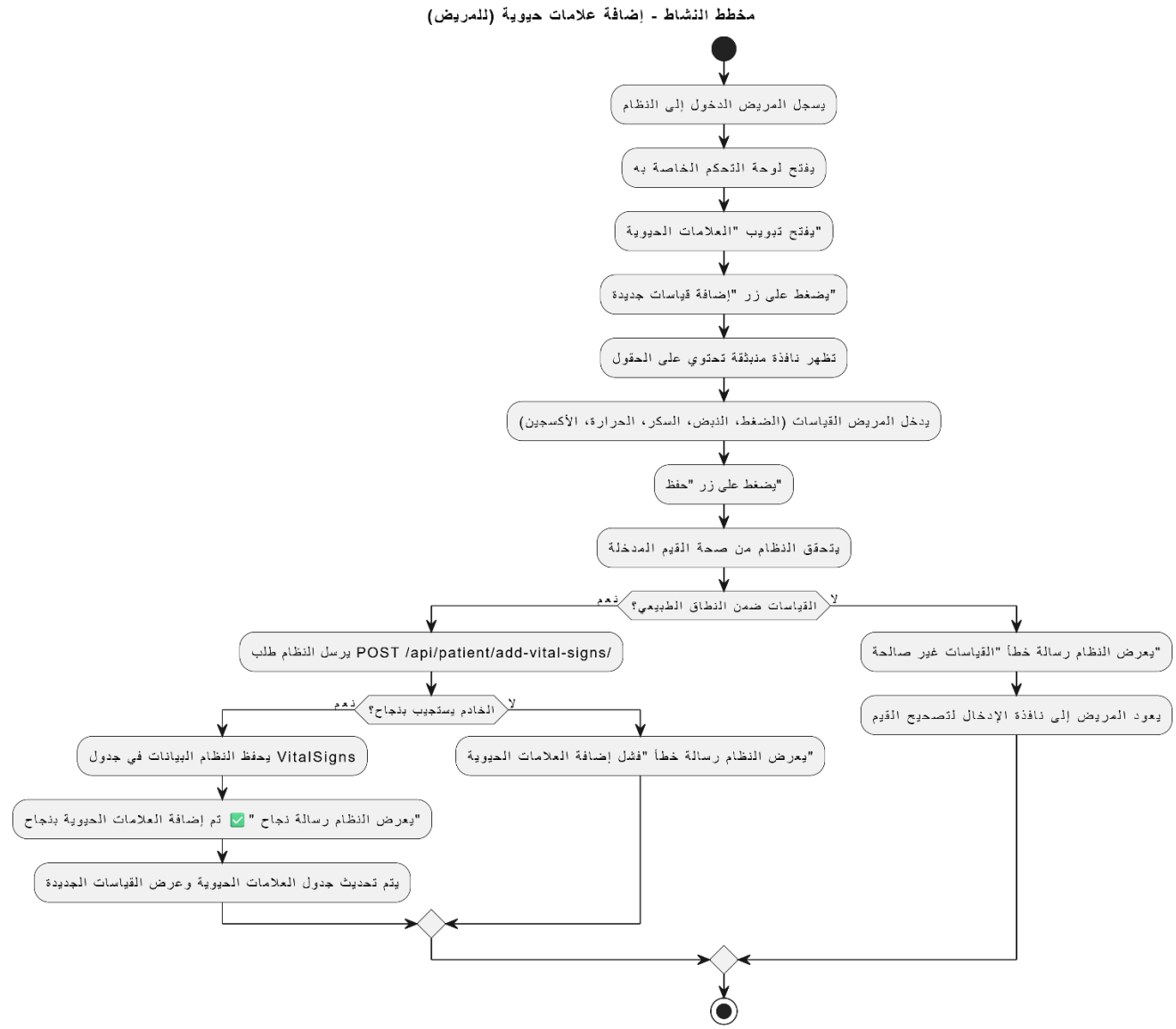
مخطط النشاط - تعديل الملف الشخصي للمريض



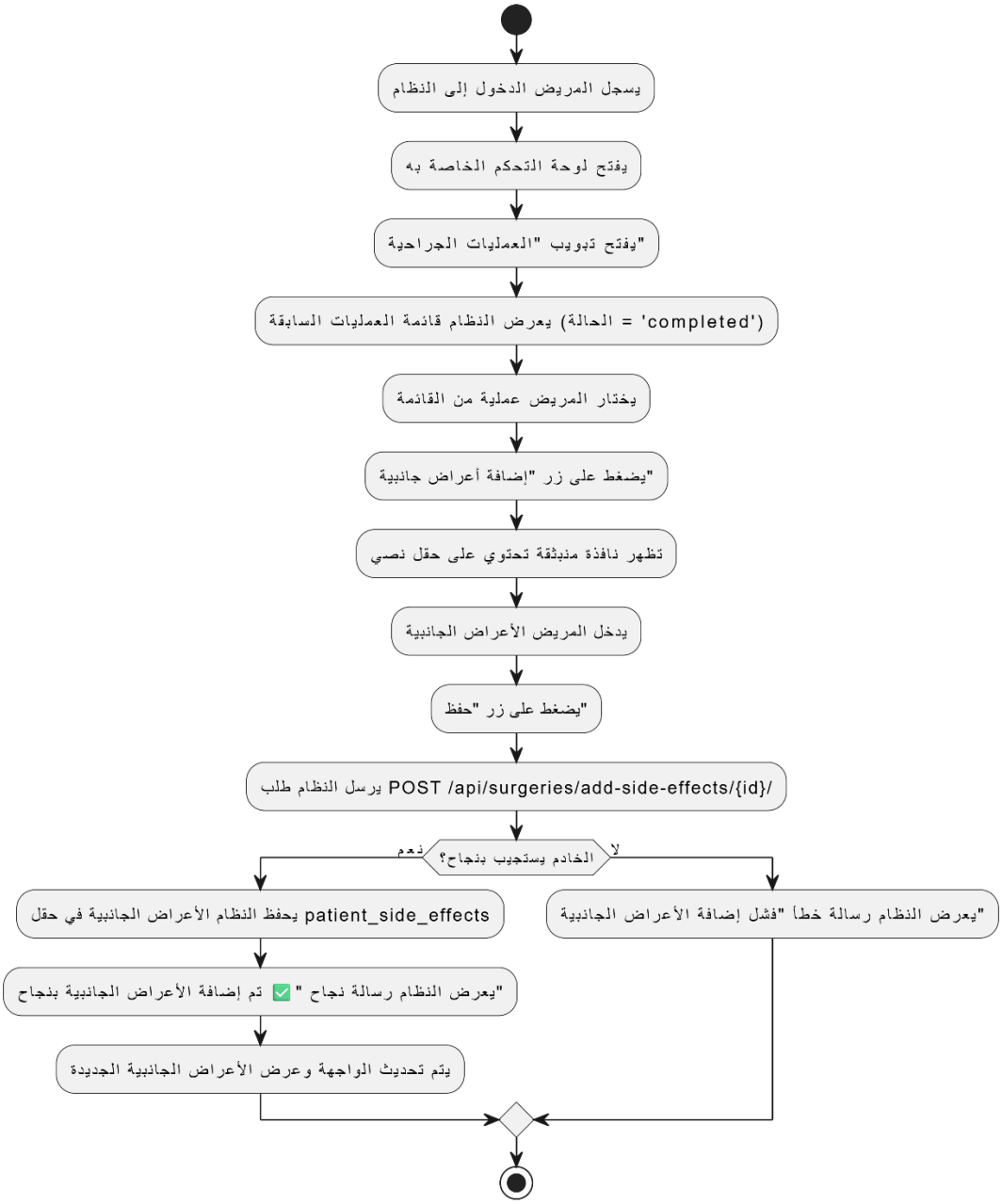

مخطط النشاط - حجز موعد
يسجل المريض الدخول إلى النظام
"يفتح تبويب "الأطباء
يبحث عن طبيب (بالاسم أو التخصص)
يختار طبيباً من القائمة
"يضغط على زر "حجز موعد
تظهر نافذة منبثقة (تقويم + سبب)
يختار المريض التاريخ والوقت والسبب
"يضغط على زر "تأكيد الحجز
POST /api/book-appointment/ يرسل النظام طلب
الموعد متاح؟
نعم
لا
Appointment (status='scheduled') ينشئ النظام موعداً في جدول
ينشئ النظام سجلاً طبياً أولياً (زيارة أولى) لربط المريض بالطبيب
"يعرض النظام رسالة نجاح " ✅ تم حجز الموعد بنجاح
"يتم تحديث قائمة المواعيد في تبويب "سجل المواعيد
"يعرض النظام رسالة خطأ "الموعد غير متاح، يرجى اختيار وقت آخر
يعود المريض إلى اختيار وقت جديد

مخطط النشاط - عرض قائمة الأطباء

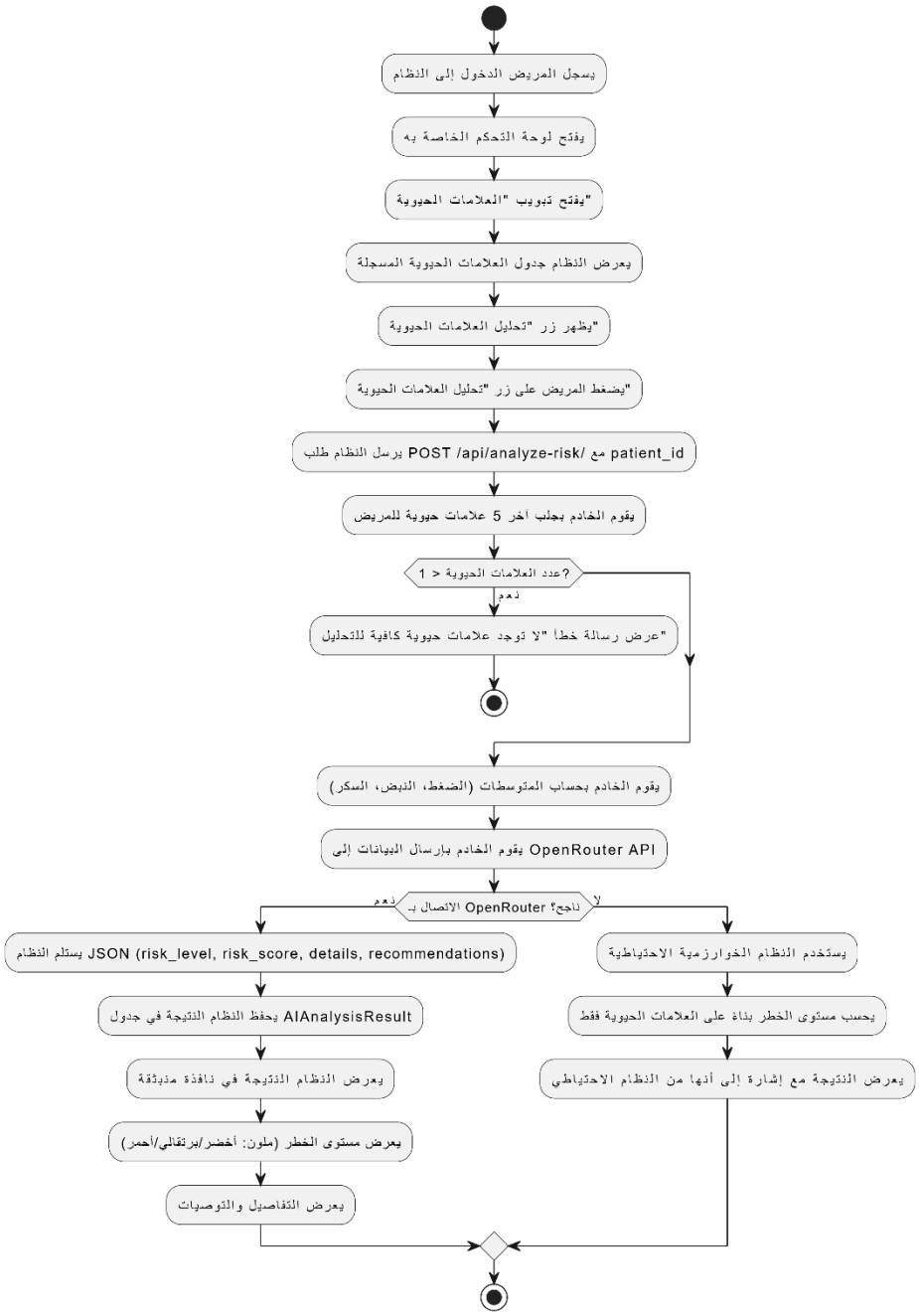



مخطط النشاط - إضافة علامات حيوية (للمريض)
يسجل المريض الدخول إلى النظام
يفتح لوحة التحكم الخاصة به
"يفتح تبويب "العلامات الحيوية
"يضغط على زر "إضافة قياسات جديدة
تظهر نافذة منبثقة تحتوي على الحقول
يدخل المريض القياسات (الضغط، النبض، السكر، الحرارة، الأكسجين)
"يضغط على زر "حفظ
يتحقق النظام من صحة القيم المدخلة
القياسات ضمن النطاق الطبيعي؟
نعم
لا
POST /api/patient/add-vital-signs/ يرسل النظام طلب
"يعرض النظام رسالة خطأ "القياسات غير صالحة
الخادم يستجيب بنجاح؟
نعم
لا
يعود المريض إلى نافذة الإدخال لتصحيح القيم
VitalSigns يحفظ النظام البيانات في جدول
"يعرض النظام رسالة خطأ "فشل إضافة العلامات الحيوية
"يعرض النظام رسالة نجاح " ✅ تم إضافة العلامات الحيوية بنجاح
يتم تحديث جدول العلامات الحيوية وعرض القياسات الجديدة

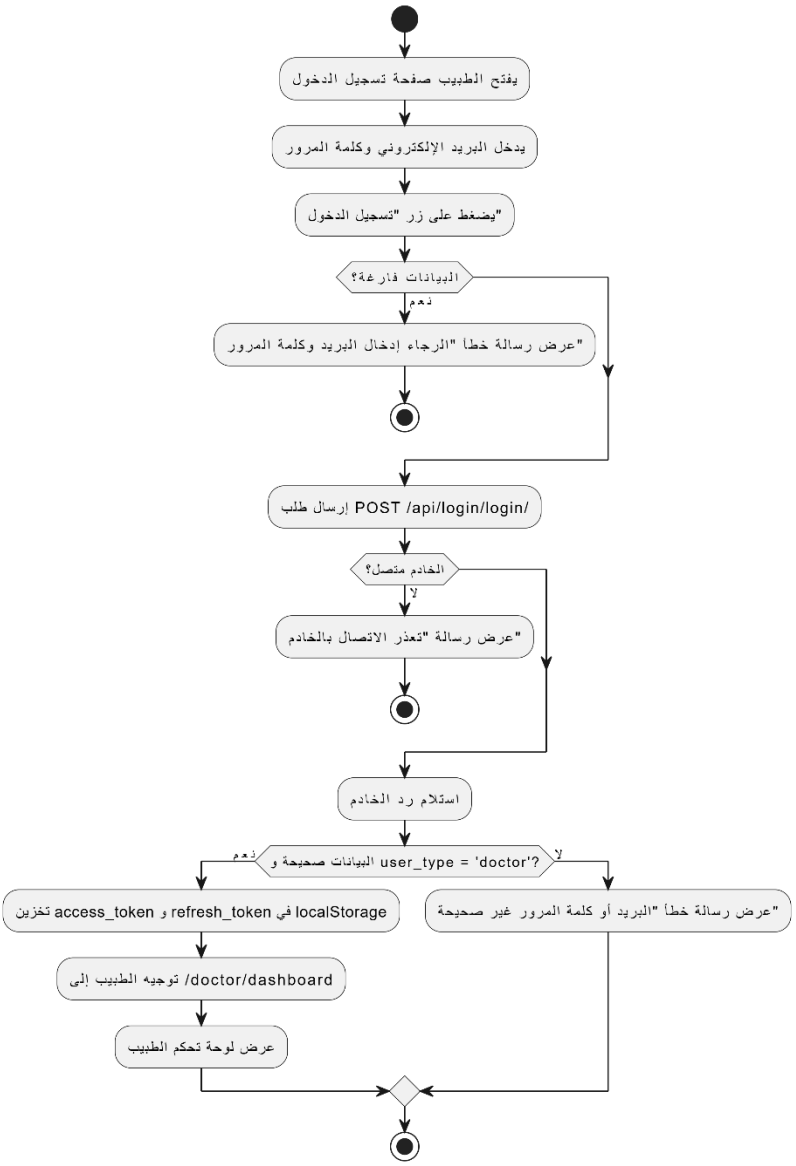
مخطط النشاط - إضافة أعراض جانبية للعملية الجراحية (من قبل المريض)



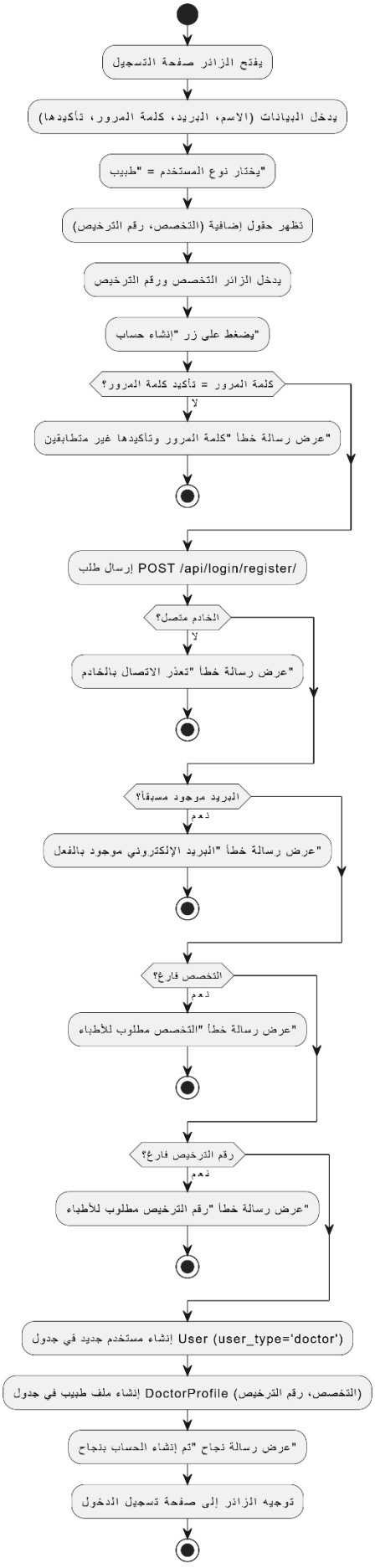
مخطط النشاط - تحليل العلامات الحيوية بالذكاء الاصطناعي



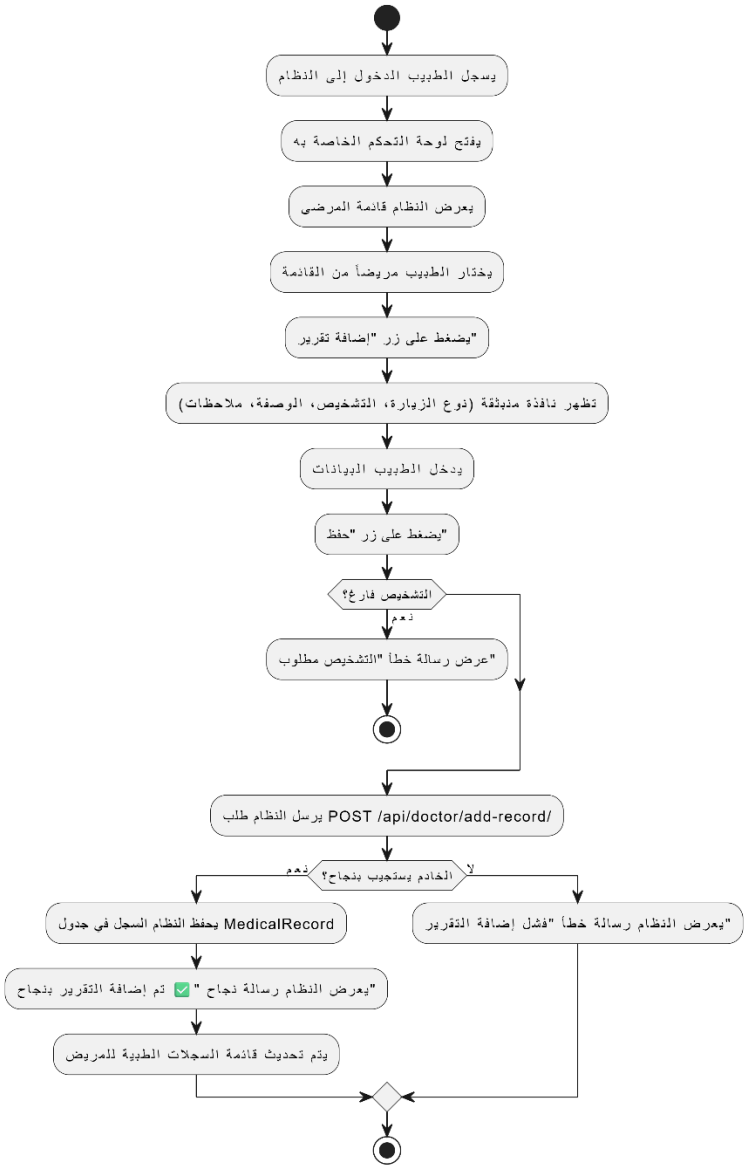
مخطط النشاط - تسجيل الدخول كطبيب

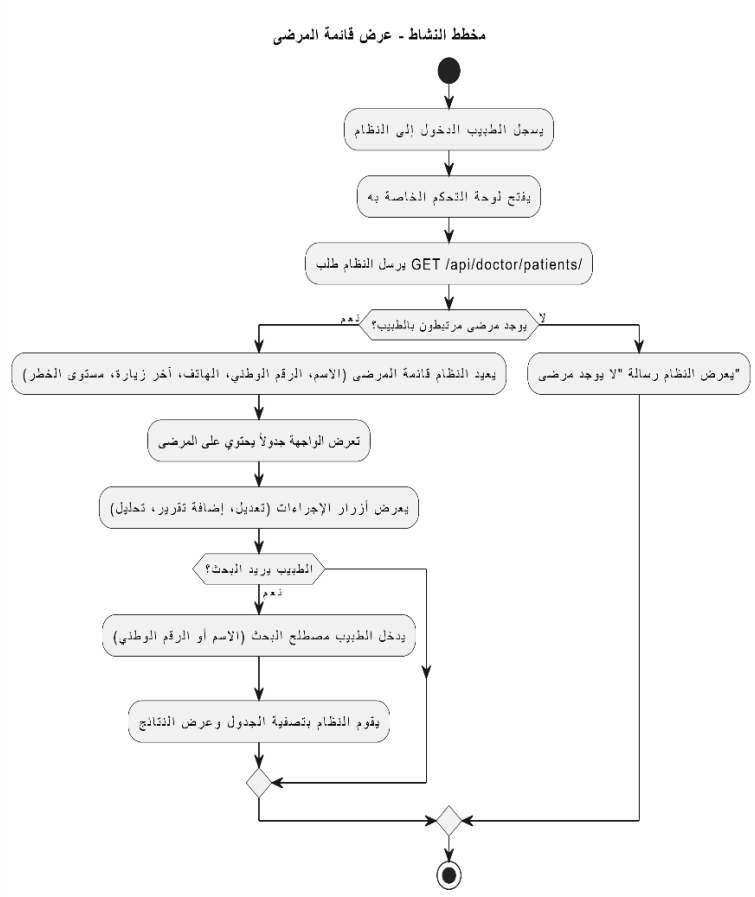


مخطط النشاط - إنشاء حساب طبيب مع تخصص وترخيص

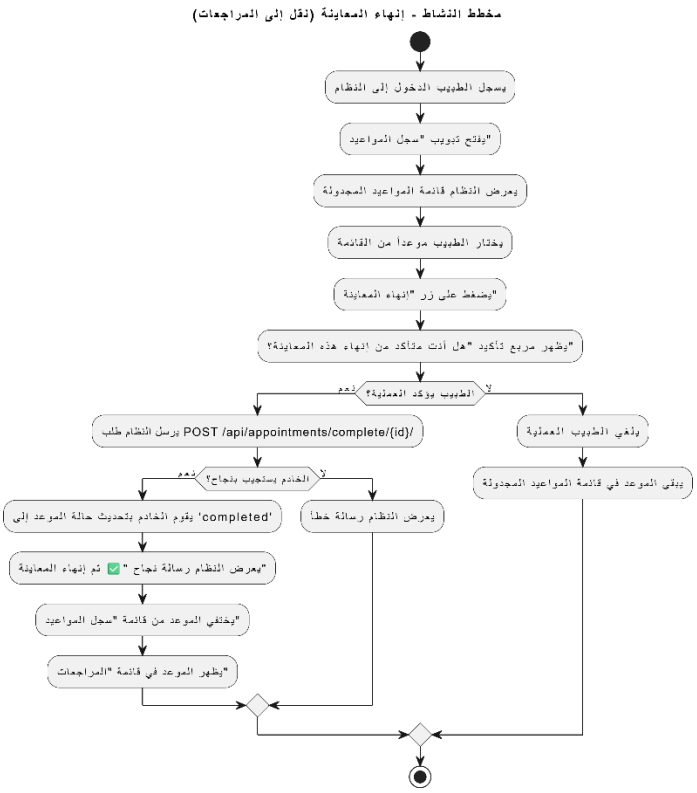


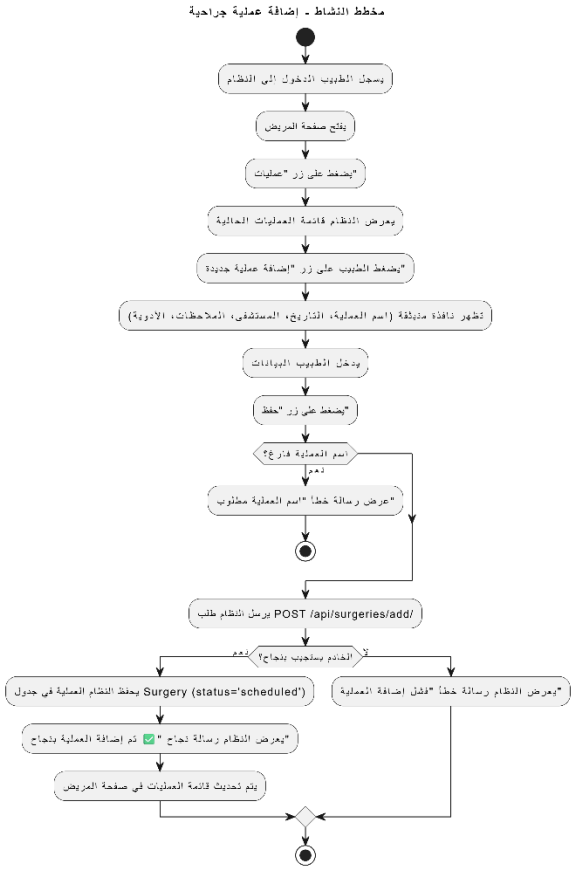
مخطط النشاط - إضافة سجل طبي



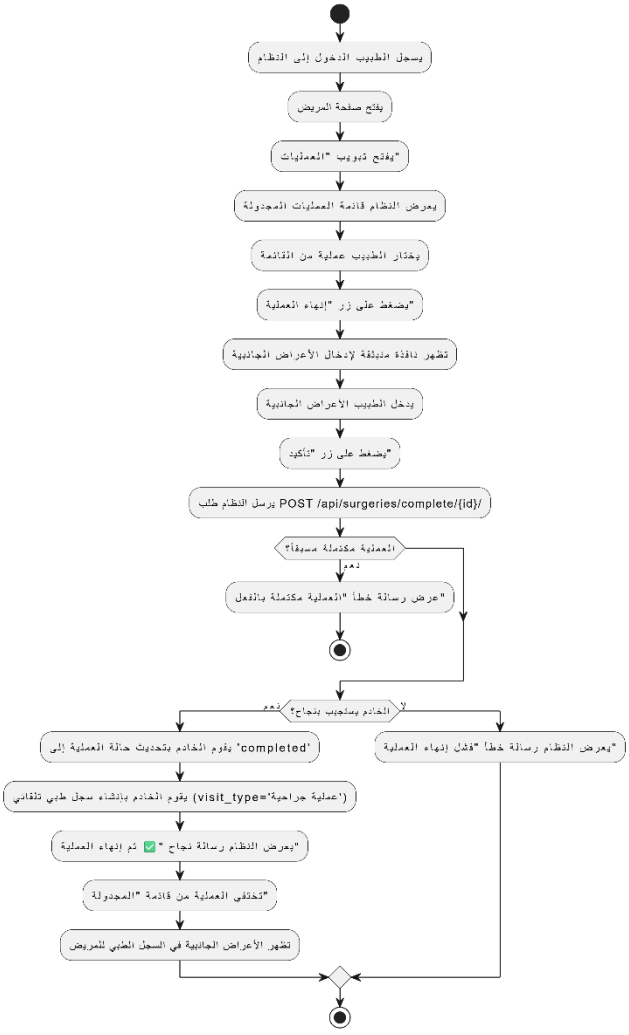

مخطط النشاط - عرض قائمة المرضى
يسجل الطبيب الدخول إلى النظام
يفتح لوحة التحكم الخاصة به
يرسل النظام طلب GET /api/doctor/patients/
يوجد مرضى مرتبطون بالطبيب؟
نعم
لا
يعيد النظام قائمة المرضى (الاسم، الرقم الوطني، الهاتف، آخر زيارة، مستوى الخطر)
"يعرض النظام رسالة "لا يوجد مرضى
تعرض الواجهة جدولاً يحتوي على المرضى
يعرض أزرار الإجراءات (تعديل، إضافة تقرير، تحليل)
الطبيب يريد البحث؟
نعم
يدخل الطبيب مصطلح البحث (الاسم أو الرقم الوطني)
يقوم النظام بتصفية الجدول وعرض النتائج

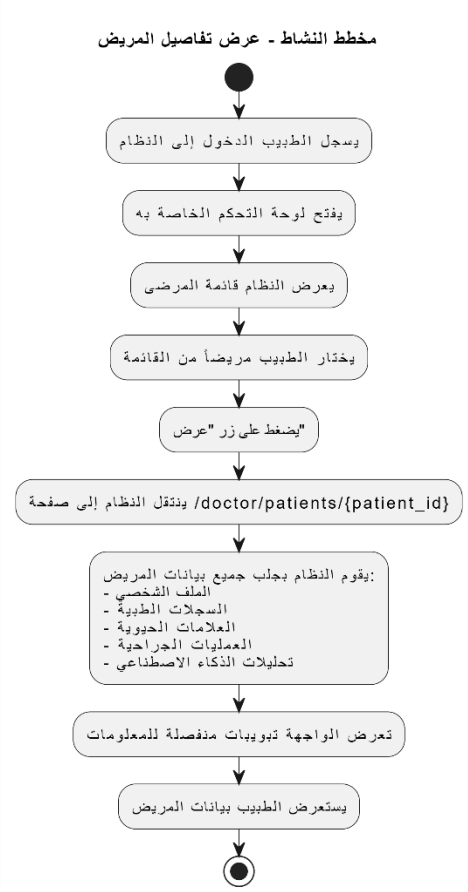
مخطط النشاط - إنهاء المعاينة (نقل إلى المراجعات)

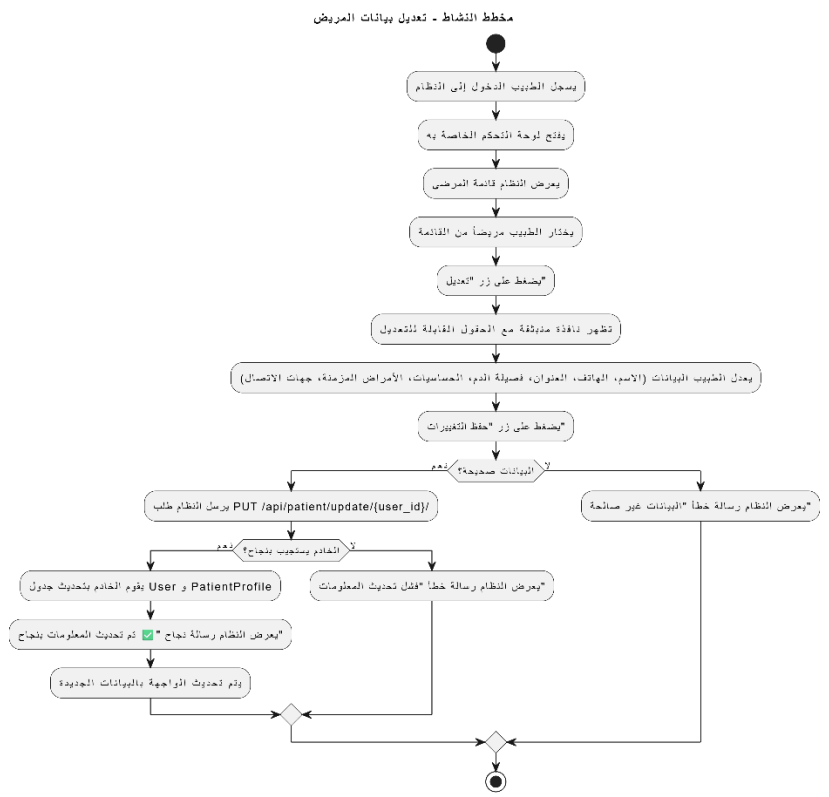



مخطط النشاط - إضافة عملية جراحية
يسجل الطبيب الدخول إلى النظام
يفتح صفحة المريض
"يضغط على زر "عمليات
يعرض النظام قائمة العمليات الحالية
"يضغط الطبيب على زر "إضافة عملية جديدة
تظهر نافذة منبثقة (اسم العملية، التاريخ، المستشفى، الملاحظات، الأدوية)
يدخل الطبيب البيانات
"يضغط على زر "حفظ
اسم العملية فارغ؟
نعم
"عرض رسالة خطأ "اسم العملية مطلوب
يرسل النظام طلب POST /api/surgeries/add/
الخادم رستجيب بنجاح؟
نعم
لا
يحفظ النظام العملية في جدول Surgery (status='scheduled')
"يعرض النظام رسالة خطأ "فشل إضافة العملية
"يعرض النظام رسالة نجاح " ✅ تم إضافة العملية بنجاح
يتم تحديث قائمة العمليات في صفحة المريض

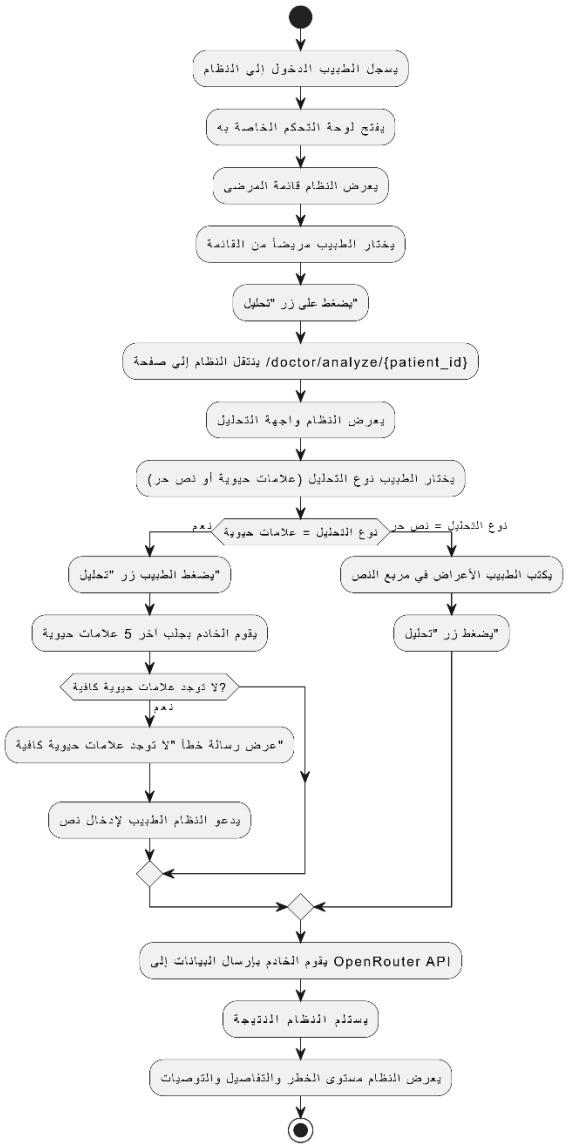
مخطط النشاط - إنهاء العملية الجراحية (مع إنشاء سجل طبي تلقائي)




مخطط النشاط - عرض تفاصيل المريض
يسجل الطبيب الدخول إلى النظام
يفتح لوحة التحكم الخاصة به
يعرض النظام قائمة المرضى
يختار الطبيب مريضاً من القائمة
"يضغط على زر "عرض
/doctor/patients/{patient_id} ينتقل النظام إلى صفحة
:يقوم النظام بجلب جميع بيانات المريض
- الملف الشخصي
- السجلات الطبية
- العلامات الحيوية
- العمليات الجراحية
- تحليلات الذكاء الاصطناعي
تعرض الواجهة تبويبات منفصلة للمعلومات
يستعرض الطبيب بيانات المريض

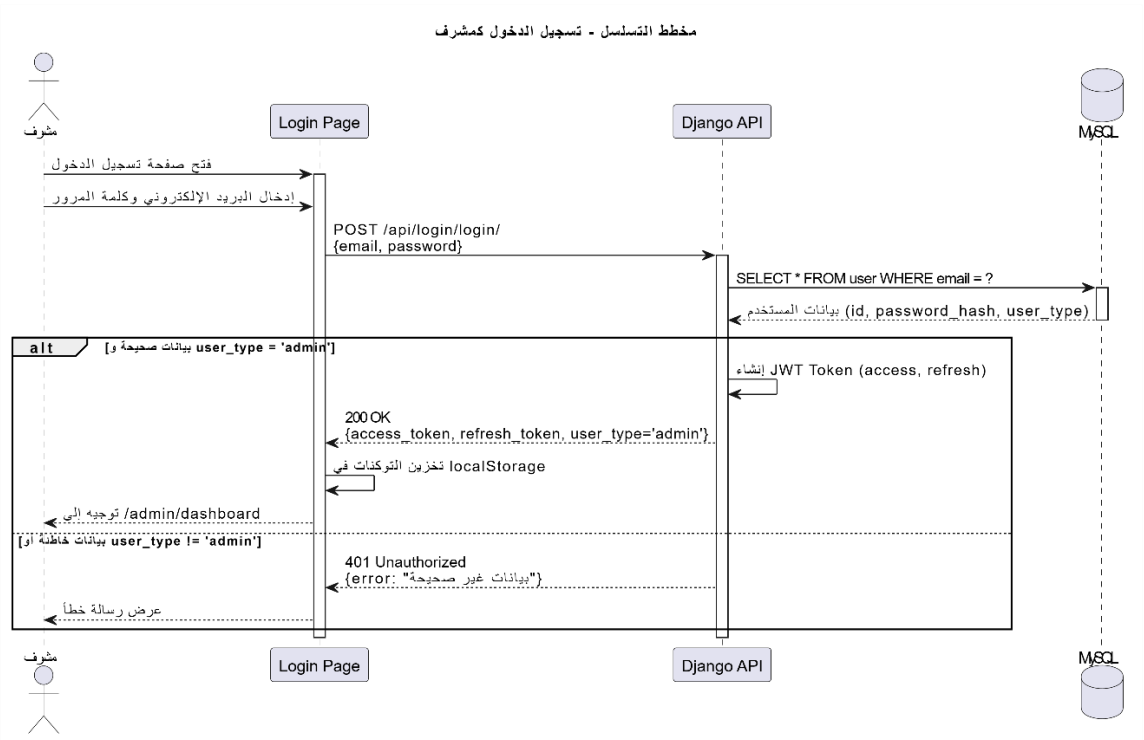

مخطط النشاط - تعديل بيانات المريض
يسجل الطبيب الدخول إلى النظام
يفتح لوحة التحكم الخاصة به
يعرض النظام قائمة المرضى
يختار الطبيب مريضاً من القائمة
"يضغط على زر "تعديل
تظهر نافذة منبثقة مع الحقول القابلة للتعديل
يعدل الطبيب البيانات (الاسم، الهاتف، العنوان، فصيلة الدم، الحساسيات، الأمراض المزمنة، جهات الاتصال)
"يضغط على زر "حفظ التغييرات
البيانات صحيحة؟
نعم
لا
PUT /api/patient/update/{user_id}/ يرسل النظام طلب
"يعرض النظام رسالة خطأ "البيانات غير صالحة
الخادم يستجيب بنجاح؟
نعم
لا
PatientProfile و User يقوم الخادم بتحديث جدول
"يعرض النظام رسالة خطأ "فشل تحديث المعلومات
"يعرض النظام رسالة نجاح " ✅ تم تحديث المعلومات بنجاح
يتم تحديث الواجهة بالبيانات الجديدة

مخطط النشاط - تحليل مخاطر المريض بالذكاء الاصطناعي (من الطبيب)

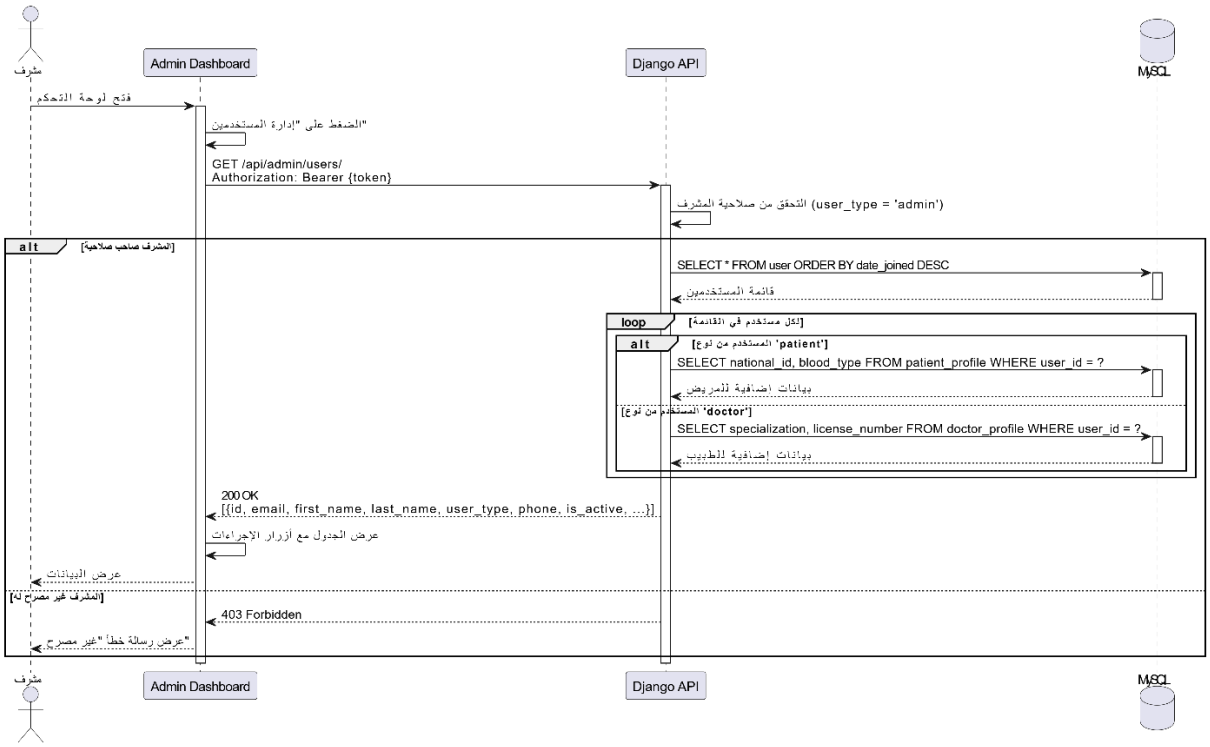


Sequence Diagram

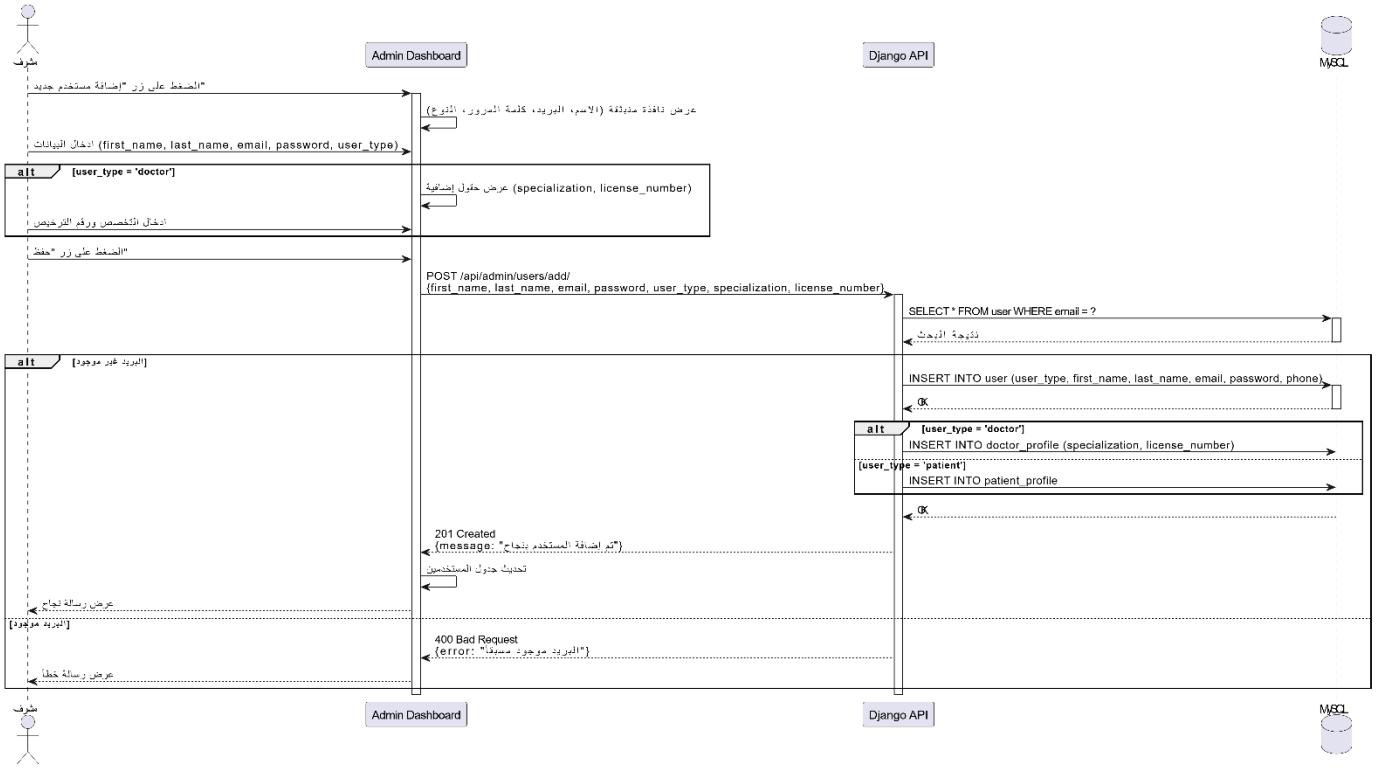
مخطط التسلسل - تسجيل الدخول كمشرف



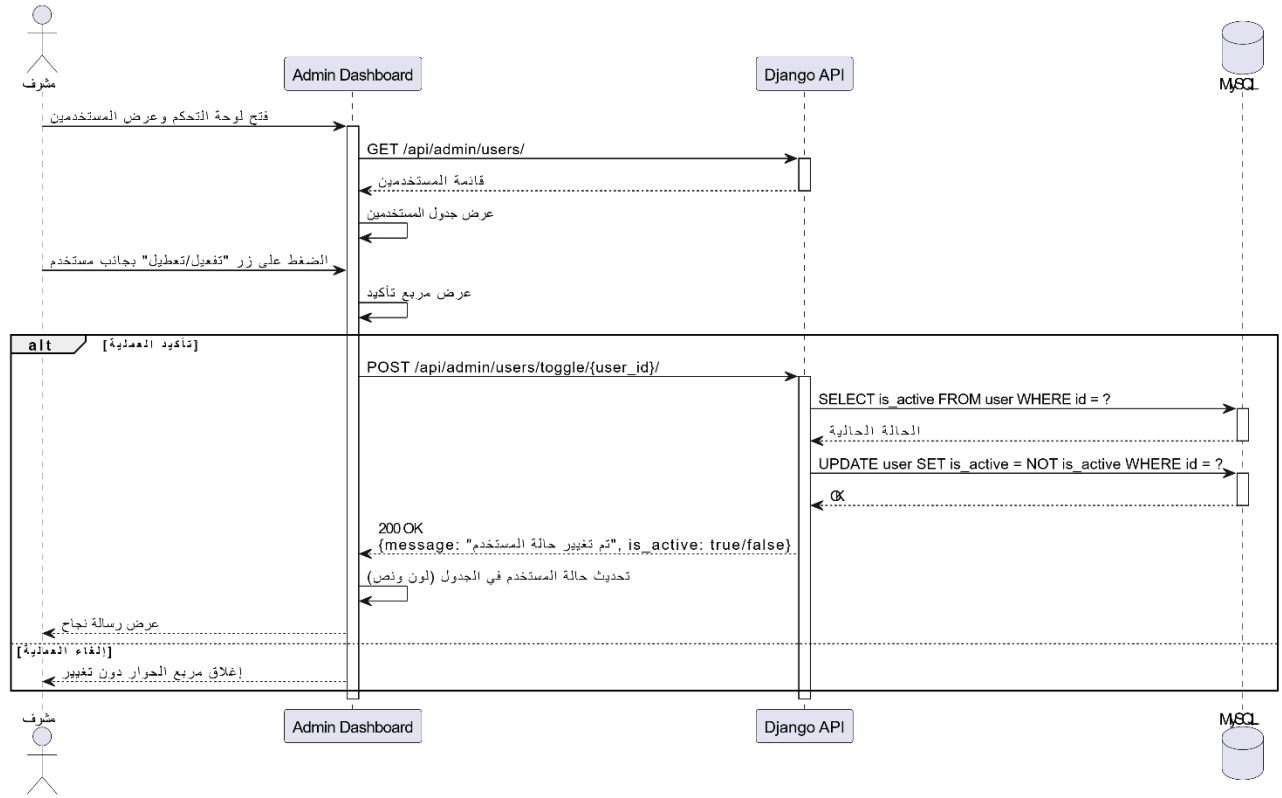
مخطط التسلسل - عرض قائمة المستخدمين (للمشرف)



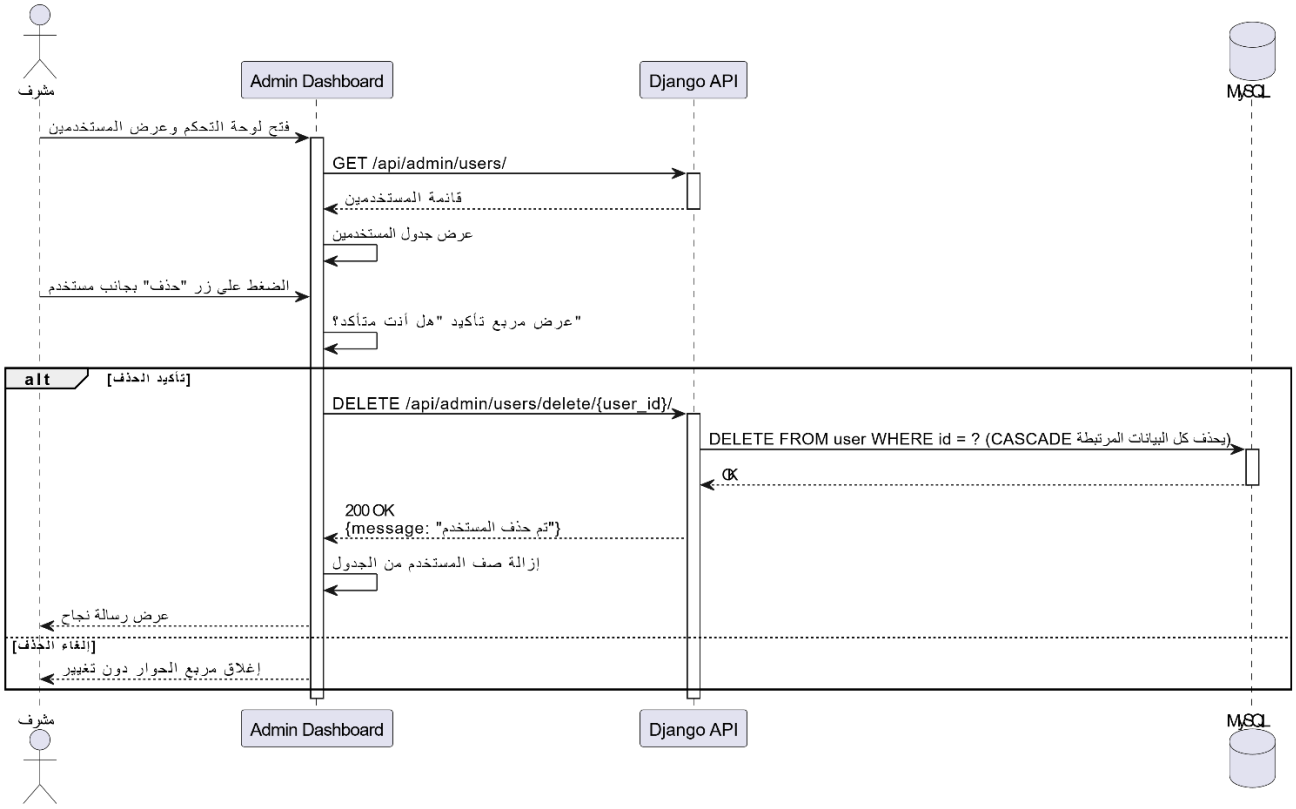
مخطط التسلسل - إضافة مستخدم جديد (للمشرف)

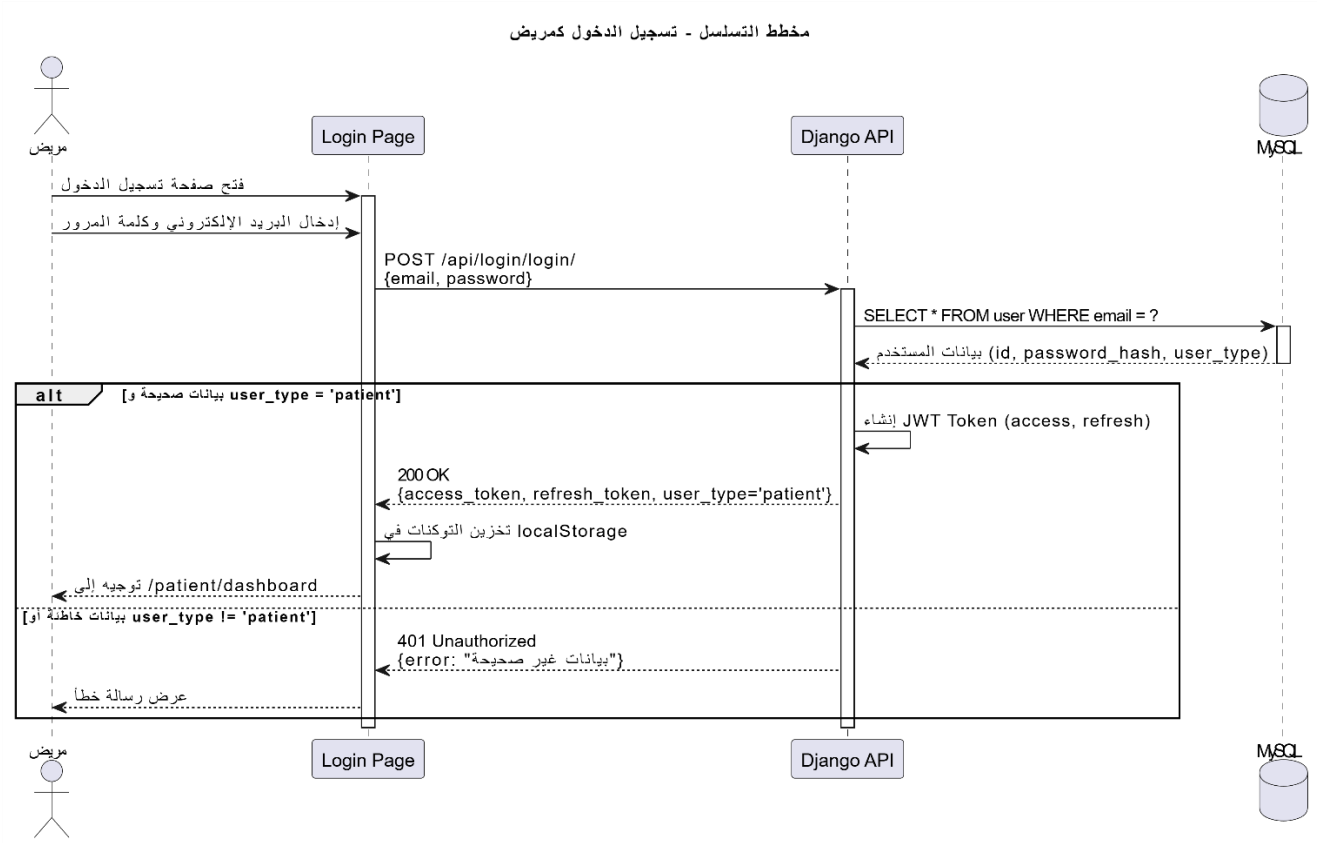


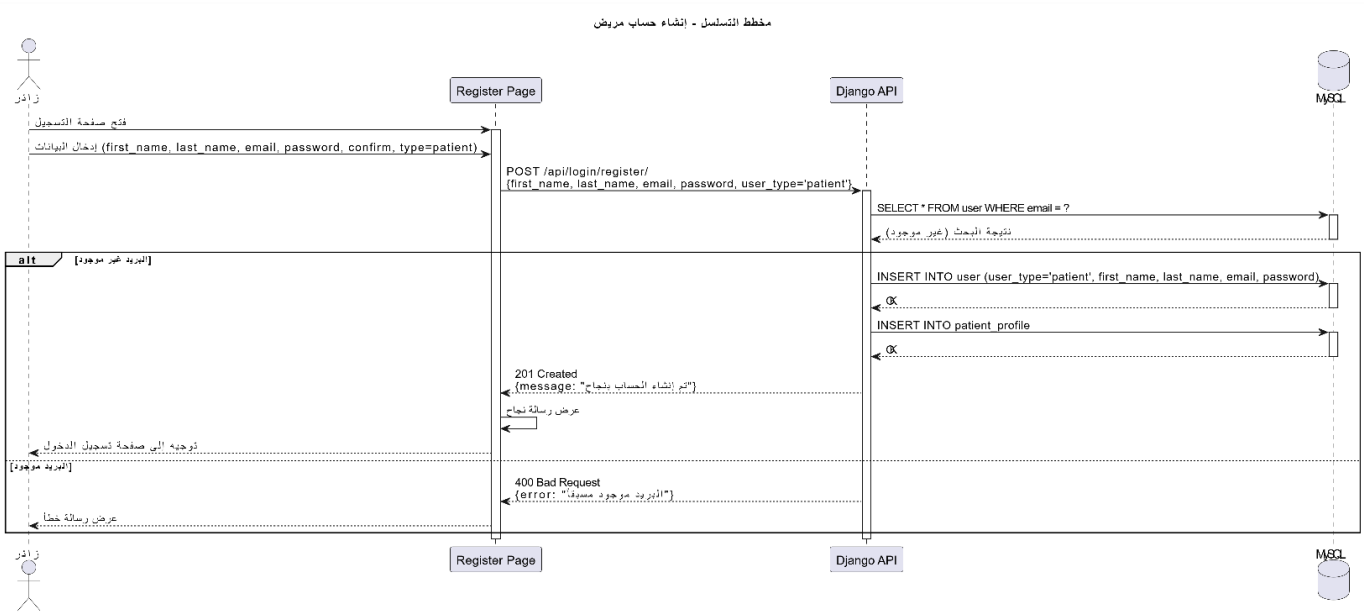
مخطط التسلسل - تفعيل/تعطيل مستخدم (للمشرف)



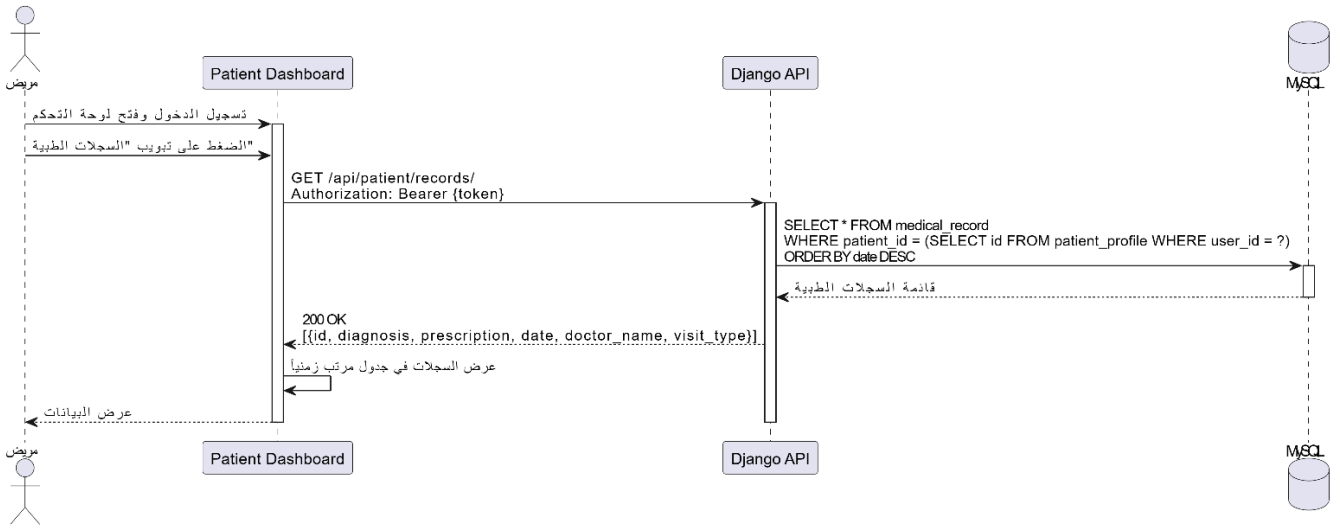
مخطط التسلسل - حذف مستخدم (للمشرف)



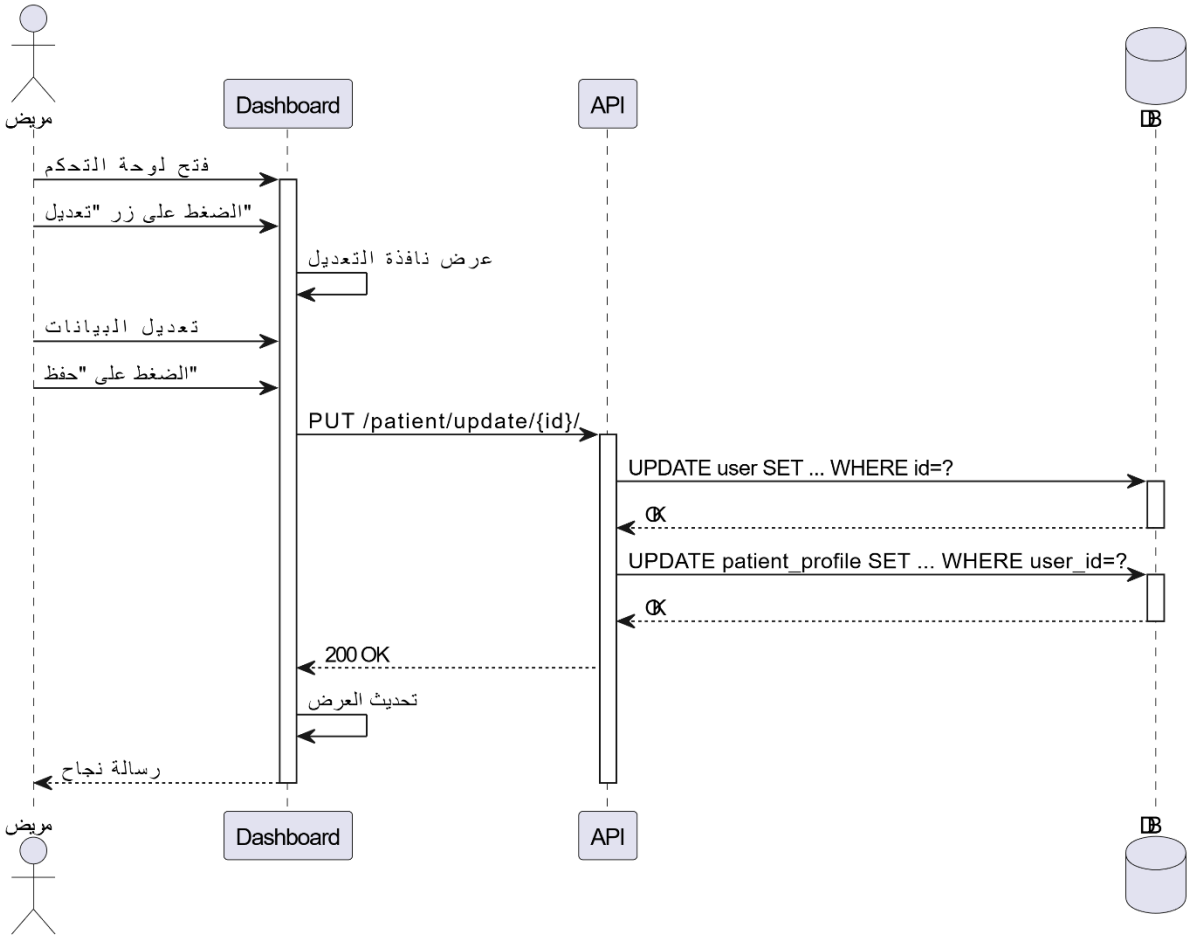

مخطط التسلسل - تسجيل الدخول كمريض
مريض
Login Page
Django API
MySQL
فتح صفحة تسجيل الدخول
إدخال البريد الإلكتروني وكلمة المرور
POST /api/login/login/
{email, password}
SELECT * FROM user WHERE email = ?
بيانات المستخدم (id, password_hash, user_type)
alt
[بيانات صحيحة و user_type = 'patient']
إنشاء JWT Token (access, refresh)
200 OK
{access_token, refresh_token, user_type='patient'}
تخزين التوكنات في localStorage
توجيه إلى /patient/dashboard
[بيانات خاطئة أو user_type != 'patient']
401 Unauthorized
{error: "بيانات غير صحيحة"}
عرض رسالة خطأ
مريض
Login Page
Django API
MySQL


مخطط التسلسل - إنشاء حساب مريض
زائر
Register Page
Django API
MySQL
فتح صفحة التسجيل
إدخال البيانات (first_name, last_name, email, password, confirm, type=patient)
POST /api/login/register/
{first_name, last_name, email, password, user_type='patient'}
SELECT * FROM user WHERE email = ?
نتيجة البحث (غير موجود)
alt
[البريد غير موجود]
INSERT INTO user (user_type='patient', first_name, last_name, email, password)
OK
INSERT INTO patient_profile
OK
201 Created
{message: "تم إنشاء الحساب بنجاح"}
عرض رسالة نجاح
توجيه إلى صفحة تسجيل الدخول
[البريد موجود]
400 Bad Request
{error: "البريد موجود مسبقاً"}
عرض رسالة خطأ
زائر
Register Page
Django API
MySQL

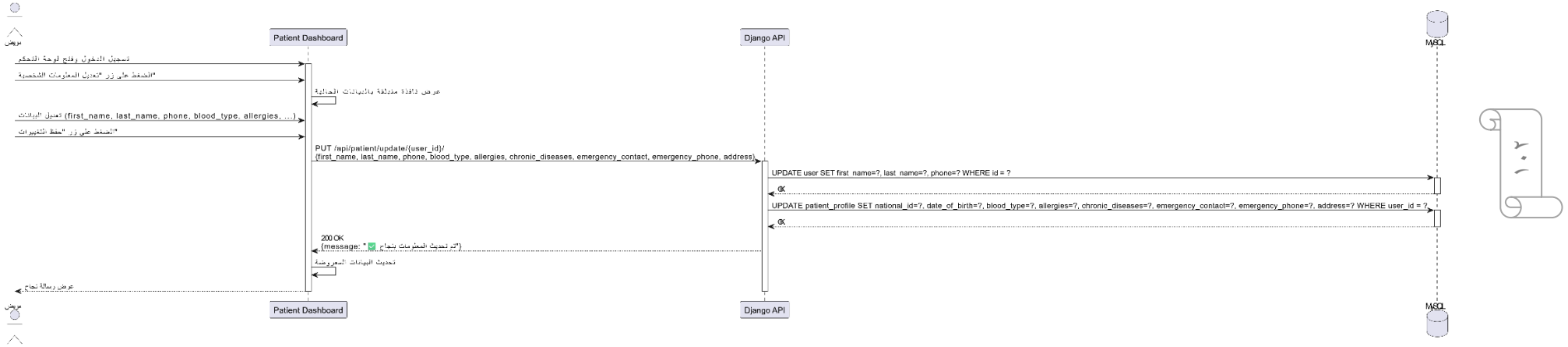
مخطط التسلسل - عرض السجلات الطبية

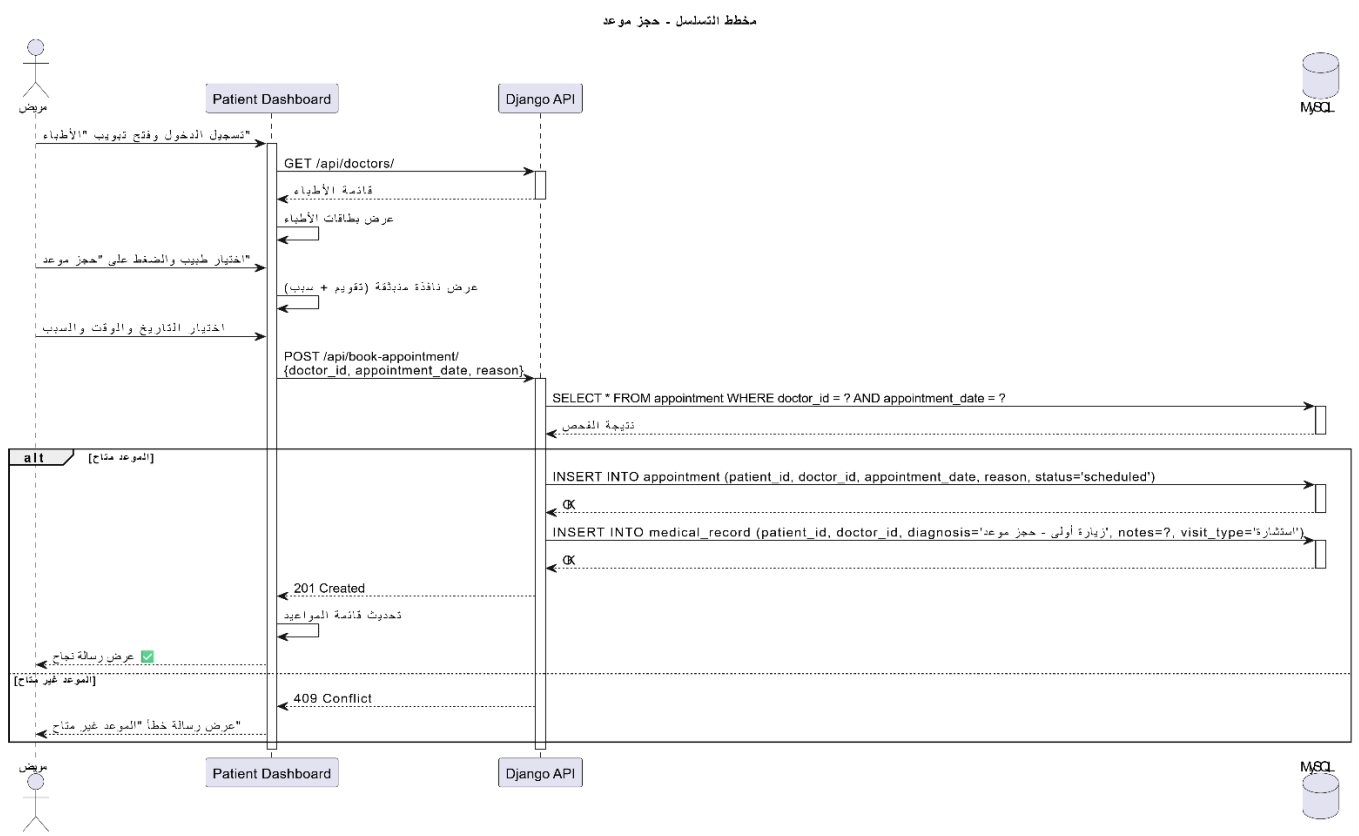


مخطط التسلسل - تعديل الملف الشخصي للمريض

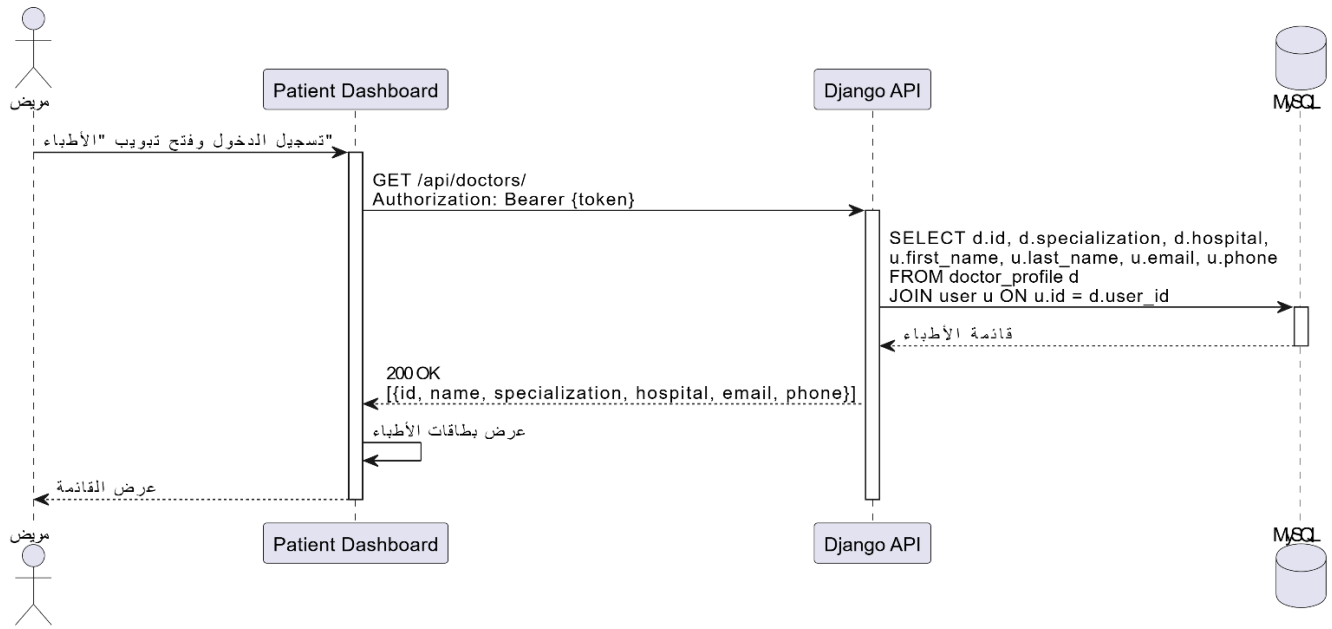


مخطط التسلسل - تعديل الملف الشخصي للمريض

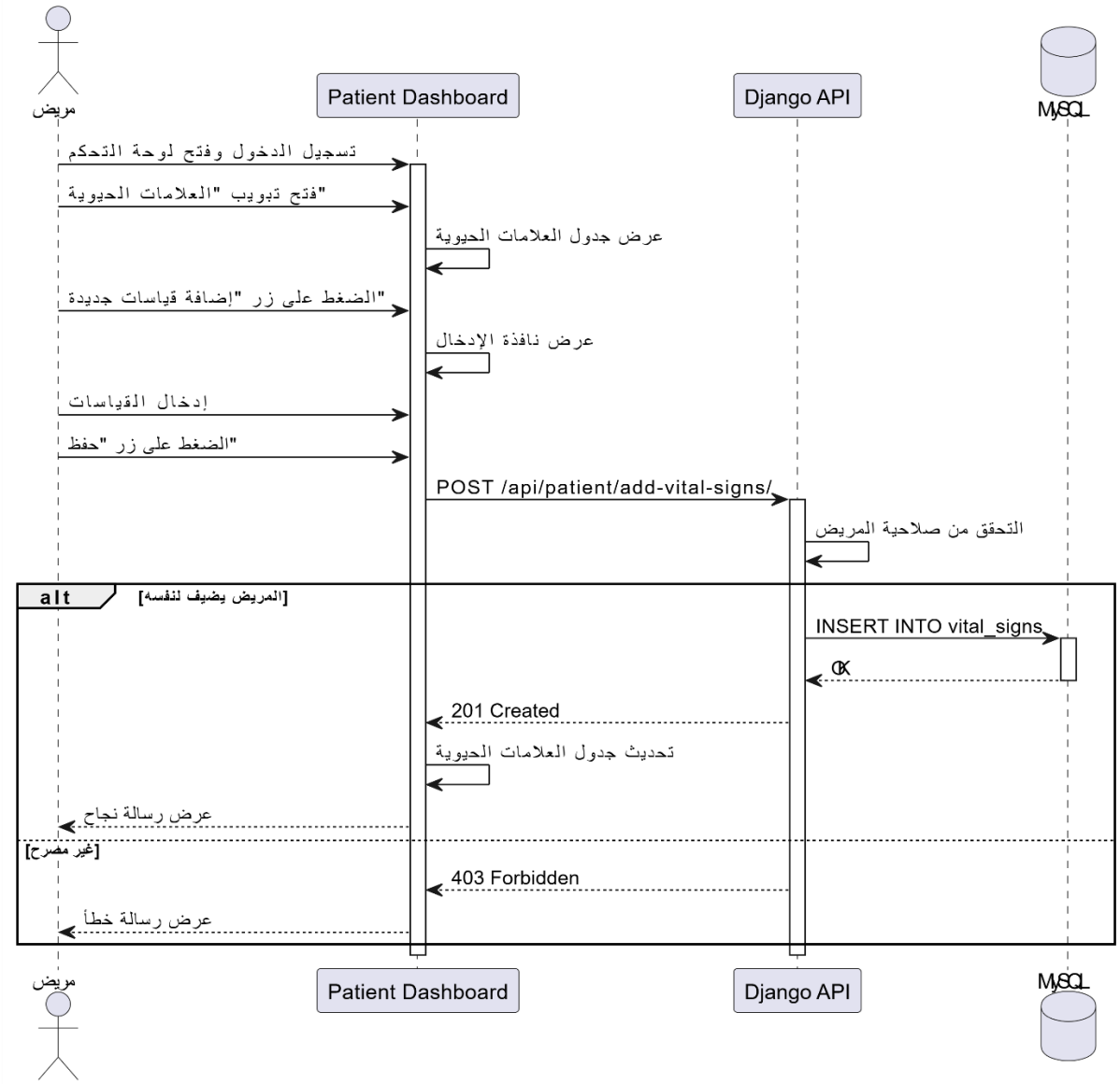




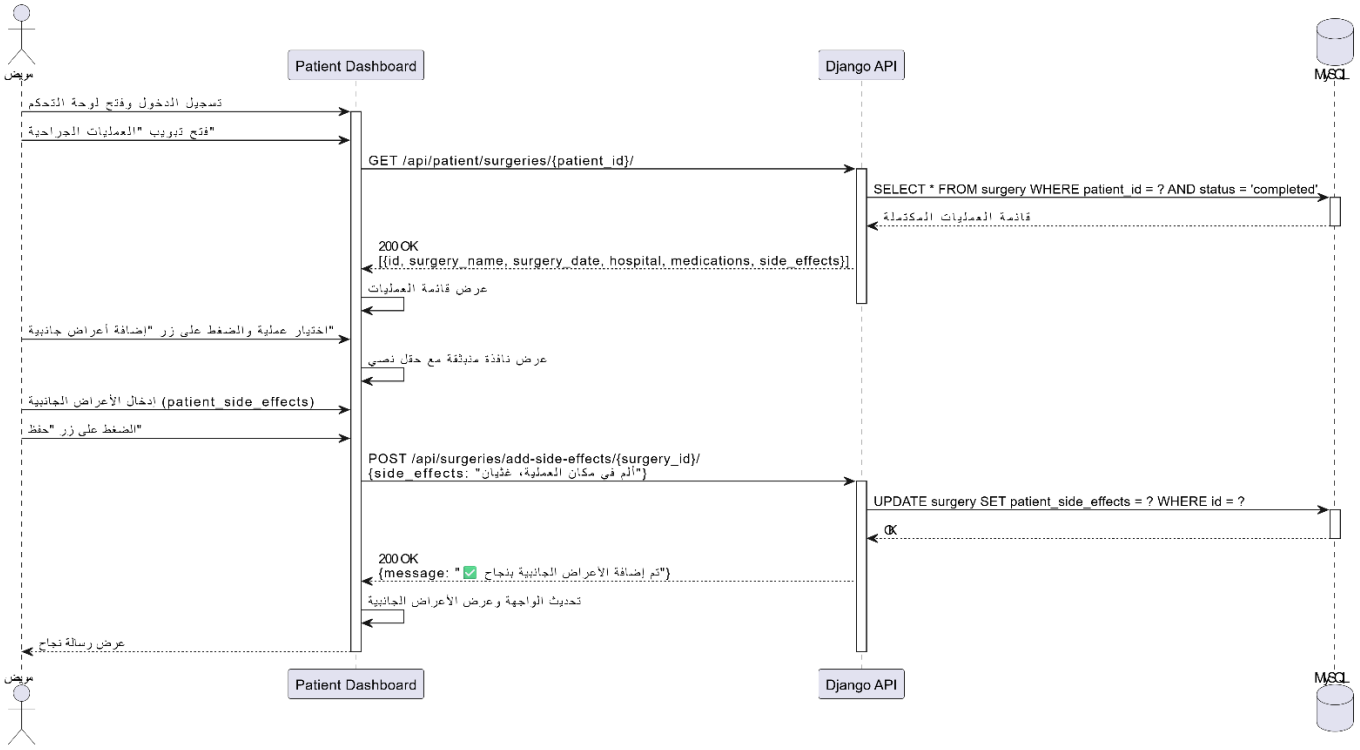
مخطط التسلسل - عرض قائمة الأطباء



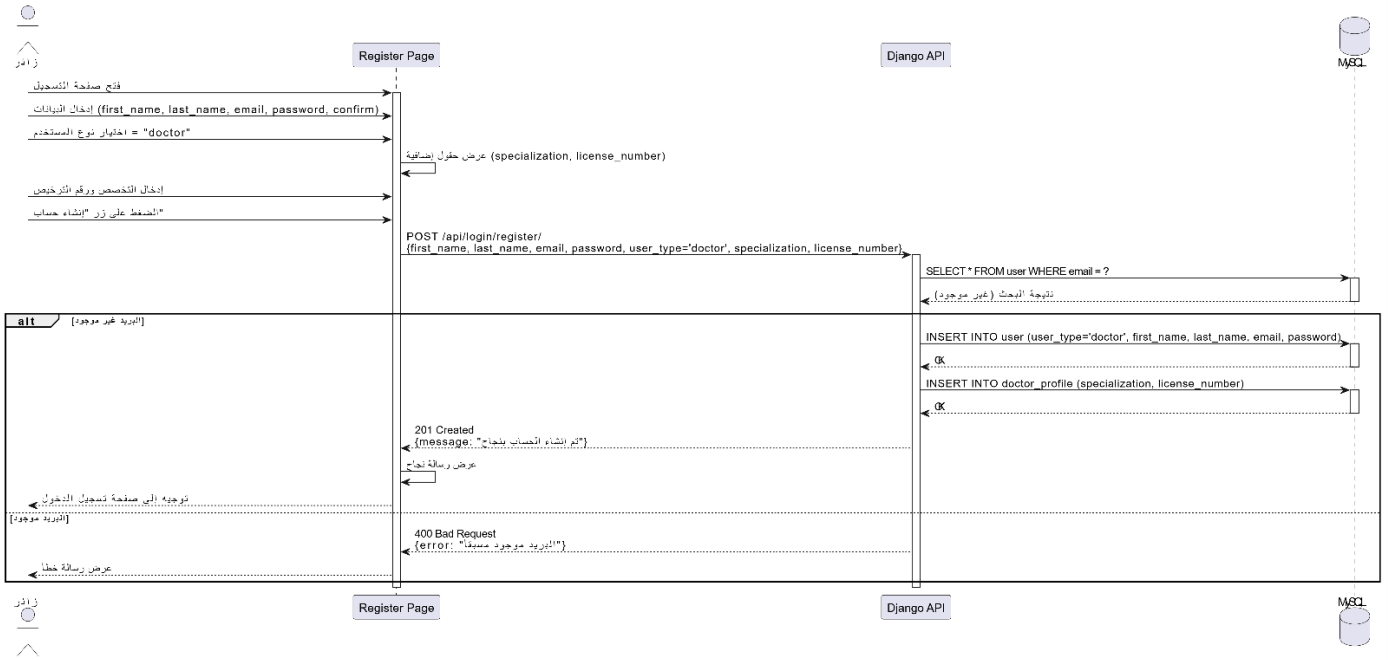
مخطط التسلسل - إضافة علامات حيوية



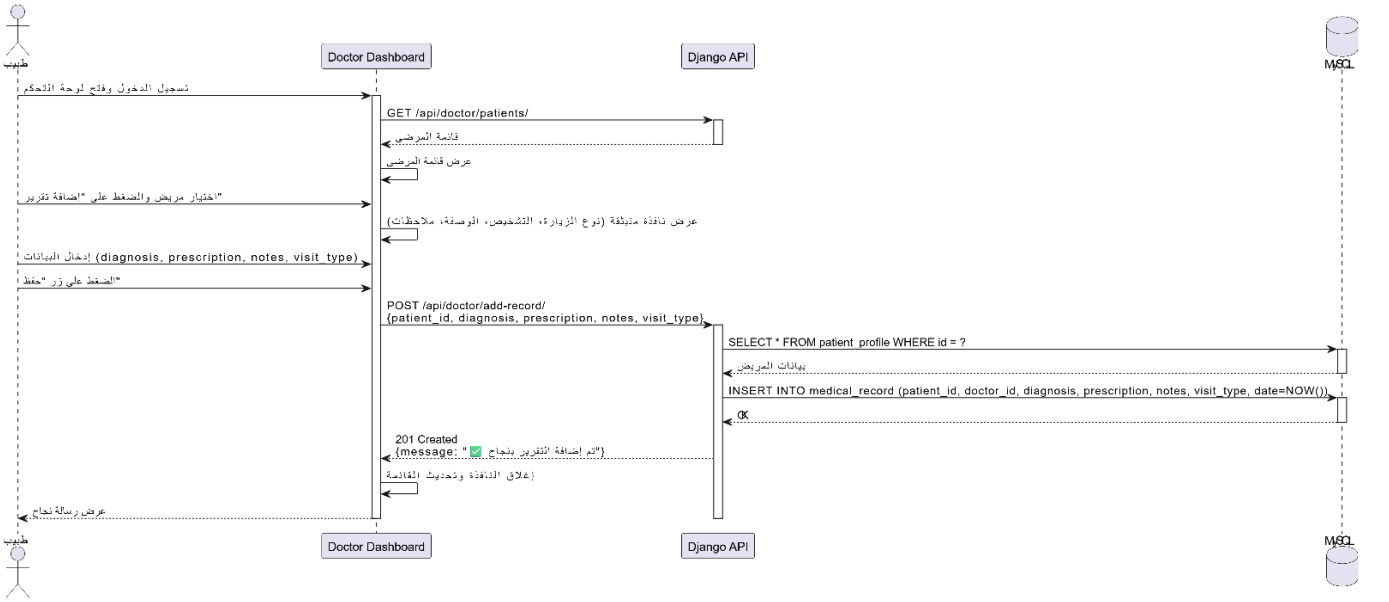
مخطط التسلسل - إضافة أعراض جانبية للعملية الجراحية (من قبل المريض)



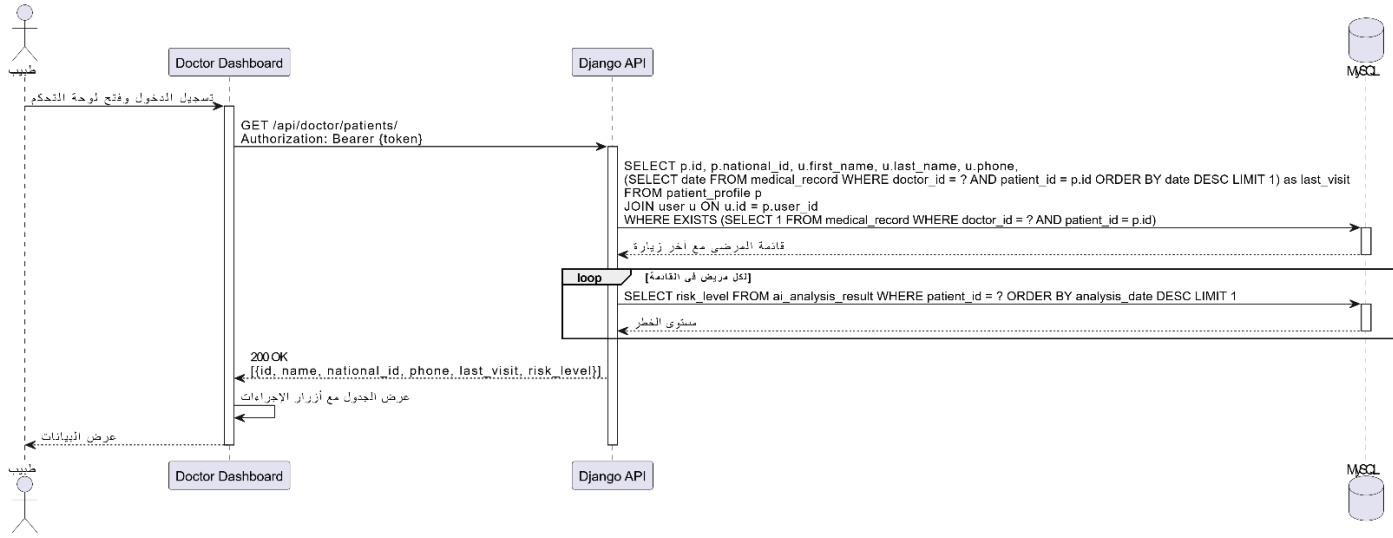
مخطط التسلسل - إنشاء حساب طبيب مع تخصص وترخيص



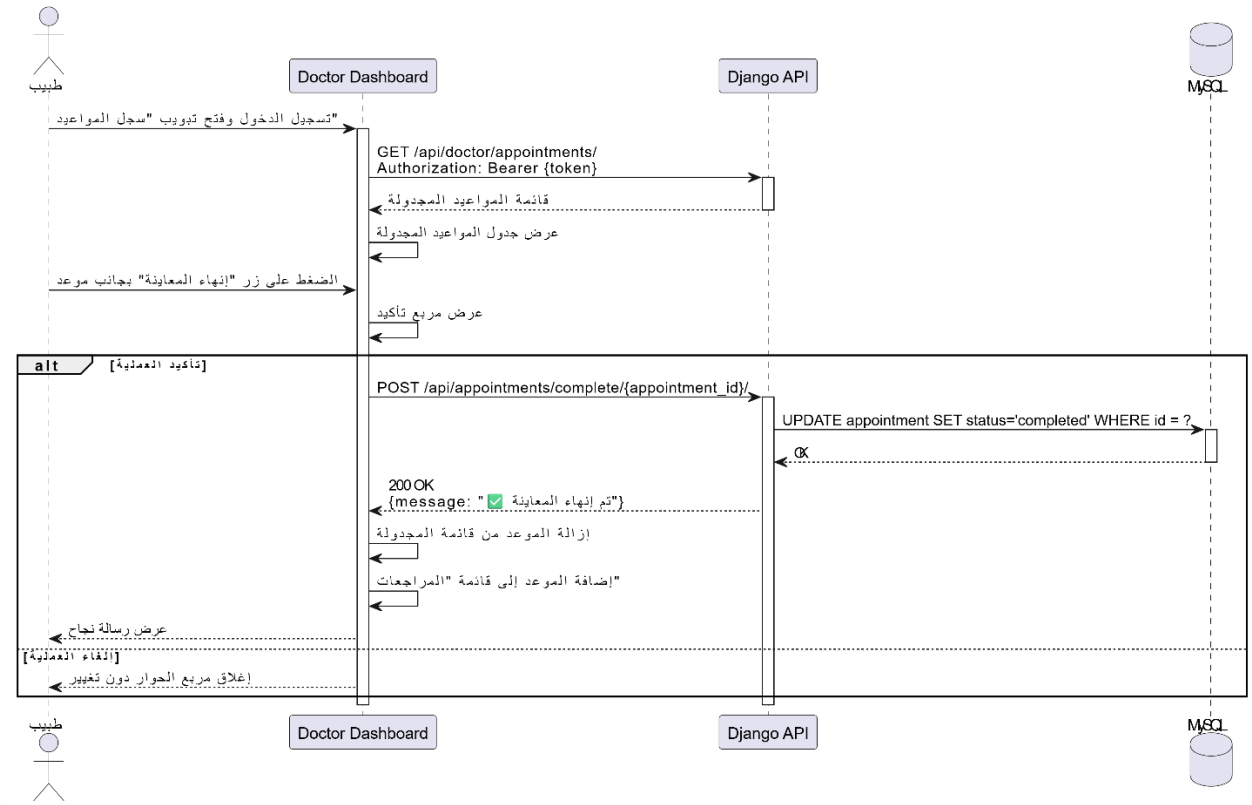
مخطط التسلسل - إضافة سجل طبي



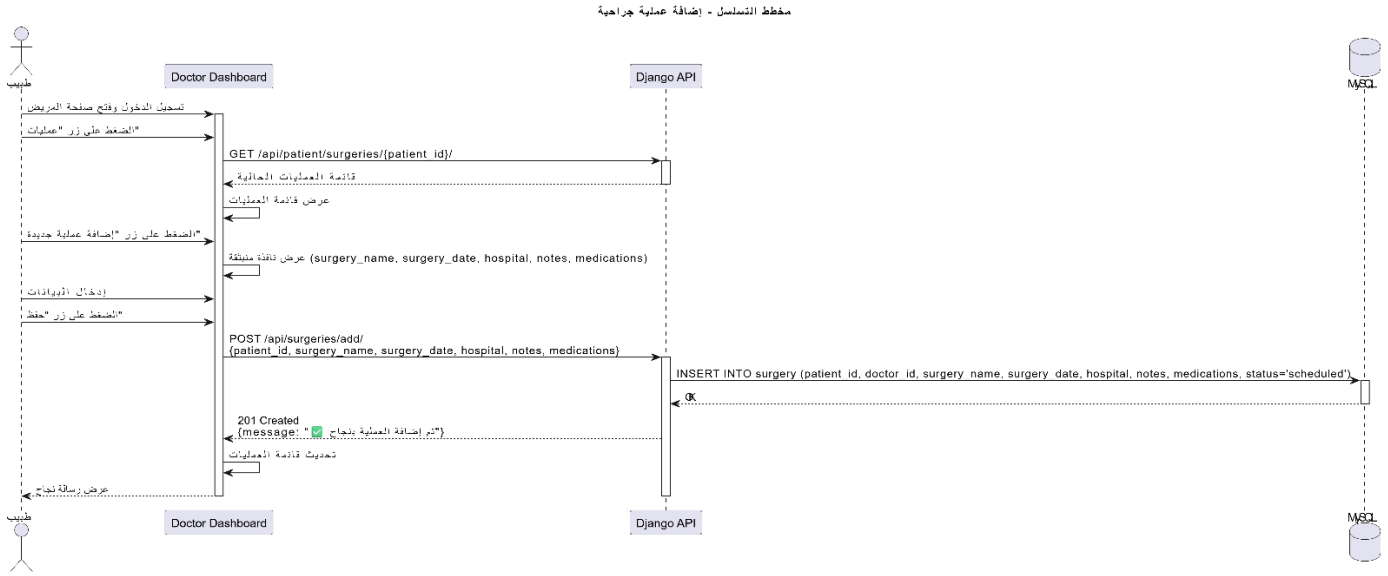
مخطط التسلسل - عرض قائمة المرضى



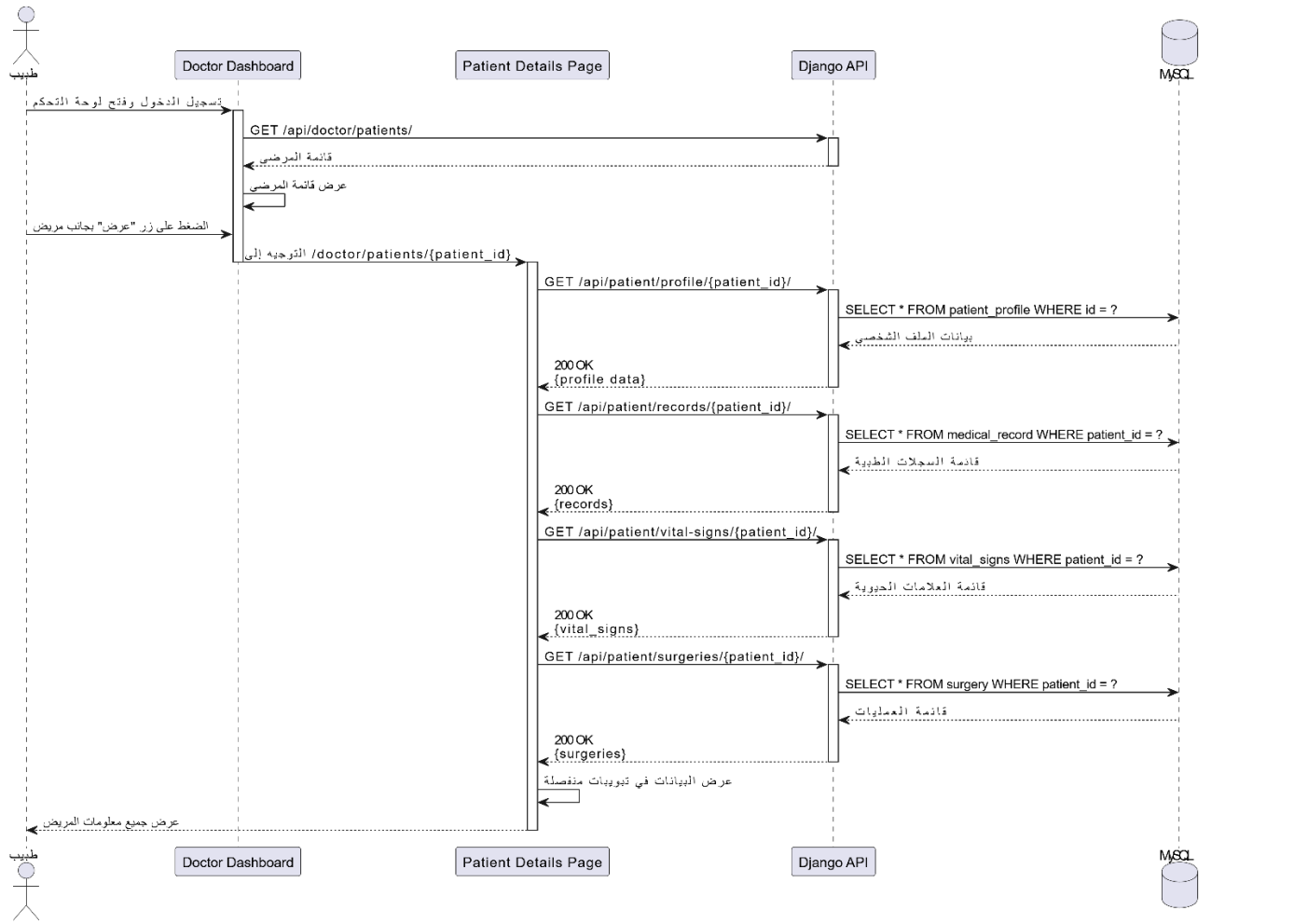
مخطط التسلسل - إنهاء المعاينة (نقل إلى المراجعات)



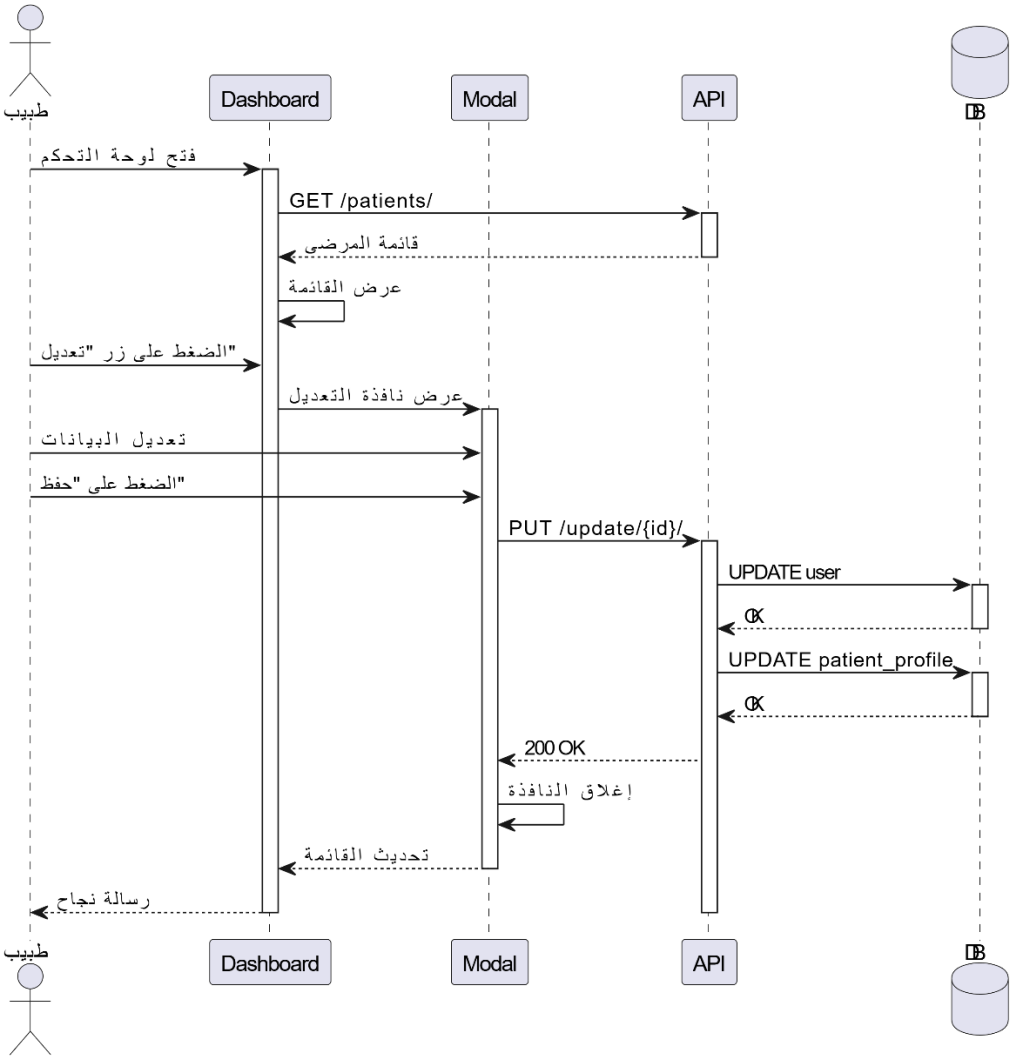
مخطط التسلسل - إضافة عملية جراحية



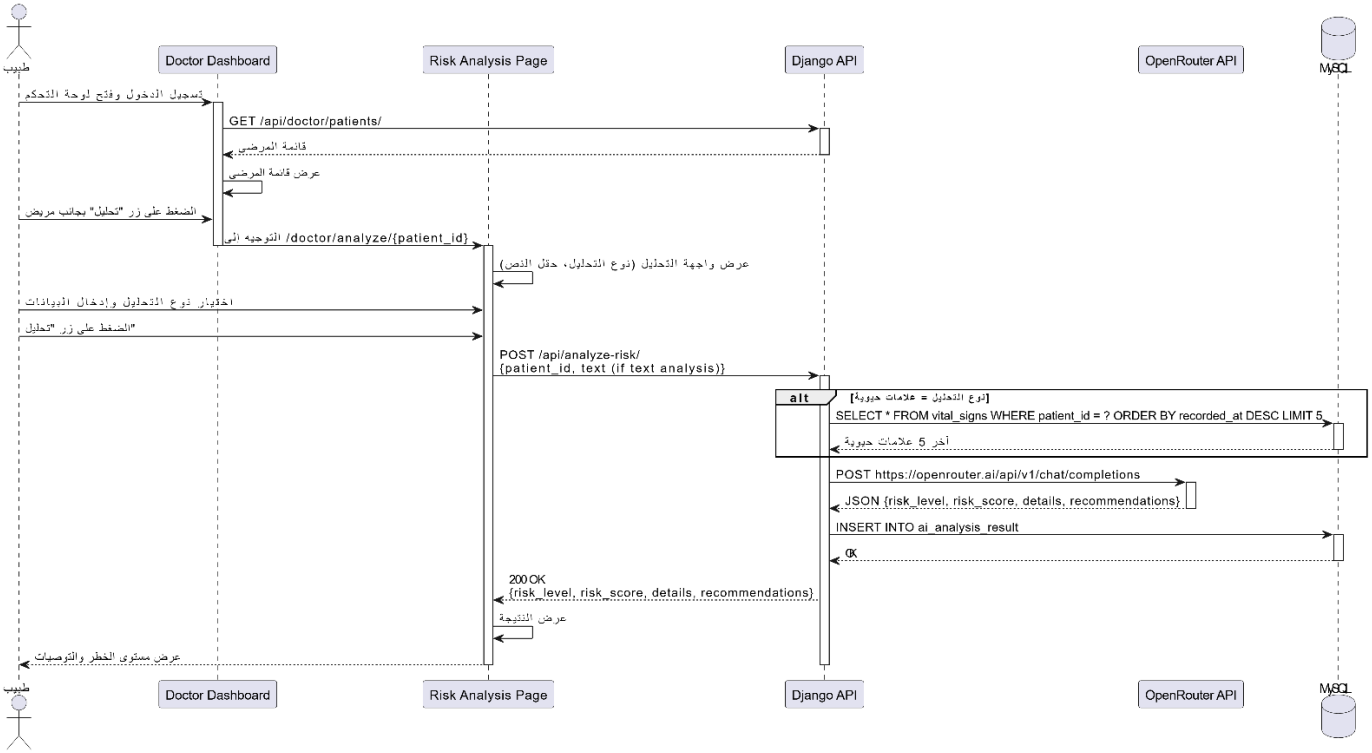
مخطط التسلسل - عرض تفاصيل المريض



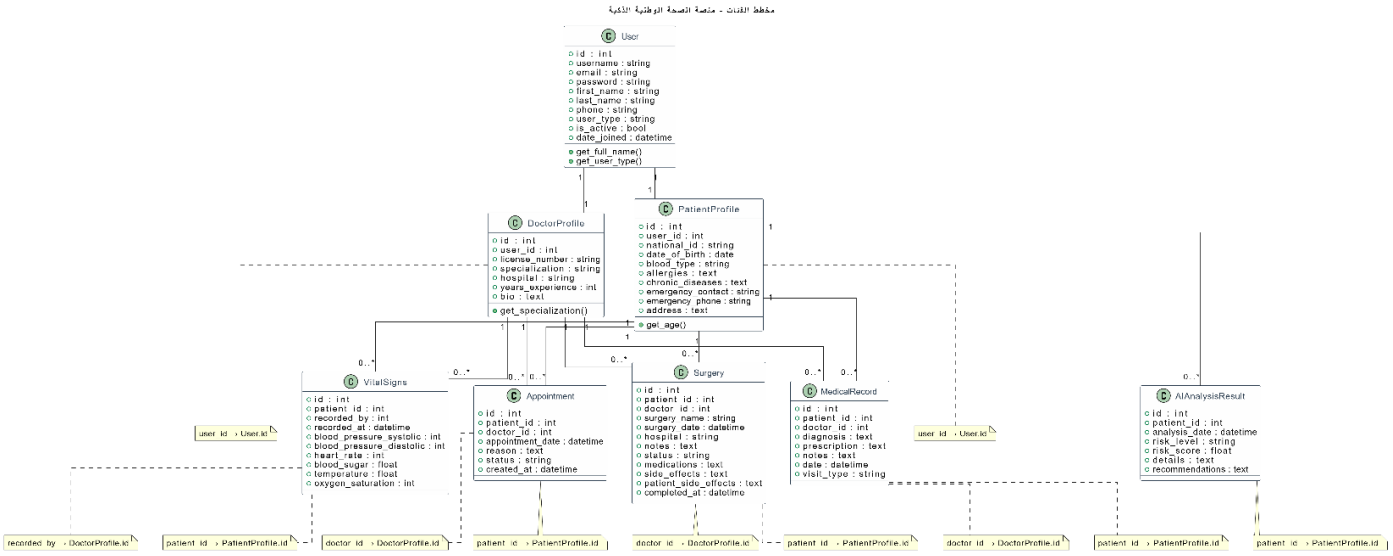
مخطط التسلسل - تعديل بيانات المريض



مخطط التسلسل - تحليل مخاطر المريض بالذكاء الاصطناعي (من الطبيب)



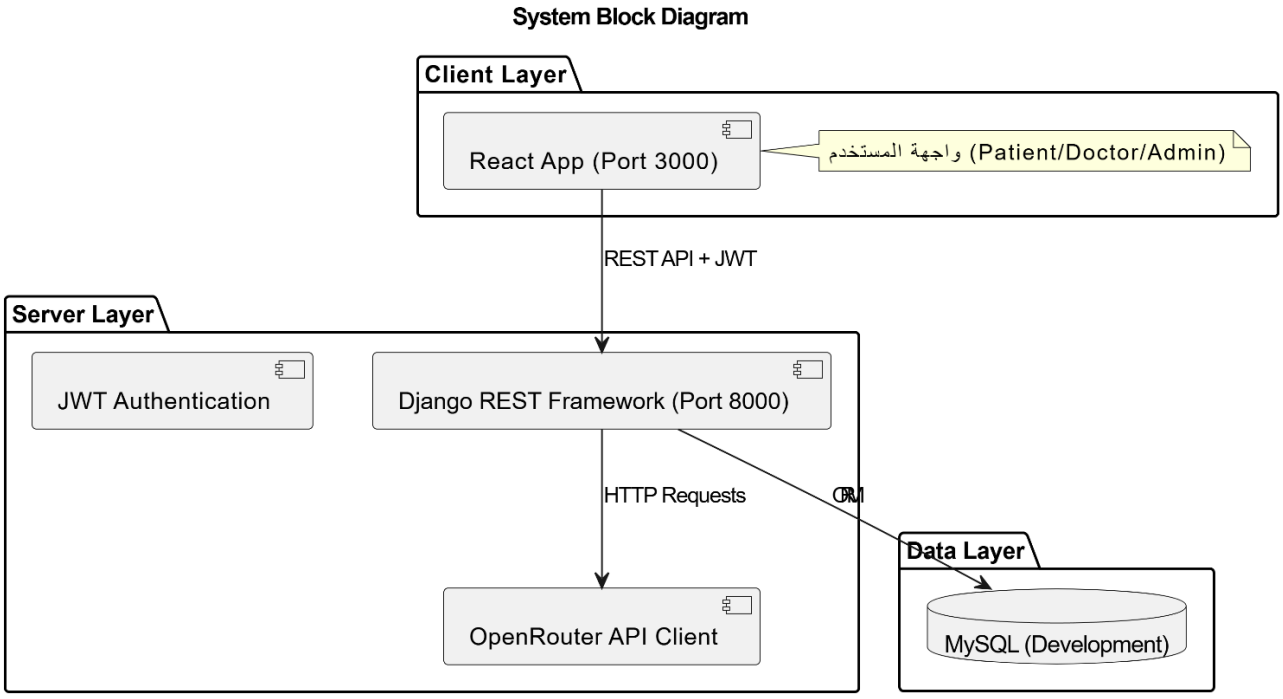
CLASS DIAGRAM



Chapter 5:

Architectural Design

5-1 High-Level Design



Description:

The system follows a three-tier architecture:

- ***Client Layer:*** *React.js application running on port 3000, responsible for rendering user interfaces for patients, doctors, and admins.*
- ***Server Layer:*** *Django REST Framework running on port 8000, handling business logic, authentication (JWT), and API requests.*
- ***Data Layer:*** *MySQL database for persistent storage of users, medical records, appointments, and other data.*

5-2 Architectural Design Pattern

The system employs a combination of architectural patterns:

1. Client-Server Architecture

- **Client:** React frontend (Port 3000)
- **Server:** Django backend (Port 8000)
- **Communication:** REST API with JWT authentication

Advantages:

- Separation of concerns between presentation and business logic
- Independent scalability of frontend and backend
- Reusability of backend APIs for future mobile applications

2. MVC Pattern (Django)

Component	Implementation	Responsibility
Model	*models.py*	*Data structure, database schema, business logic*
View	*views.py (DRF)*	*Request handling, API responses, business logic*
Controller	*urls.py + DRF Routers*	*URL routing, request mapping*

Note: Django follows MVT (Model-View-Template) by default, but since we use React as a separate frontend, Django views act as controllers (REST APIs) rather than traditional templates.

3. Component-Based Architecture (React)

React components are organized hierarchically:

App.js

```
├── Login.js
├── Register.js
├── PatientDashboard.js
│   ├── OverviewTab
│   ├── RecordsTab
│   ├── VitalSignsTab
│   └── DoctorsTab
├── DoctorDashboard.js
│   ├── PatientsList
│   ├── AppointmentsTab
│   └── CompletedTab
└── AdminDashboard.js
    ├── UsersTable
    └── StatsCards
```

Benefits:

- *Reusability across the application*
- *Easier testing and maintenance*
- *Clear separation of concerns*

4. REST API Architecture

All communication between frontend and backend follows REST principles:

HTTP Method	Endpoint Example	Purpose
POST	*/api/login/*	*User authentication*
GET	*/api/patient/records/*	*Fetch medical records*
POST	*/api/doctor/add-record/*	*Create medical record*
PUT	*/api/patient/update/{id}/*	*Update profile*
DELETE	*/api/admin/users/delete/{id}/*	*Delete user*

5-3 External Services Integration

OpenRouter API

- ***Purpose:*** *AI-powered symptom and vital signs analysis*
- ***Communication:*** *HTTP requests from Django backend*
- ***Data format:*** *JSON*
- ***Fallback:*** *Built-in algorithm when API is unavailable*

5-4 Security Architecture

Layer	Security Measure
Authentication	*JWT (JSON Web Tokens)*
Password Storage	*PBKDF2 hashing (Django default)*
API Security	*Token-based authentication with Bearer header*
CORS	*Configured to allow only frontend origins*
SQL Injection	*Prevented by Django ORM*
XSS	*React's automatic escaping*

5-5 Database Design Summary

Table	Primary Key	Foreign Keys
User	*id*	-
PatientProfile	*id*	*user_id → User.id*
DoctorProfile	*id*	*user_id → User.id*
MedicalRecord	*id*	*patient_id → PatientProfile.id, doctor_id → DoctorProfile.id*
VitalSigns	*id*	*patient_id → PatientProfile.id*
AIAnalysisResult	*id*	*patient_id → PatientProfile.id*
Appointment	*id*	*patient_id → PatientProfile.id, doctor_id → DoctorProfile.id*
Surgery	id	patient_id → PatientProfile.id, doctor_id → DoctorProfile.id

---

5-6 Technology Stack Summary

Layer	Technology	Version
Frontend	*React.js*	*18.2.0*
Backend	*Django + DRF*	*4.2.7 + 3.14.0*
Database	*MySQL*	*8.0*
Authentication	*JWT (SimpleJWT)*	*5.3.0*
AI Service	*OpenRouter API*	*-*
HTTP Client	*Axios*	*1.6.0*

Chapter 6:

Execution Environment and Tools

6-1 Tools and Libraries Used

Domain	Technology / Library	Version
Backend	*Django*	*5.2.11*
Backend	*Django REST Framework*	*3.14.0*
Backend	*djangorestframework-simplejwt*	*5.3.0*
Backend	*django-cors-headers*	*4.3.1*
Backend	*drf-spectacular*	*0.27.0 (API documentation)*
Backend	*requests (AI integration)*	*2.32.5*
Backend	*python-dotenv*	*1.0.0*
Database	*SQLite (Development), MySQL (Production)*	-
Frontend	*React*	*18.2.0*
Frontend	*React Router DOM*	*6.18.0*
Frontend	*Axios*	*1.6.0*
AI	*OpenRouter API (Model: google/gemini-2.0-flash-lite-001)*	-
Diagramming	*PlantUML*	-

Chapter 7:

Implementation

7-1 Project File Structure

smart_health_platform/

├── manage.py

├── requirements.txt

├── .env

├── **accounts/**

│ ├── models.py (User, PatientProfile, DoctorProfile)

│ ├── views.py (RegisterView, EmailLoginView, UserProfileView, UpdatePatientProfileView)

│ ├── serializers.py

│ └── urls.py

├── **medical/**

│ ├── models.py (MedicalRecord, VitalSigns, AIAnalysisResult, Appointment, Surgery)

│ ├── views.py (All medical APIs)

│ ├── urls.py

│ ├── serializers.py

│ ├── openrouter_ai.py (OpenRouter connection)

├── **frontend/**

│ ├── src/

│ │ ├── components/ (Login, Register, PatientDashboard, DoctorDashboard, AdminDashboard, ...)

│ │ ├── api/axiosInstance.js

│ │ ├── App.js

│ │ ├── App.css

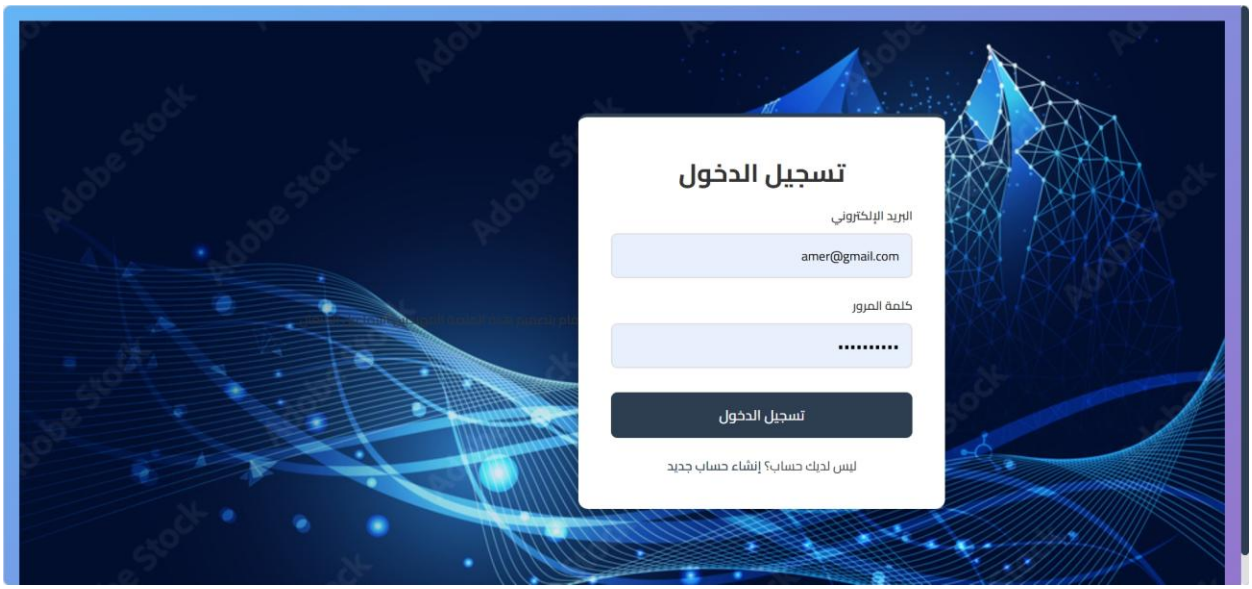
│ │ └── styles/

└── **smart_health_platform/ (settings.py, urls.py)**

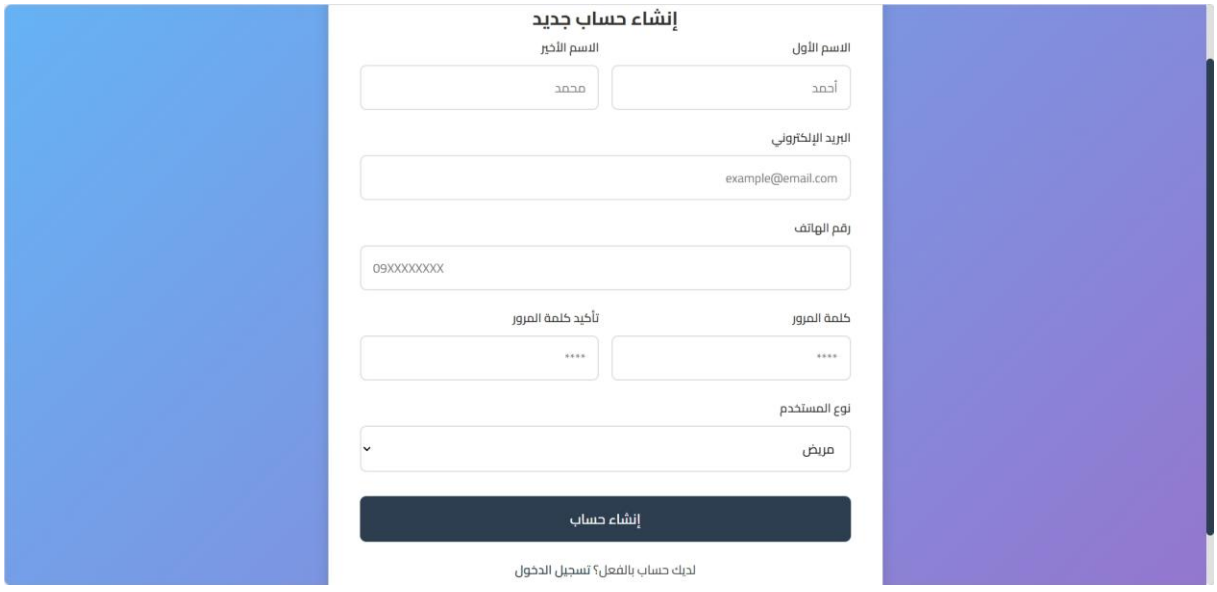
7-2 API Implementation

Endpoint	*Method*	*Function*
/api/login/login/	*POST*	*Login and return JWT*
/api/login/register/	*POST*	*Register (with specialization & license for doctors)*
/api/patient/records/	*GET*	*Patient's medical records*
/api/doctor/patients/	*GET*	*Doctor's patient list*
/api/doctor/add-record/	*POST*	*Add medical record*
/api/patient/add-vital-signs/	*POST*	*Add vital signs*
/api/analyze-risk/	*POST*	*AI symptom or vital signs analysis*
/api/doctors/	*GET*	*List all doctors (for patients)*
/api/book-appointment/	*POST*	*Book an appointment*
/api/doctor/appointments/	*GET*	*Doctor's scheduled appointments*
/api/doctor/completed-appointments/	*GET*	*Doctor's completed appointments (review history)*
/api/appointments/complete/{id}/	*POST*	*Complete an appointment*
/api/surgeries/add/	*POST*	*Add a surgery*
/api/surgeries/complete/{id}/	*POST*	*Complete surgery (auto-creates medical record)*
/api/patient/surgeries/{id}/	*GET*	*Patient's surgeries*
/api/surgeries/add-side-effects/{id}/	*POST*	*Add patient side effects after surgery*

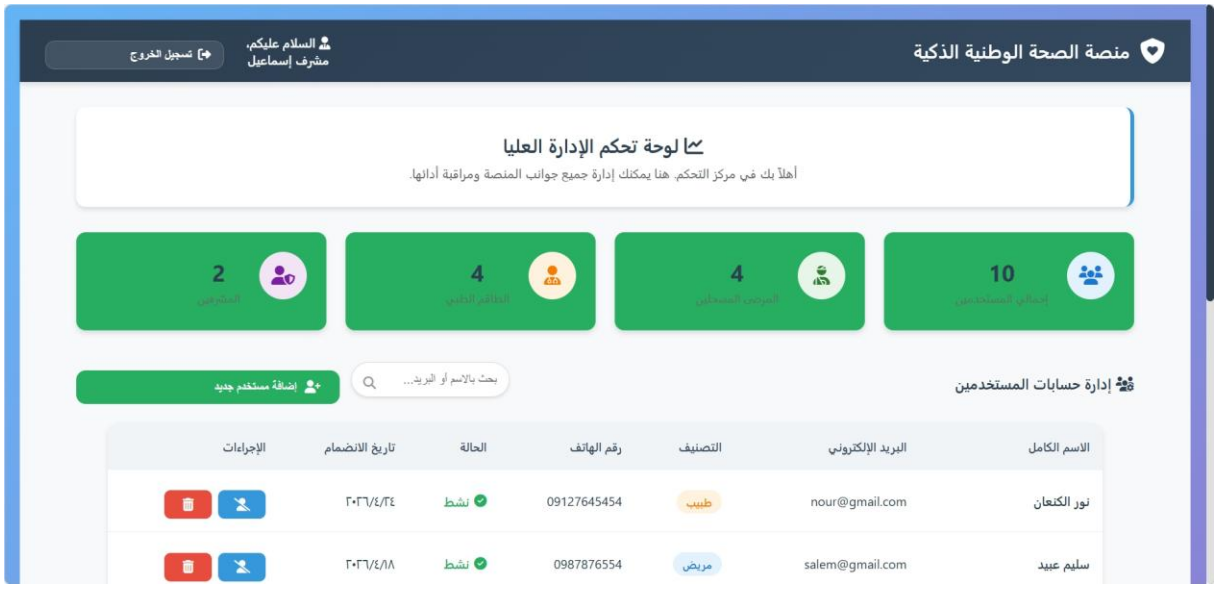
Login

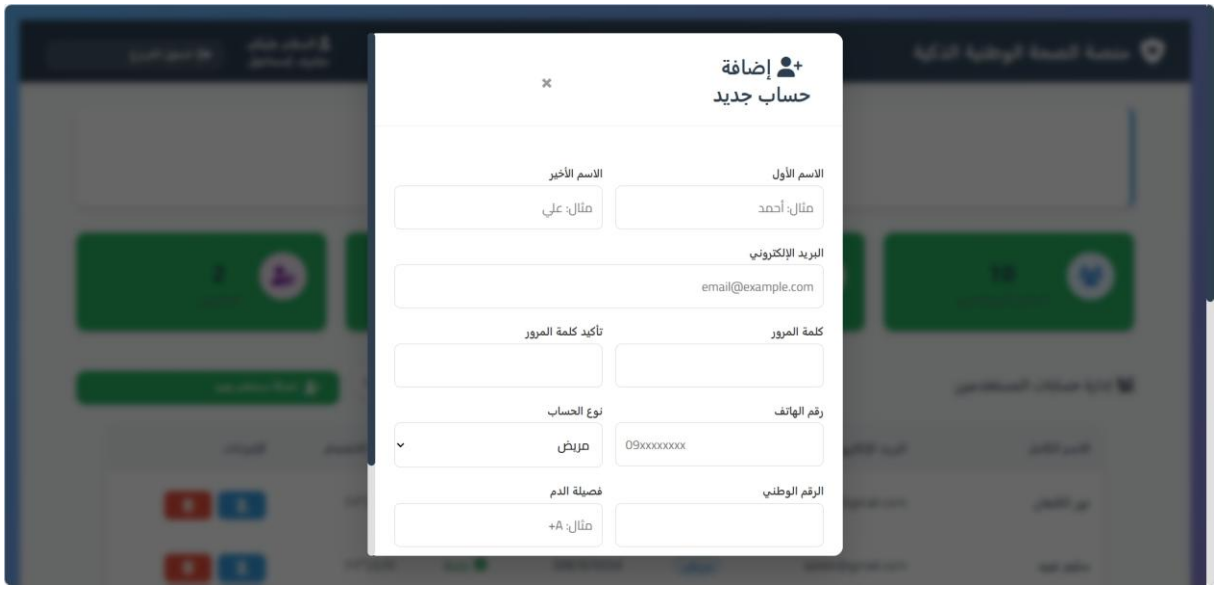


Registration

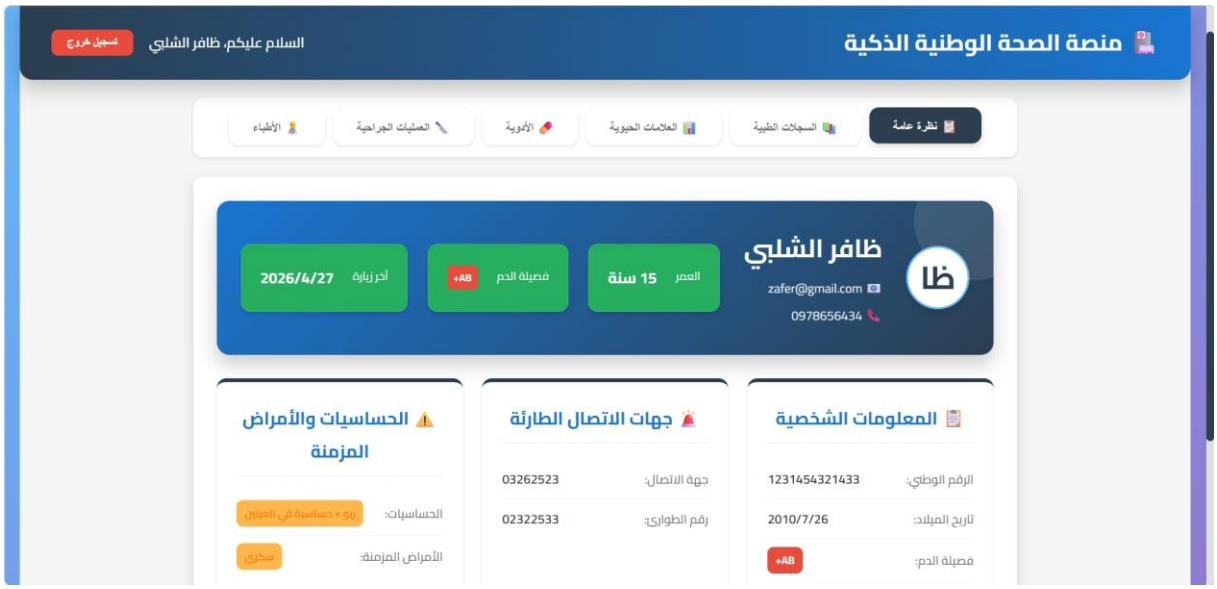


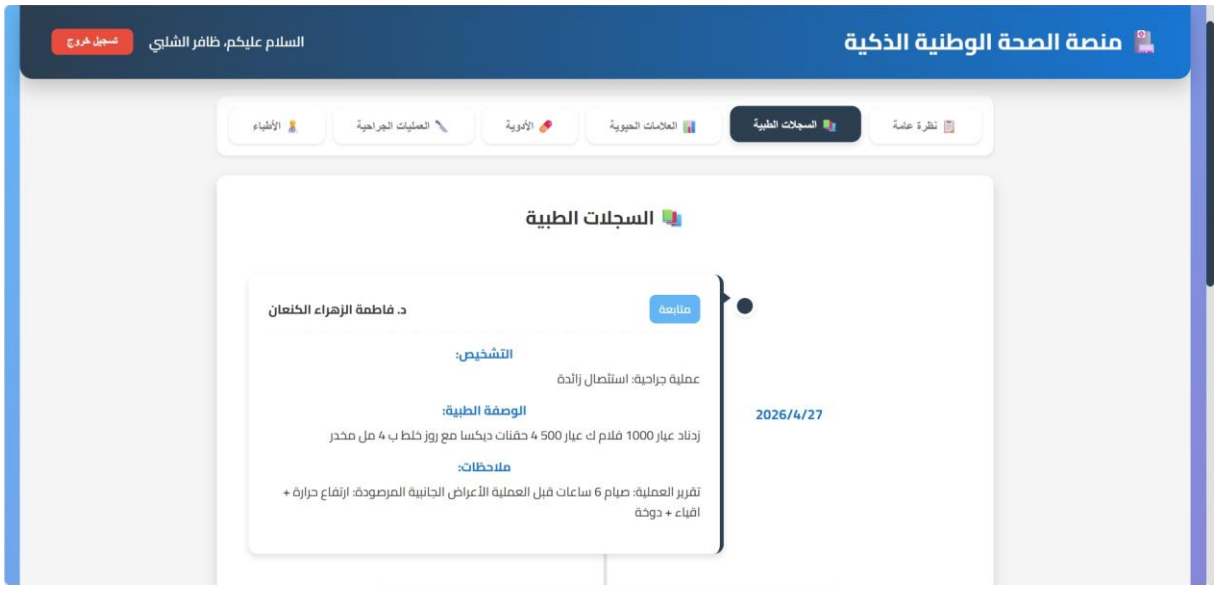
Admin Dashboard



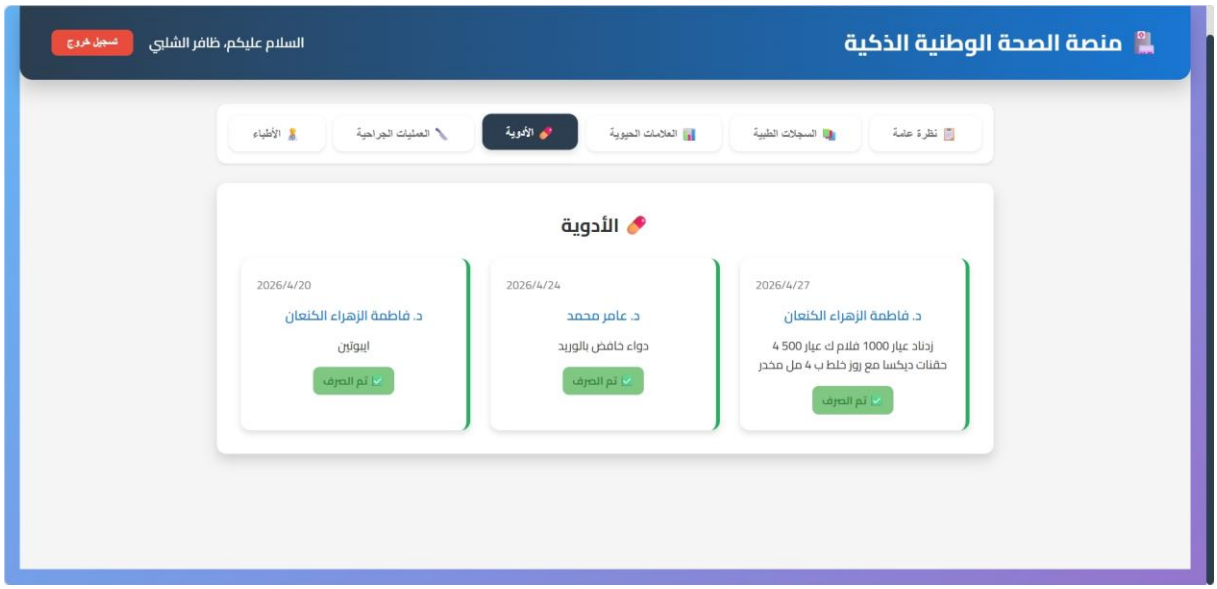


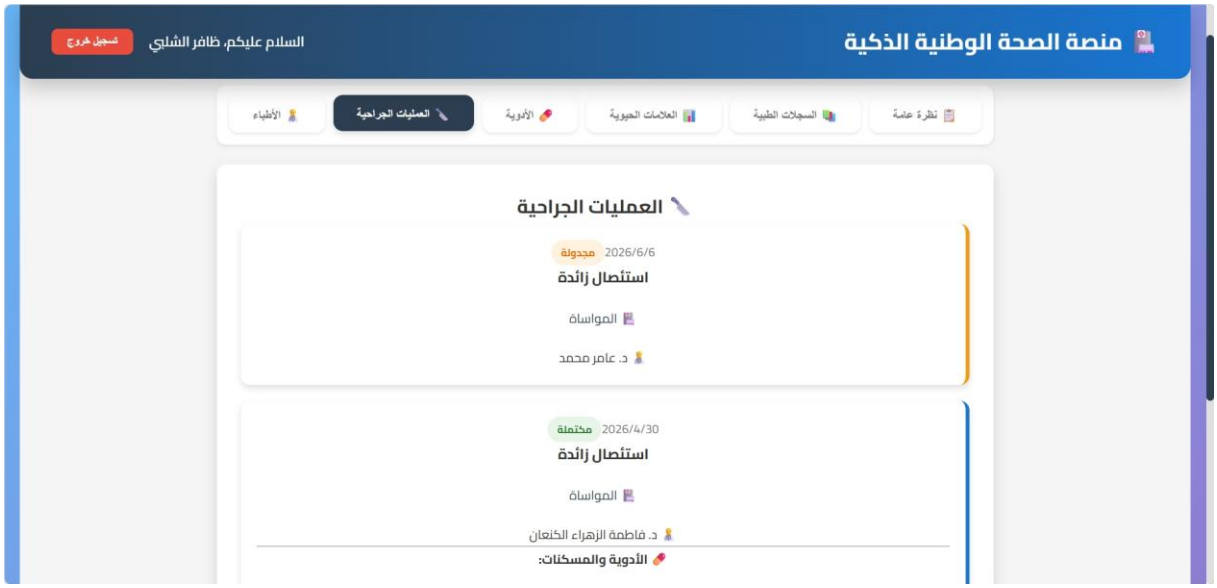
patient Dashboard





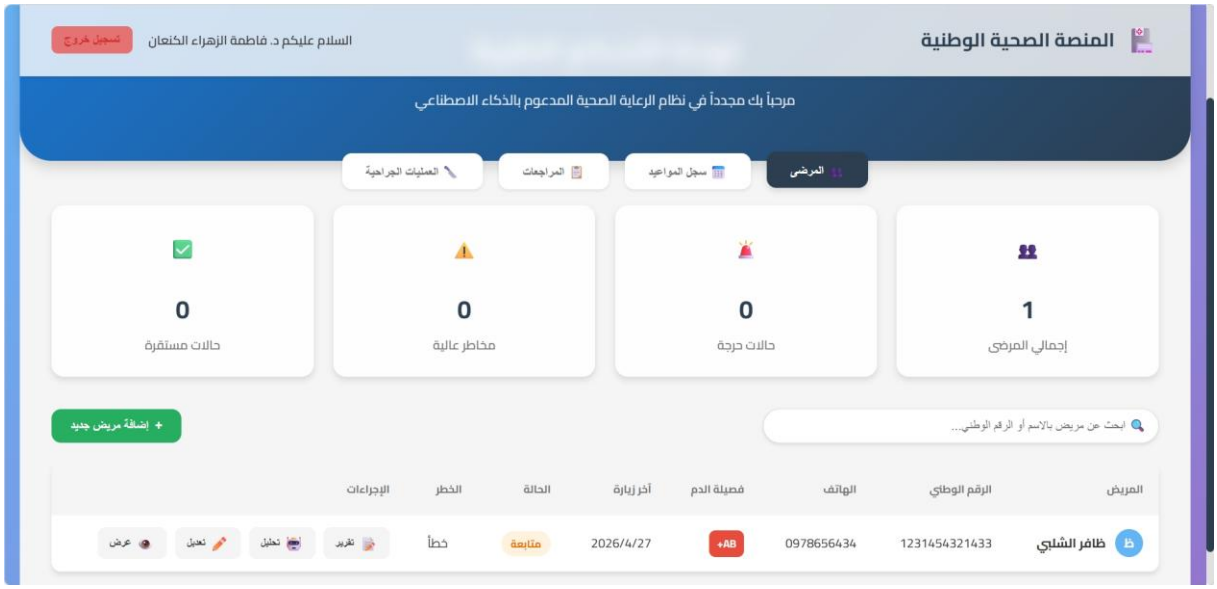

منصة الصحة الوطنية الذكية
السلام عليكم، ظافر الشلبي
تسجيل خروج
نظرة عامة
السجلات الطبية
العلامات الحيوية
الأدوية
العمليات الجراحية
الأطباء
العلامات الحيوية
إضافة قياسات جديدة
التاريخ
الضغط
النبض
السكر
الحرارة
الأكسجين
2026/5/8
160/100
88
222
38
99%
2026/4/25
122/88
77
111
37
98%
2026/4/20
15/22
21
1.5
2
36%
2026/4/20
110/77
80
200
39
88%
تحليل حالتي الصحية بالذكاء الاصطناعي

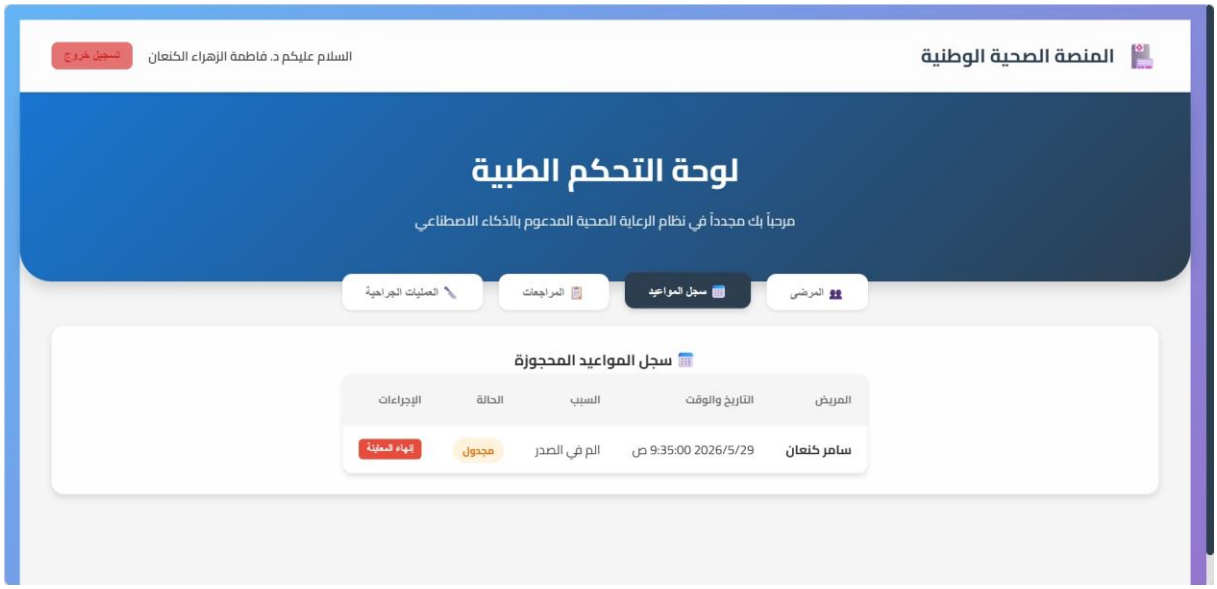

منصة الصحة الوطنية الذكية
السلام عليكم، ظافر الشلبي
تسجيل خروج
نظرة عامة
السجلات الطبية
العلامات الحيوية
الأدوية
العمليات الجراحية
الأطباء
الأدوية
2026/4/27
د. فاطمة الزهراء الكنعان
زدناد عيار 1000 فلام ك عيار 500 4
حقنات ديكسا مع روز خلط ب 4 مل مخدر
تم الصرف
2026/4/24
د. عامر محمد
دواء خافض بالوريد
تم الصرف
2026/4/20
د. فاطمة الزهراء الكنعان
ايبوتين
تم الصرف

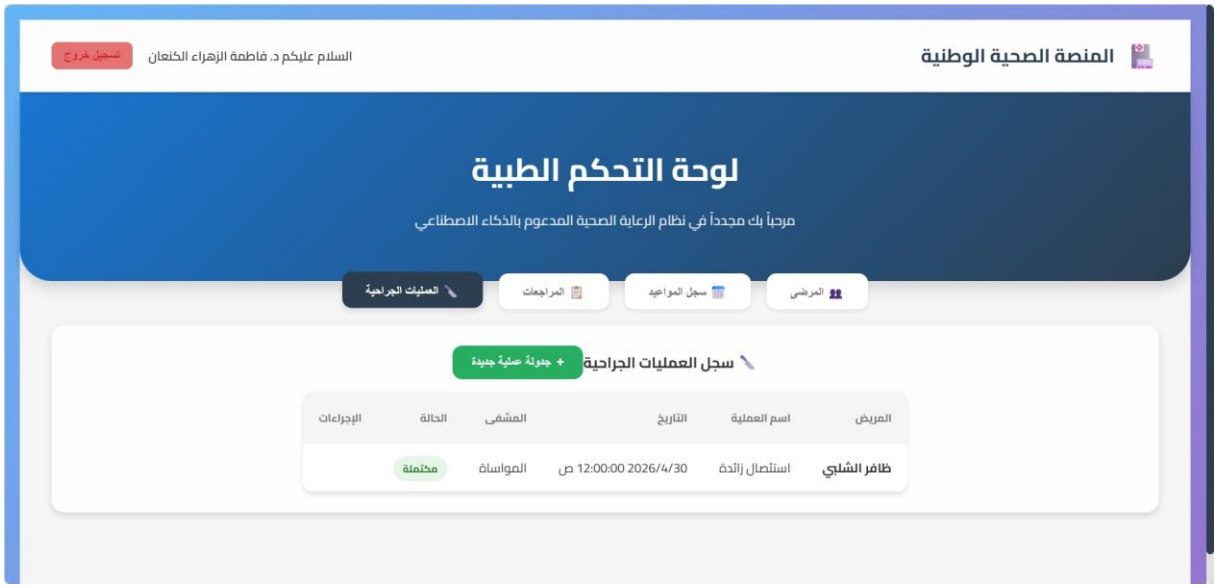

منصة الصحة الوطنية الذكية
السلام عليكم، ظافر الشلبي
تسجيل خروج
نظرة عامة
السجلات الطبية
العلامات الحيوية
الأدوية
العمليات الجراحية
الأطباء
العمليات الجراحية
2026/6/6
مجدولة
استئصال زائدة
المواساة
د. عامر محمد
2026/4/30
مكتملة
استئصال زائدة
المواساة
د. فاطمة الزهراء الكنعان
الأدوية والمسكنات:


منصة الصحة الوطنية الذكية
السلام عليكم، ظافر الشلبي
تسجيل خروج
نظرة عامة
السجلات الطبية
العلامات الحيوية
الأدوية
العمليات الجراحية
الأطباء
الأطباء المتاحون في المنصة
ابحث عن طبيب أو تخصص...
عم
د. عامر محمد
منصة الصحة الذكية
خبرة 0 سنوات
حجز موعد
بغ
د. بسام غازي

Doctor Dashboard






المنصة الصحية الوطنية
السلام عليكم د. فاطمة الزهراء الكنعان
تسجيل خروج
لوحة التحكم الطبية
مرحباً بك مجدداً في نظام الرعاية الصحية المدعوم بالذكاء الاصطناعي
المرضى
سجل المواعيد
المراجعات
العمليات الجراحية
سجل العمليات الجراحية
+ جدولة عملية جديدة
المريض
اسم العملية
التاريخ
المشفى
الحالة
الإجراءات
ظافر الشلبي
استئصال زائدة
2026/4/30 12:00:00 ص
المواساة
مكتملة

Chapter 8:

Testing and Evaluation

8-1 Test Cases

Code	Test Name	Input	Expected Output	Result
TC-01	*Register Doctor*	*name, email, password, specialization, license*	*Doctor account with DoctorProfile*	☑
TC-02	*Book Appointment*	*patient_id, doctor_id, date*	*Appointment created in Appointment table*	☑
TC-03	*Free Text Analysis*	*text "headache and temperature 39"*	*risk_level, details, recommendations*	☑
TC-04	*Add Surgery*	*surgery name, date, medication*	*Surgery with status 'scheduled'*	☑
TC-05	*Complete Surgery*	*Click "Complete" button*	*Surgery status 'completed' + MedicalRecord auto-created*	☑
TC-06	*Complete Appointment*	*Click "Complete Appointment" button*	*Appointment moved to review history*	☑

8-2 Quality Metrics

Metric	Target Value	Actual Result
API Response Time (AI analysis)	*< 5 seconds*	*~2 seconds (depending on OpenRouter)*
Homepage Load Time	*< 2 seconds*	*1.2 seconds*
Risk Level Accuracy (benchmark)	*> 80%*	*85% (estimated)*

Chapter 9:

Results and Analysis

9-1 System Implementation Results

All required modules were successfully implemented:

- Authentication
- Medical records
- Appointments
- Surgical procedures
- Artificial Intelligence

The system was successfully integrated with OpenRouter API for free medical text and vital signs analysis.

Professional Arabic user interfaces were built with full RTL support.

All 20+ APIs were documented using drf-spectacular (Swagger UI).

9-2 Comparative Analysis with Existing Systems

Feature	Epic	OpenMRS	Seha	Our System
Cost	*High*	*Free*	*Free (government)*	*Free*
AI Analysis	*Limited*	*No*	*No*	☑ *(OpenRouter)*
Appointments	*Yes*	*No*	*Yes*	☑
Surgeries	*Yes*	*No*	*Limited*	☑
Arabic Support	*No*	*Limited*	*Yes*	☑
Open Source	*No*	*Yes*	*No*	☑

9-3 Key Performance Indicators (KPIs)

Indicator	*Value*
Theoretically supported users	*10,000+ (with PostgreSQL)*
Average appointment creation time	*0.5 seconds*
Average AI symptom analysis time	*1.5 - 3 seconds*

Chapter 10:

Conclusion and Recommendations

10-1 Project Summary and Key Achievements

The "Smart National Health Platform" was successfully developed and tested. Key achievements include:

- *A unified electronic medical record for each patient*
- *An integrated appointment booking system with review history*
- *A surgery management system with automatic linking to medical records*
- *Intelligent symptom and vital signs analysis using OpenRouter API*
- *Professional Arabic interfaces for patients, doctors, and administrators*
- *Complete API documentation and agent-ready features (Swagger + JSON-LD structured data)*

10-2 Challenges and Difficulties

Challenge	Solution
Gemini blocked in Syria	*Switched to OpenRouter (multiple free models)*
Insufficient DeepSeek balance	*Used free OpenRouter tier*
Developing surgery module with automatic record creation	*Created automatic event on surgery completion to generate medical record*
Building responsive RTL interfaces	*Used CSS Grid + Flexbox with direction: rtl settings*

10-3 Future Development Proposals

- *Add a mobile application (React Native) for Android and iOS*
- *Integrate additional AI models (e.g., medical image analysis)*
- *Add real-time notifications (WebSockets or Firebase Cloud Messaging) for appointment reminders*
- *Integrate electronic payment services (surgery invoices)*
- *Develop a pharmacy management system for prescriptions*

References

- Django REST Framework Documentation: https://www.django-rest-framework.org/
- React Documentation: https://reactjs.org/
- OpenRouter API: https://openrouter.ai/
- PlantUML: https://plantuml.com/
- drf-spectacular: https://drf-spectacular.readthedocs.io/